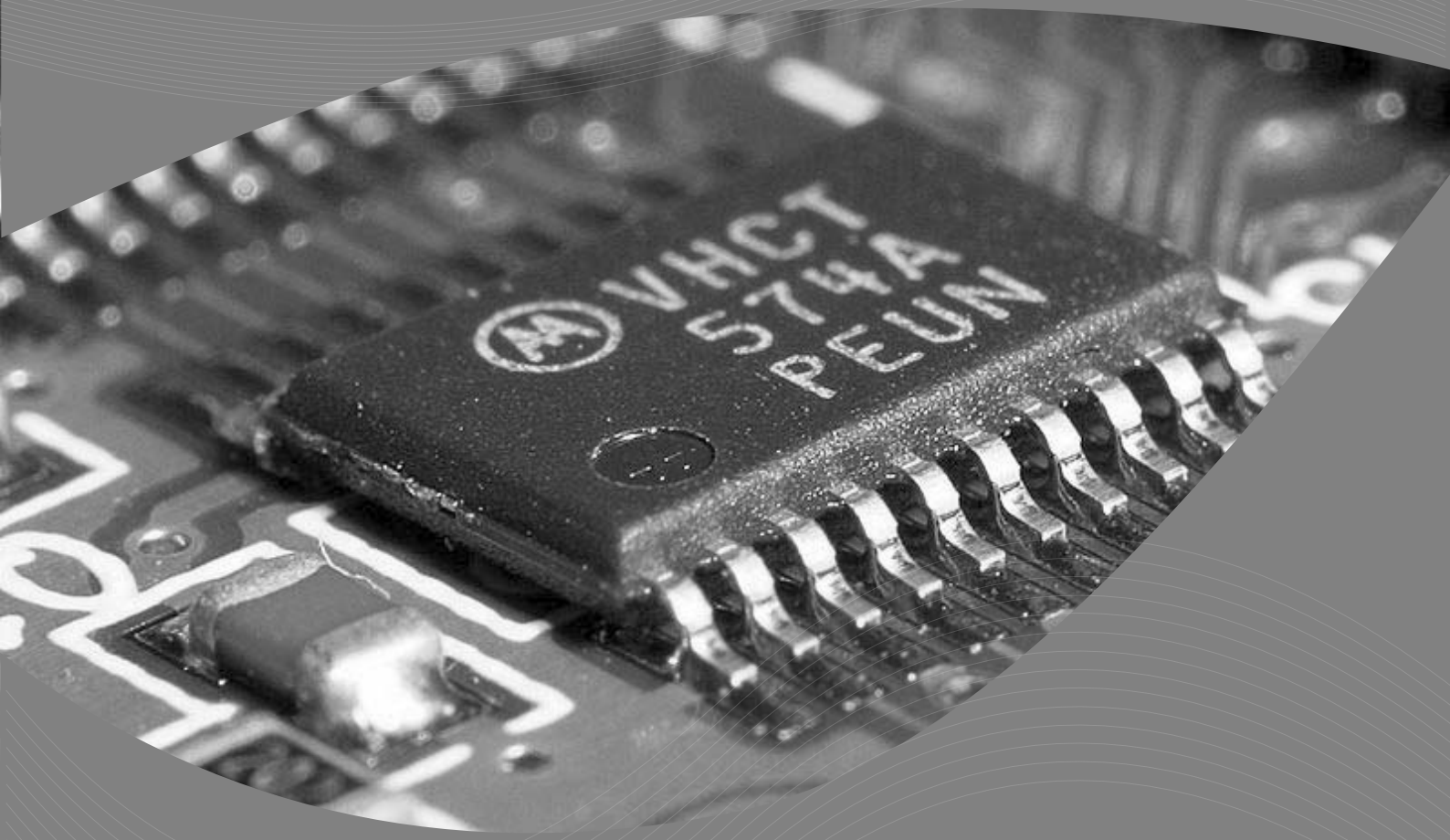
UNIVERSIDADE FEDERAL DO PIAUÍ
CENTRO DE EDUCAÇÃO ABERTA E A DISTÂNCIA

CIRCUITOS DIGITAIS



Autoria
Francisco Vieira de Souza



Módulo II

Circuitos Digitais

Francisco Vieira de Souza

PRESIDENTE DA REPÚBLICA
Luiz Inácio Lula da Silva

MINISTRO DA EDUCAÇÃO
Fernando Haddad

GOVERNADOR DO ESTADO
Wellington Dias

UNIVERSIDADE FEDERAL DO PIAUÍ
Luiz de Sousa Santos Júnior

SECRETÁRIO DE EDUCAÇÃO A DISTÂNCIA DO MEC
Carlos Eduardo Bielschowsky

COORDENADORIA GERAL DA UNIVERSIDADE ABERTA DO BRASIL
Celso Costa

SECRETÁRIO DE EDUCAÇÃO DO ESTADO DO PIAUÍ
Antônio José Medeiro

COORDENADOR GERAL DO CENTRO DE EDUCAÇÃO ABERTA A DISTÂNCIA DA UFPI
Gildásio Guedes Fernandes

SUPERITENDENTE DE EDUCAÇÃO SUPERIOR NO ESTADO
Eliane Mendonça

CENTRO DE CIENCIAS DA NATUREZA
Helder Nunes da Cunha

COORDENADOR DO CURSO DE SISTEMA DE INFORMAÇÃO NA MODALIADE DE EAD
Luiz Cláudio Demes da Mata Sousa

COORDENADORA DE MATERIAL DE DIDÁTICO DO CEAD/UFPI
Cleidinalva Maria Barbosa Oliveira

DIAGRAMAÇÃO
Joaquim Carvalho de Aguiar Neto

S729c Souza, Francisco Vieira de

Circuitos Digitais./Francisco Vieira de Souza. – Teresina: EDUFPI, 2008.

147p.

Inclui bibliografia

1. Sistemas Digitais. 2. Universidade Aberta do Piauí. I. Título.

C.D.D. – 621.381 1

APRESENTAÇÃO

Circuitos digitais são circuitos eletrônicos que baseiam o seu funcionamento na Lógica binária em que toda a informação é guardada e processada sob a forma de zeros e uns. Esta representação é conseguida usando dois níveis discretos de Tensão elétrica.

O objetivo desta Apostila é proporcionar ao estudante do EAD o entendimento de princípios fundamentais, sem exigir que ele decore uma quantidade imensa de detalhes técnicos, normalmente confusos.

Na **Unidade 1**, são analisados os sistemas numéricos e as representações aceitas pelos computadores que as processam e mostram os resultados deste processamento.

Na **Unidade 2,** são analisadas as expressões booleanas, os métodos de simplificação destas expressões e as diversas formas de implementação.

Na **Unidade 3**, serão analisados os circuitos combinacionais, onde os resultados dependem apenas dos valores de suas entradas.

Na **Unidade 4**, são analisados os circuitos seqüenciais, onde as saídas dependem dos valores das entradas e também dos valores que elas detinham anteriormente.

Na **Unidade 5**, são analisadas as memórias, que são os circuitos utilizados no armazenamento de valores para serem manipulados pelos computadores.

SUMÁRIO GERAL

UNIDADE 1 – SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS

1 SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS 9
1.1Introdução 9
1.2Notação posicional 10
1.2.1 Sistemas octais e hexadecimais 12
1.2.2 Conversão de representações numéricas 15
1.3 Soma e subtração binária 18
1.4 Representação de números negativos 21
1.4.1 Representação em forma complementar 22
1.5 SAIBA MAIS 24
1.6 WEB-BIBLIOGRAFIA 25
1.7 REFERÊNCIAS BIBLIOGRÁFICAS 25

UNIDADE 2 – ÁLGEBRA BOOLEANA E CIRCUITOS LÓGICOS

2 Álgebra booleana e circuitos lógicos 28
2.1Introdução 28
2.2Operações básicas da Álgebra booleana 28
2.2.1Operação OU (adição lógica) 28
2.2.2Operação E (multiplicação lógica) 31
2.2.3Complementação (negação ou inversão) 32
2.3Avaliação de expressões booleanas 33
2.4Portas lógicas 35
2.4.1 Portas OR (ou) 36
2.4.2 Portas AND (E) 36
2.4.3 Inversores 37
2.4.4 Exemplo de circuito lógico 37
2.4.5 Propriedades da Álgebra booleana 38
2.4.6 Teoremas de De Morgan 39
2.5Derivação de expressões booleanas 40
2.5.1 Expressões usando Soma de Produtos (SdP) 40
2.5.2 Produtos de somas usando maxtermos (PdS) 42
2.6 Formas canônicas 43
2.7 Circuitos lógicos para formas canônicas 47
2.8 Simplificação de funções booleanas usando mapas de Karnaugh 51
2.8.1 Mapas de Karnaugh e subcubos 52
2.8.2 Cobertura dos mapas de Karnaugh **Erro! Indicador não definido.**

2.9 SAIBA MAIS....64
2.10 WEB-BIBLIOGRAFIA....65
2.11 REFERÊNCIAS BIBLIOGRÁFICAS....65

UNIDADE 3 – CIRCUITOS COMBINACIONAIS

3 Circuitos combinacionais 68
3.1 Introdução 68
3.2 Análise de circuitos combinacionais 69
3.3 Projeto de circuitos combinacionais 70
3.4 Interconexão de Circuitos combinacionais 71
3.4.1 Decodificadores 72
3.4.2 Seletores 75
3.5 Circuitos aritméticos 76
3.5.1 Meio somador e somador completo 77
3.5.2 O somador paralelo 81
3.5.3 O somador/subtrator 84
3.5.4 O multiplicador 85
3.6 SAIBA MAIS....88
3.7 WEB-BIBLIOGRAFIA....88
3.8 REFERÊNCIAS BIBLIOGRÁFICAS....88

UNIDADE 4 – CIRCUITOS SEQÜENCIAIS

4 Circuitos seqüenciais....90
4.1 Introdução 91
4.2 Fundamentação teórica 91
4.3 Latches 96
4.3.1 O *latch* RS 96
4.3.2 O latch RS controlado 102
4.3.3 O *latch* D 104
4.3.4 *Latches* com lógica de ativação complementar106
4.4 Flip-flops 108
4.4.1 *Flip-flop* D mestre–escravo....109
4.4.2 Flip-flops disparados pela borda 111
4.4.3 Flip-flops disparados pela borda descendente 114
4.4.4 *Set* e *reset* assíncronos....115
4.5 SAIBA MAIS....116
4.6 WEB-BIBLIOGRAFIA....117
4.7 REFERÊNCIAS BIBLIOGRÁFICAS....117

UNIDADE 5 – ARMAZENAMENTO DE DADOS

5 ARMAZENAMENTO DE DADOS 120
5.1 Introdução 120
5.2 Registradores 120
5.2.1 Registrador com carga paralela 122
5.2.2 Registradores de deslocamento 123
5.2.3 Registrador de deslocamento com sinal de carga paralela 125
5.2.4 Contador assíncrono 126
5.3 Memórias 128
5.3.1 Memória RAM (Random-Access Memory) 129
5.4 SAIBA MAIS 136
5.5 WEB-BIBLIOGRAFIA 137
5.6 REFERÊNCIAS BIBLIOGRÁFICAS 137

SUMÁRIO

UNIDADE 1 – SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS

1 SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS.....9
1.1Introdução 9
1.2Notação posicional 10
1.2.1 Sistemas octais e hexadecimais.....12
1.2.2 Conversão de representações numéricas 15
1.3 Soma e subtração binária.....18
1.4 Representação de números negativos 21
1.4.1 Representação em forma complementar 22
1.5 SAIBA MAIS 24
1.6 WEB-BIBLIOGRAFIA 25
1.7 REFERÊNCIAS BIBLIOGRÁFICAS 25

Unidade 1

SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS

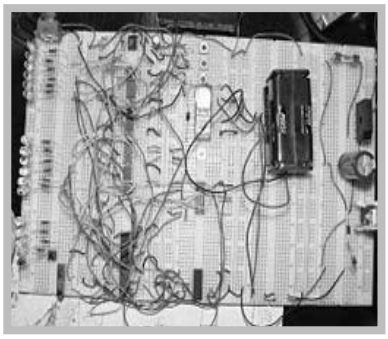
Resumo

O objetivo principal desta unidade é apresentar a maioria dos tipos de dados encontrados nos sistemas digitais, mostrando como eles são representados em sua forma binária, ou seja, usando apenas os dígitos 0 e 1. Os dados encontrados nos sistemas digitais podem ser classificados em três categorias: os números: os usados na computação aritmética, as letras do alfabeto e uma variedade de símbolos discretos usados para uma variedade de propósitos. Todos estes três tipos de dados são representados em um computador em forma binária porque é fácil construir circuitos eletrônicos que exibam duas condições alternativas interpretadas pelos valores 0 e 1 de um dígito binário.

Apesar de toda informação poder ser representada desta forma, nem sempre ela é adequada para usuários humanos. Neste caso, a representação binária deve ser convertida para uma representação decimal, onde estão presentes os dígitos 0,1, ..., 9 e as letras do alfabeto.

1 SISTEMAS DE REPRESENTAÇÕES NUMÉRICAS

1.1 – Introdução



Ao longo do tempo, o homem tem criado diversos sistemas numéricos, de acordo com suas necessidades. O sistema numérico mais conhecido é o sistema decimal que é baseado em 10 símbolos, chamados dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9 para representar um conjunto com até 9 elementos. Para representar um conjunto com 10 elementos, não podemos mais proceder da mesma maneira. Neste caso, tomamos o dígito 1 e ao seu lado direito adicionamos um novo dígito da base, no caso o 0, para indicar a quantidade de elementos que passam de 10. Este processo é similar para os outros sistemas, onde as diferenças se verificam apenas nos elementos da base. Por exemplo, o sistema de base 8 (pouco utilizado atualmente) tem apenas os dígitos 0, 1, 2, 3, 4, 5, 6 e 7. Dois outros sistemas muito utilizados são o sistema binário, onde sua base tem apenas dois elementos, 0 e 1 e o sistema hexadecimal, onde sua base tem 16 elementos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E e F. Note que para representar o valor 10 usamos a letra Ä, para representar o 11 usamos a letra B, etc. até F que representa o número 15.

Nesta unidade, serão estudadas as formas como os dados são representados, destacando a representação binária. Os dados encontrados nos sistemas digitais podem ser classificados em uma das seguintes categorias:

- Números usados em cálculos aritméticos;
- Letras do alfabeto, usadas no processamento de dados;
- Símbolos discretos, usados para diversos propósitos

Todos os dados são representados em formato binário porque este formato facilita o projeto de circuitos eletrônicos que exibem duas condições possíveis, que podem ser convenientemente interpretadas pelos valores 0 e 1 de um dígito binário (***bit*** = ***bi***nary digi***t***).

1.2 – Notação posicional

Quando é que 111 é igual a 7?

Todos os sistemas numéricos utilizados pelo ser humano são posicionais. Em um sistema posicional, cada dígito possui um peso associado. É o que é conhecido como valor relativo de cada dígito que compõe o número. Desta forma, 456 = 400 + 50 + 6, ou seja, o valor relativo do dígito 4, neste número, é 400, do dígito 5 é 50 e do dígito 6 é 6 mesmo. Assim, o valor de um dado número corresponde a uma soma ponderada de seus dígitos, como por exemplo:

$$1958 = 1.*1000 + 9.*100 + 5.*10 + 8.*1$$

Note que neste número o peso de cada posição é 10^i, onde i corresponde à posição do dígito, contada a partir da direita, e sendo i=0. para o dígito mais à direita. Em geral, um número decimal D no formato $d_1 d_0 . d_{-1} d_{-2}$ tem como valor:

$$D = d_1.*10^1 + d_0.*10^0 + d_{-1}.*10^{-1} + d_{-2}*10^{-2}$$

onde 10 é a base da representação. Num sistema posicional genérico, a base pode ser qualquer valor inteiro r, e um dígito numa posição i assume um peso r^i. Logo, podemos escrever o formato genérico de um número em tal sistema como sendo:

$$d_{m-1}\ d_{m-2} \ldots d_1\ d_0\ .\ d_{-1}\ d_{-2} \ldots d_{-n}$$

onde há m dígitos à esquerda do ponto e n dígitos à direita do ponto. Note que, se não houver ponto, assume-se que este está à direita do dígito mais à direita. O valor decimal deste número é o somatório dos produtos de cada dígito pela correspondente potência da base:

$$D = \sum_{i=-n}^{m-1} d_i \times r^i$$

Para um número qualquer, o dígito mais à direita é referenciado como dígito menos significativo, ao passo que o dígito mais à esquerda é o dígito mais significativo.

Estamos particularmente interessados no sistema binário, onde a base é 2. Um número binário assume a forma:

$$b_{m-1}\, b_{m-2} \ldots b_1\, b_0 \,.\, b_{-1}\, b_{-2} \ldots b_{-n}$$

e seu valor pode ser calculado pelo somatório a seguir:

$$B = \sum_{i=-n}^{m-1} b_i \times 2^i$$

O ponto no sistema binário é denominado ponto binário, como no sistema decimal ele é chamado de ponto decimal. Normalmente, quando se trabalha com sistemas de base não-decimal, indica-se a base subscrevendo-se o valor da base à direita do número. Exemplos:

$11001_2 = 1*2^4 + 1*2^3 + 0*2^2 + 0*2^1 + 1*2^0$

$= 1*16 + 1*8 + 0*4 + 0*2 + 1*1 = 25_{10}$

$.1101_2 = 1*2^{-1} + 1*2^{-2} + 0*2^{-3} + 1*2^{-4}$

$= 1*0.50 + 1*0.25 + 0*0.125 + 1*0625 = 0.8125_{10}$

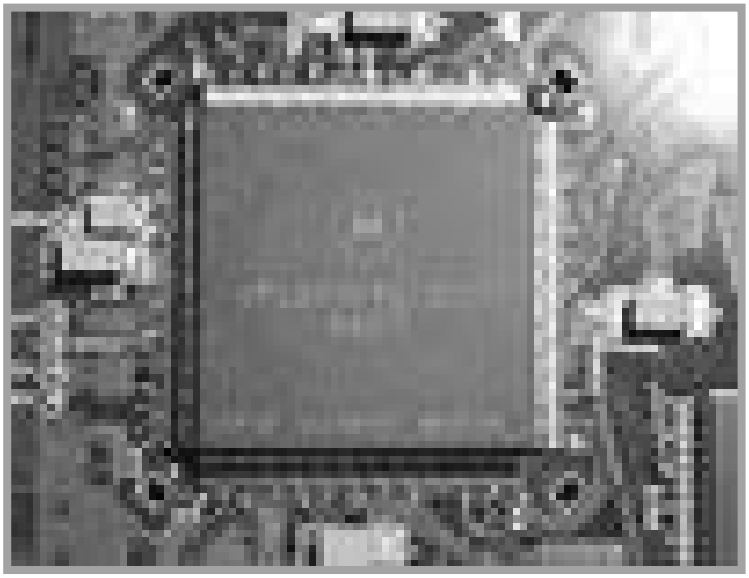
Para se encontrar o valor decimal de um número escrito em uma base b qualquer, diferente de 2, o processo é o mesmo, só trocando os pesos que devem ser potências de b. Por exemplo, para encontrar o valor decimal do número 2341_5 deve-se proceder da seguinte forma:

2341_5

$= 2x5^3 + 3x5^2 + 4x5^1 + 1x5^0 = 250 + 75 + 20 + 1 = 346_{10}$

1.2.1 – Sistemas octais e hexadecimais

Já foi visto que no sistema octal, cada dígito representa um valor entre 0 e 7. Já no sistema hexadecimal, cada dígito representa um valor entre 0 e 15. Neste sistema, para representar os valores de 10 a 15 com apenas um dígito, utilizam-se as letras romanas maiúsculas A até F.



Teoricamente, poder-se-ia construir qualquer sistema numérico e é provável que isto ocorra em um futuro não muito distante, dado o desenvolvimento rápido de novas tecnologias e da construção de computadores cada vez mais potentes. No entanto isto fica para o futuro e no momento os sistemas mais utilizados são o sistema binário e o hexadecimal. Por este motivo, é necessário entender como um número pode ser representado em vários sistemas. Isto significa que é importante saber como o mesmo número pode ser representado nos vários sistemas numéricos. A este processo chamamos de mudança de base ou conversão de um número em uma base para outra.

Uma observação importante é que são 8 dígitos usados para representar números *octais* e são necessários 4 bits para representar qualquer um dos 16 dígitos hexadecimais. Isto significa que para representar qualquer número entre 0 e 7 são necessários 3 *bits* e para representar qualquer número entre 0 e 15 são necessários 4 *bits*.

A Tabela 1.1 mostra os inteiros decimais de 0 a 17, juntamente com seus equivalentes em binário, octal e hexadecimal. A Tabela mostra também a lei de formação dos números nos diversos sistemas numéricos.

Tabela 1.1 - Representação de alguns números

Decimal	Binário	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	110	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11

Já a Tabela 1.2 mostra os inteiros de 0 a 17 com as representações binárias de seus equivalentes no sistema *octal*, decimal e hexadecimal. Deve ser observado que as representações octal e hexadecimal estão separadas para evidenciar a representação de cada dígito.

Tabela 1.2 - Representação de inteiros em binário, octal, decimal e hexadecimal.

Binário	Octal codificado em binário	Decimal codificado em binário	Hexadecimal codificado em binário
0	000	0000	0000
1	001	0001	0001
10	010	0010	0010
11	011	0011	0011
100	100	0100	0100
101	101	0101	0101
110	110	0110	0110
111	111	0111	0111
1000	001 000	1000	1000
1001	001 001	1001	1001
1010	001 010	0001 0000	1010
1011	001 011	0001 0001	1011
1100	001 100	0001 0010	1100
1101	001 101	0001 0011	1101
1110	001 110	0001 0100	1110
1111	001 111	0001 0101	1111
10000	010 000	0001 0110	10000
10001	010 001	0001 0111	10001

Para se converter um número binário em *octal* separam-se os dígitos em grupos de 3, a partir do ponto binário para a esquerda, substituindo-se cada grupo pelo dígito *octal* correspondente. Por exemplo:

$$10011100101_2 = 010\ 011\ 100\ 101_2 = 2345_8$$

Para converter-se um número em binário para hexadecimal, o procedimento é análogo, exceto que os grupos deverão ser de 4 dígitos.

Exemplo 1.1. Converter o número binário 1010011100_2 para hexadecimal

$$11010111100_2 = 0110\ 1011\ 1100_{\ 2} = 6DC_{16}$$

Note que nestes exemplos foram adicionados zeros à esquerda, para que todos os grupos tivessem 3 dígitos, no caso da conversão direta binário-octal, e 4 dígitos, no caso da conversão direta binário-hexadecimal.

A conversão no sentido oposto também é bastante simples. Substitui-se cada dígito octal ou hexadecimal pelo conjunto de 3 ou 4 dígitos binários que o representa.

Exemplo 1.2. Converter os números que seguem para binário.

$$171_8 = 001\ 111\ 001_2 = 1111001_2$$

$$FAB_{16} = 1111\ \ 1010\ 1011_2 = 11110101011_2$$

1.2.2 – Conversão geral de representações numéricas

Nem sempre é possível se converter um número representado numa determinada base para outra base simplesmente substituindo-se dígitos de uma base por seus equivalentes na outra. Isto só é possível nos casos em que as bases são potências de um mesmo número (como os casos mostrados anteriormente). Quando não é este o caso, será necessário utilizar-se um procedimento geral que pode ser empregado em qualquer mudança de representação.

1.2.2.1 – Números inteiros

O processo prático de converter um número inteiro de uma base para outra consiste em fazer a divisão inteira deste número pela base encontrando um quociente e um resto. Este resto é guardado e será o dígito menos significativo da nova representação. Continua-se o processo usando agora dividendo o quociente obtido na divisão anterior. Este processo continua até que o quociente seja menor que a base. Neste caso, o quociente será o dígito mais significativo.

Exemplo 1.3. Converter o número 30_{10} para a base 2. Vamos mostrar como isto é feito, utilizando todos os passos.

O processo consiste na divisão inteira do número a ser convertido pela base. Neste caso, a divisão inteira de 30 por 2. Neste caso, o resto é 0, ou seja, 30 = 15 x 2 + 0. Ou seja, 30 = 15 x 2^1 + 0 x 2^0. O quociente 15, por sua vez, pode também ser reescrito e a expressão acima se torna:

$30 = (7 \times 2 + 1) \times 2^1 + 0 \times 2^0 = 7 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

$= (3 \times 2 + 1) \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

$= 3 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

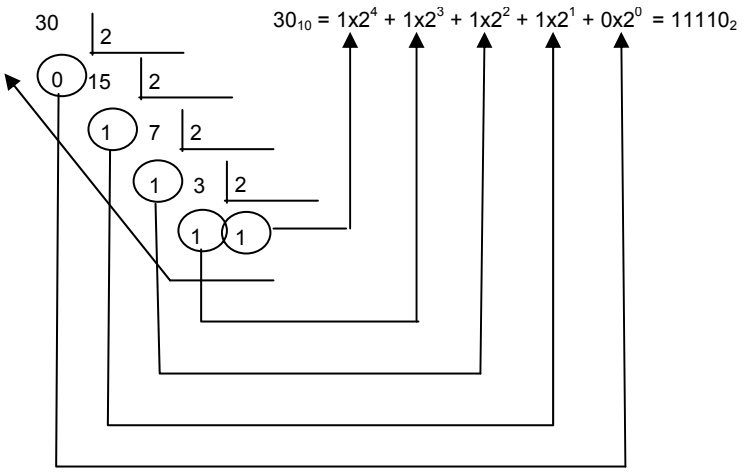
$= (1 \times 2 + 1) \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

$= 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 11110_2$

Esta nova representação para o número 30 nos permite fazer duas observações:

- O número 30 em decimal foi reescrito em uma representação que é uma soma de parcelas onde cada uma delas é o produto de 1 ou 0 por uma potência de 2 que é a base para a qual desejamos mudar a representação;
- Os números, 0 ou 1 das somas parciais são os restos das divisões sucessivas realizadas, em ordem inversa.

Podemos mostrar graficamente este processo, da seguinte forma:



Exercício. Converter o número 74_{10} para a base 2 e verifique se o resultado está correto fazendo a conversão inversa, ou seja da base 2 para a base 10 e veja se o resultado está correto.

1.2.2.2 – Números fracionários

Também é possível converter números decimais fracionários em binários. Vamos explicar isto com um exemplo.

Exemplo 1.4. Vamos converter o número decimal $23{,}375_{10}$ para a base 2.

O número 23,375 = 23 + 0,375. A conversão do número 23, inteiro, já sabemos realizar. $23_{10} = 10111_2$. Vamos nos dedicar à fração 0,375. O processo prático consiste na multiplicação por 2, separando a parte inteira até que o produto seja 0 ou que cheguemos a uma fração já encontrada..

Neste caso, 0,375x2=0,750; separamos o 0 que é o primeiro dígito e continuamos o processo. 0,750x2=1,500; separamos o 1 e continuamos. 0,500x2=1,000. Separamos o 1 e,

neste ponto, o processo termina porque a parte fracionária encontrada é nula.

Assim $23{,}375_{10} = 10111{,}011_2$

Exercício. Converter o número $345{,}146_{10}$ para a base 2 e verifique se esta conversão está correta.

1.3 – Soma e subtração binária

O procedimento para a adição e subtração de números binários é semelhante ao que se usa para números decimais. A diferença está nas tabelas de adição e subtração, que contêm apenas os dígitos 0 e 1.

Para somar dois números decimais, faz-se a soma de um par de dígitos de cada vez, começando com o dígito menos significativo de cada número. Para a adição usa-se a tabela verdade a seguir:

Tabela 1.3 – Tabela da adição.

X	**Y**	**S = X + Y**	***carry***
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Se a soma de um dado par de dígitos for igual ou maior que 10, o excesso (também chamado de transporte ou *carry*) é usado na soma do próximo par de dígitos mais significativos. Para somar dois números binários, usa-se um *carry* inicial c_0 igual a 0. A Tabela 1.4 mostra a soma s_i e o *bit* de *carry* c_i+1 para cada possível combinação de x_i, y_i e c_i.

Tabela 1.4 - Adição de números binários.

$x_i + y_i + c_i$	s_i	c_i+1
000	0	0
001	1	0
010	1	0
011	0	1
100	1	0
101	0	1
110	0	1
111	1	1

Exemplo 1.3. Somar os números binários 987 e 123.

		512	256	128	64	32	16	8	4	2	1
x		1	1	1	1	0	1	1	0	1	1
y					1	1	1	1	0	1	1
carries	1	1	1	1	1	1	1	0	1	1	
x + *y*	1	0	0	0	1	0	1	0	1	1	0
	S_{10}	S_9	S_8	S_7	S_6	S_5	S_4	S_3	S_2	S_1	S_0

Primeiro soma-se $x_0 = 1$ com $y_0 = 1$, produzindo *carry* $c_1 = 1$ e soma $s_0 = 0$. Em seguida, soma-se $x_1 = 1$, $y_1 = 1$ e $c_1 = 1$, obtendo-se *carry* $c_2 = 1$ e soma $s_1 = 1$. Este processo continua até se gerar $c_{11} = 1$ e *carry* $s_{10} = 0$.

A subtração de números binários é realizada de maneira semelhante, subtraindo-se um par de bits a cada vez, no entanto, é sempre gerado um bit de empréstimo (*borrow)* e não um *bit* de *carry*, e um *bit* de diferença ao invés de um *bit* de soma.

A Tabela 1.4 mostra a operação de subtração entre dois números, gerando um bit de empréstimo.

Tabela 1.4 – Operação de subtração.

X	Y	D = X - Y	*borrow*
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

A Tabela 1.5 mostra a diferença d_i e o *bit* de *borrow* b_i+1 para cada possível combinação de x_i, y_i e b_i.

Tabela 1.5 - Subtração de números binários.

xi - yi – bi	di	bi+1
000	0	0
001	1	1
010	1	1
011	0	1
100	1	0
101	0	0
110	0	0
111	1	1

O procedimento para subtração é semelhante ao de adição: a partir dos dígitos menos significativos, geram-se os bits de *borrow* b_1 e de diferença d_0, de acordo com a Tabela 1.5, e assim por diante, da direita para a esquerda, até o *bit* de *borrow* mais significativo b_m e o *bit* de diferença mais significativo d_{m-1}.

Exemplo 1.4. Subtrair o número binário 123 de 987.

	512	256	128	64	32	16	8	4	2	1
x	1	1	1	1	0	1	1	0	1	1
y				1	1	1	1	0	1	1
borrows	0	0	1	1	0	0	0	0	0	
x - *y*	1	1	0	1	1	0	0	0	0	0
	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	d_0

Primeiro faz-se a subtração entre $x_0 = 1$ e $y_0 = 1$, produzindo *borrow* $b_1 = 0$ e diferença $d_0 = 0$. Em seguida, faz-se a subtração de $y_1 = 1$, $b_1 = 1$ de $x_1 = 1$, obtendo-se *borrow* $b_2 = 0$ e soma $s_1 = 0$. Este processo continua até se gerar $d_9 = 1$.

1.4 – Representação de números em sinal magnitude

Números binários positivos podem ser representados como números sem sinal. No entanto, para se representarem números negativos, é necessária a utilização de alguma notação. A forma de representação mais simples é sinal-magnitude. Nesta representação, o número consiste de duas partes: a magnitude e o sinal. A magnitude expressa a quantidade e o sinal indica se a quantidade é positiva ou negativa.

Dado um número binário N, sua representação em sinal magnitude é $N = sa_{n-1}a_{n-2}...a_0.a_{-1}a_{-2}...a_{-m}$ onde

s = 0 se N for positivo e

s = 1 se N for negativo

Quando sinal-magnitude é usado para números binários, o sinal é representado pelo dígito mais significativo: “0’” indica sinal positivo e “1” indica sinal negativo.Assim, os números +9 e –9 escritos em binário se diferenciam somente pelo bit mais significativo:

+9 = 01001
-9 = 11001

Note que foram necessários 5 bits para representar esses números: 4 para a magnitude e 1 (o mais da esquerda) para representar o sinal.

A representação em sinal magnitude é importante porque permite uma operação de subtração, por exemplo, seja transformada em uma operação de adição. Isto significa que o

mesmo circuito pode ser utilizado para ambas operações. Isto implica em diminuição de hardware, portanto em economia.

Para verificarmos isto devemos antes definir o que significa complemento de um número em uma base r.

1.4.1 – Representação em forma complementar

A representação em complemento foi criada com o intuito de simplificar a operação de subtração e manipulações lógicas, evitando a necessidade de comparações de magnitude e sinal.

Definição. Seja N um número em uma base r. Define-se o complemento de r do número N e denota-se por $[N]_r = r^n - N_r$, onde n é o número de dígitos de N.

Decorre, a partir desta definição, que uma subtração entre dois números $A_r - B_r = A_r - B_r + r^n - r^n = A_r + (r^n - B_r) - r^n = A_r + [B]_r - r^n$.

O complemento de r de um número binário é chamado ***complemento de dois*** e a representação dele derivada é chamada representação em ***complemento de dois***.

No sistema binário, temos $[N]_2 = 2^n - N$. Mas esta subtração é entre uma potência de 2 e o número N. Esta subtração é feita de forma otimizada. uma vez que isto pode ser conseguido apenas trocando os bits 0 por 1 e os bits 1 por 0 e somando-se o resultado com o valor 1.

Por exemplo, sendo $N = 10111010100_2$ então $[N]_2 = 2^{11} - (10111010100)_2 = 100000000000_2 - (10111010100)_2 = (001000101100)_2$ que é o mesmo valor obtido pela troca dos bits 0 por 1 e vice-versa, somando-se o resultado com 1, ou seja, $01000101011_2 + 1$ que é $(001000101100)_2$.

Assim temos: $A_2 - B_2 = A_2 + [B]_2 - r^n$. Isto significa que uma subtração entre dois números foi transformada em uma

soma com o complemento (fácil de ser encontrado) e uma outra subtração entre um número e uma potência da base do sistema. Esta subtração também é feita de forma facilitada.

No sistema decimal, o complemento do número 74 é 100 – 74 = 26. O complemento de 590 é 410, ou seja, é o que falta para completar a próxima potência da base.

O algoritmo prático para encontrar $N_2 = A_2 – B_2$ é o seguinte:

adicione $A_2 + [B]_2$,

se houver carry bit na (n + 1)-ésima posição então

então $N = A_2 + [B]_2$ /* despreza-se o *carry bit* */

senão $N = -([A_2 + [B]_2]_2)$

Exemplo 1. Sejam $A = 22_2$ e $B = 11_2$. Faça a subtração A – B.

$A – B = (10110)_2 – (01011)_2$ (n = 5)

```
  10110
 +10101
 ---------
 101011
```

Neste caso, houve *carry bit*, logo despreza-se e o resultado é $(01011)_2 = 11_{10}$

Exemplo 2. Faça a subtração $7_{10} – 118_{10}$.

$A – B = (0000111)_2 – (1110110)_2$ (n = 7)

```
   0000111
 + 0001010
 --------------
   0010001          /*  não houve carry bit  */
```

Neste caso, toma-se o complemento deste valor e assim fica

$A – B = -(1101111)_2 = 111_{10}$

EXERCÍCIOS

Exercício 1.1 - Realizar as conversões indicadas a seguir.

a) 179 para binário
b) 467 para *octal*
c) 3417 para hexadecimal

Exercício 1.2 - Encontrar a representação em complemento de r para os exemplos anteriores.

Exercício 1.3 - Encontrar, para os exemplos anteriores, a representação em complemento de 2 usando operações de complemento.

1011001_2:

0001111_2:

1.5 SAIBA MAIS

Existem muitos bons textos e alguns deles estão listados na Bibliografia colocada ao final da Unidade 2. Outros estão na Internet à disposição . Estes estão listados a seguir.

1.6 WEB-BIBLIOGRAFIA

www.ufpi.br/uapi

(A Página da Universidade Aberta do Piauí - UAPI)

www.uab.gov.br

(O Site da Universidade Aberta do Brasil- UAB)

www.seed.mec.gov.br

(A Homepage da Secretaria de Educação a Distância do MEC - SEED)

www.abed.org.br

(O site da Associação Brasileira de Educação a Distância - ABED)

1.7 REFERÊNCIAS BIBLIOGRÁFICAS

GAJSKI, Daniel D. Principles of Digital Design, New Jersey:

Prentice Hall, 1997 (ISBN 0-13-301144-5).

MANO, M. Morris; Computer Engineering: Hardware Design.

New Jersey: Prentice Hall, 1988 (ISBN 0-13-162926-3)

IDOETA, Ivan V. et CAPUANO, Francisco G. Elementos de Eletrônica Digital. 40ª. Edição. Editora Érica Ltda. São Paulo, 2008.

ERCEGOVAC, Milos; LANG, Tomás; MORENO, Jaime H. Introdução aos Sistemas Digitais. Porto Alegre: Bookman, 2002 (ISBN: 85-7307-698-4)

UYEMURA, John. Sistemas digitais: Uma Abordagem Integrada. Pioneira Thompson Learning Ltda. 2002.

Unidade 2

ÁLGEBRA BOOLEANA E CIRCUITOS LÓGICOS

Resumo

O objetivo principal desta unidade é apresentar os fundamentos dos circuitos digitais. Eles são baseados na Álgebra de Boole, um tema que já deve ser conhecido por quem deseja entender este estudo.
Serão vistas as portas lógicas como os elementos principais para a construção destes circuitos. Serão estudadas as diversas formas utilizadas nas simplificações de expressões booleanas, em busca de economia na construção de circuitos.
A forma de apresentação utilizada é de acordo com o exigido para o ensino à distância, ou seja, tendo em vista sempre esta nova modalidade de ensino.

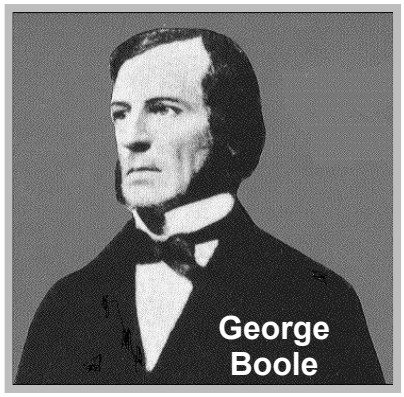
SUMÁRIO

UNIDADE 2 – ÁLGEBRA BOOLEANA E CIRCUITOS LÓGICOS

2 Álgebra booleana e circuitos lógicos 28
2.1 Introdução 28
2.2 Operações básicas da Álgebra booleana 28
2.2.1 Operação OU (adição lógica) 28
2.2.2 Operação E (multiplicação lógica) 31
2.2.3 Complementação (negação ou inversão) 32
2.3 Avaliação de expressões booleanas 33
2.4 Portas lógicas 35
2.4.1 Portas OR (ou) 36
2.4.2 Portas AND (E) 36
2.4.3 Inversores 37
2.4.4 Exemplo de circuito lógico 37
2.4.5 Propriedades da Álgebra booleana 38
2.4.6 Teoremas de De Morgan 39
2.5 Derivação de expressões booleanas 40
2.5.1 Expressões usando Soma de Produtos (SdP) 40
2.5.2 Produtos de somas usando maxtermos (PdS) 42
2.6 Formas canônicas 43
2.7 Circuitos lógicos para formas canônicas 47
2.8 Simplificação de funções booleanas usando mapas de Karnaugh 51
2.8.1 Mapas de Karnaugh e subcubos . **Erro! Indicador não definido.**
2.8.2 Cobertura dos mapas de Karnaugh ... **Erro! Indicador não definido.**
2.9 SAIBA MAIS 64
2.10 WEB-BIBLIOGRAFIA 65
2.11 REFERÊNCIAS BIBLIOGRÁFICAS 65

2 Álgebra booleana e circuitos lógicos

2.1 – Introdução

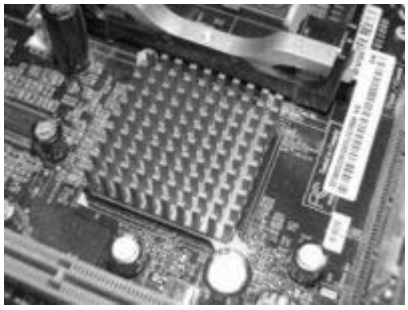


Em 1854, George Boole introduziu o formalismo que até hoje se usa para o tratamento sistemático da lógica, que é a chamada Álgebra Booleana. Em 1938, C. E. Shannon aplicou esta álgebra para mostrar que as propriedades de circuitos elétricos de chaveamento podem ser representadas por uma Álgebra Booleana com dois valores.

Uma Álgebra Booleana é definida com um conjunto de operadores e um conjunto de axiomas, que são assumidos Diferentemente da Álgebra ordinária dos reais, onde as variáveis podem assumir valores no intervalo (-∞;+∞), as variáveis Booleanas só podem assumir dois valores, que podem ser denotados de várias formas. Entre elas, {F,V} ou {0,1}. Nesta Apostila, será adotada a notação {0,1} por sua grande utilização na Eletrônica Digital. As funções booleanas podem ser representadas por tabelas que recebem o nome de tabelas verdade, onde são listadas todas as combinações de valores que as variáveis de entrada podem assumir e os correspondentes valores de saídas da função.

2.2 – Operações básicas da Álgebra booleana

Na Álgebra Booleana, existem três operações ou funções básicas. São elas: operação OU, operação E e complementação. Todas as outras funções booleanas podem ser representadas em termos destas operações básicas.



2.2.1 – Operação OU (adição lógica)

Uma operação OU entre dois operandos booleanos resulta o valor 1 se pelo menos um destes dois operandos tiver o valor 1. Se os dois valores dos operandos for 0, o resultado é também 0.

Um símbolo utilizado para representar a operação OU é “+”, como o símbolo da adição algébrica (dos reais). Porém, sabemos que não se trata da adição algébrica, mas sim da adição lógica. Outro símbolo também encontrado na bibliografia é “v”.

Listando as possibilidades de combinações entre dois valores Booleanos e os respectivos resultados para a operação OU, tem-se:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 1

Note que a operação OU é definida para duas variáveis, ou seja, a operação exige que existam dois operandos. A isto chama-se “aridade da função”, que é a quantidade de operandos que ela exige para ser realizada. Devido a isso, o operador “+” (OU) é dito binário. A tabela verdade para esta operação é a seguinte:

A	B	A + B
0	**0**	**0**
0	**1**	**1**
1	**0**	**1**
1	**1**	**1**

A tabela verdade pode ser também utilizada para representar uma expressão booleana envolvendo mais de duas variáveis Por exemplo, para representar a operação A+B+C (A ou B ou C). Neste caso, o resultado será 1 se pelo menos uma das variáveis de entrada valer 1.

A	B	C	A+B+C
0	**0**	**0**	**0**
0	**0**	**1**	**1**
0	**1**	**0**	**1**
0	**1**	**1**	**1**
1	**0**	**0**	**1**
1	**0**	**1**	**1**
1	**1**	**0**	**1**
1	**1**	**1**	**1**

É importante notar que, devido ao fato de haver somente um operador na equação, pode-se também avaliar a equação decompondo-a em pares. Por exemplo, pode-se primeiramente achar o resultado de A+B, para depois operar os valores resultantes com os respectivos valores de C. Esta propriedade é conhecida como associativa. Também a ordem em que são avaliadas as variáveis A, B e C é irrelevante (propriedade comutativa). Estas propriedades estão ilustradas na tabela verdade a seguir. Nela, os parêntesis indicam sub-expressões já avaliadas anteriores. Note que os valores das colunas referentes às expressões A+B+C, (A+B)+C e (B+C)+A são os mesmos.

A	B	C	A+B+C	A+B	(A+B)+C	B+C	A+(B+C)
0	0	0	0	0	0	0	0
0	0	1	1	0	1	1	1
0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	1	1
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

2.2.2 Operação E (multiplicação lógica)

A operação E, ou multiplicação lógica entre dois operandos resulta 0 se pelo menos uma das variáveis de entrada for 0 e será 1 todas as entradas valerem 1.

O símbolo usualmente utilizado na operação E é ".", porém outra notação possível é "Λ". Listando as possibilidades de combinações e os resultados da operação E, temos:

$$0 \cdot 0 = 0$$
$$0 \cdot 1 = 0$$
$$1 \cdot 0 = 0$$
$$1 \cdot 1 = 1$$

A operação E também é binária. Para mostrar o comportamento da equação A. B (lê-se A e B), escreve-se uma tabela verdade, como segue:

A	B	A . B
0	0	0
0	1	0
1	0	0
1	1	1

De forma semelhante, pode-se determinar o resultado da equação A.B.C (A e B e C) utilizando diretamente a definição da operação E: o resultado será 0 se pelo menos uma das variáveis de entrada valer 0.

A	B	C	A.B.C
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Também para a operação E valem as propriedades associativa e comutativa. Então, a equação A.B.C pode ainda ser avaliada tomando-se as variáveis aos pares, em qualquer ordem. Veja a tabela verdade a seguir e compare os resultados.

A	B	C	A.B.C	A.B	(A.B).C	B.C	A . (B.C)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0
0	1	1	0	0	0	1	0
1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0
1	1	1	1	1	1	1	1

2.2.3 – Complementação (negação ou inversão)

A complementação é a operação cujo resultado é simplesmente o valor complementar ao que a variável apresenta. Se o valor de uma variável for 0, o seu complemento será 1 e se o valor da variável for 1, o complemento será 0.

Os símbolos utilizados para representar a operação complementação sobre uma variável Booleana A são $\bar{A}$, ~A e A' (lê-se A negado). Nesta Apostila, será adotada a nota-

ção (A') pela dificuldade do editor Word. O resultado da operação complementação pode ser listado:

0' = 1
1' = 0

Diferentemente das operações OU e E, a complementação só é definida sobre uma única variável, ou sobre o resultado de uma expressão. Ou seja, o operador complementação é unário. A tabela verdade para a complementação de A é:

A	$\bar{A}$
0	1
1	0

2.3 Avaliação de expressões booleanas

Dada a equação que descreve uma função Booleana qualquer, deseja-se saber detalhadamente como esta função se comporta para qualquer combinação das variáveis de entrada. O comportamento de uma função é descrito pela sua tabela verdade e este problema é conhecido como avaliação da função ou da expressão que descreve a função considerada. Em suma, deseja-se achar a tabela verdade para a função Booleana. Uma tabela verdade consiste basicamente de um conjunto de colunas, nas quais são listadas todas as combinações possíveis entre as variáveis de entrada (à esquerda) e o resultado da função (à direita). Também, pode-se criar colunas intermediárias, onde são listados os resultados de sub-expressões contidas na expressão principal. Isto normalmente facilita a avaliação, principalmente no caso de equações muito complexas e/ou contendo muitas variáveis.

Quando numa mesma equação Booleana aparecem operações E e OU, é necessário seguir a ordem de precedência. Tal como na Álgebra dos reais, a multiplicação (lógica) tem precedência sobre a adição (lógica). Além disso,

expressões entre parêntesis têm precedência sobre operadores E e OU que estejam no mesmo nível. Quanto à complementação, esta deve ser avaliada tão logo seja possível. Caso a complementação seja aplicada sobre uma sub-expressão inteira, é necessário que se avalie primeiramente a sub-expressão para, só após, inverter o seu resultado.

O número de combinações que as variáveis de entrada podem assumir pode ser calculado por 2n, onde n é o número de variáveis de entrada.

O procedimento para a criação da tabela verdade a partir de uma equação Booleana é:

- Criar colunas para as variáveis de entrada e listar todas as combinações possíveis, utilizando a fórmula no de Combinações = 2^n (onde n é o número de variáveis de entrada);

- Criar uma coluna para cada variável de entrada que apareça complementada na equação e anotar os valores resultantes;

- Avaliar a equação seguindo a ordem de precedência, a partir do nível de parêntesis mais internos:
  1º multiplicação lógica
  2º adição lógica

Tomemos como exemplo a expressão W = X + Y .Z'. A variável W representa a função Booleana propriamente dita. Esta variável depende das variáveis que estão à direita do sinal =, ou seja, depende de X, Y e Z. Logo, são 3 as variáveis de entrada. O total de combinações entre 3 variáveis será 23 = 8. Então, a tabela verdade para W deverá ter 3 colunas à esquerda e 8 linhas. Seguindo o procedimento dado acima, cria-se uma coluna, na qual listam-se os valores para Z. Após, inicia-se a avaliação propriamente dita, a partir do nível mais interno de parêntesis. Como não há parêntesis na expressão, resolvem-se as sub-expressões que envolvem a operação E. No caso em questão, há somente uma tal sub-expressão, que é X . Y'. Então, cria-se uma coluna para X . Y', na qual anotam-se os resultados para este produto. Finalmente, utilizam-se os resultados de X.Y', listados na coluna anterior, para operar o OU com a variável X.

Repare os passos descritos na tabela verdade que segue. Nela, os parêntesis em torno do produto X .Y' indicam somente que este termo já foi avaliado e que no passo referente a esta coluna, tomaram-se apenas os valores previamente encontrados.

X	Y	Z	Z'	Y.Z'	W=X+Y.Z'
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	1
1	1	1	0	0	1

2.4 – Portas lógicas

por uma equação ou detalhada pela sua tabela verdade. Mas uma função Booleana também pode ser representada de forma gráfica, onde cada operador está associado a um símbolo específico, permitindo o imediato reconhecimento visual. Tais símbolos são conhecidos por portas lógicas.

Na realidade, mais do que símbolos de operadores lógicos, as portas lógicas representam recursos físicos, isto é, circuitos eletrônicos, capazes de realizar as operações lógicas. Na eletrônica que trabalha com somente dois estados, a qual é denominada eletrônica digital, o nível lógico 0 normalmente está associado à ausência de tensão (0 volt) enquanto o nível lógico 1, à presença de tensão (a qual geralmente é 5 volts). Nesta disciplina, nos limitaremos ao mundo da Álgebra Booleana, admitindo que as portas lógicas representam também circuitos eletrônicos que, de alguma maneira, realizam as funções Booleanas simbolizadas. Então, ao conjunto de portas lógicas e respectivas conexões que simbolizam uma equação Booleana, denominaremos circuito lógico.

2.4.1 – Portas OR (ou)

O símbolo da porta OU pode ser visto na Figura 2.1. Tal como na porta E, as entradas são colocadas à esquerda e a saída, à direita. Deve haver no mínimo duas entradas, mas há somente uma saída. O funcionamento da porta E segue a definição da operação E.

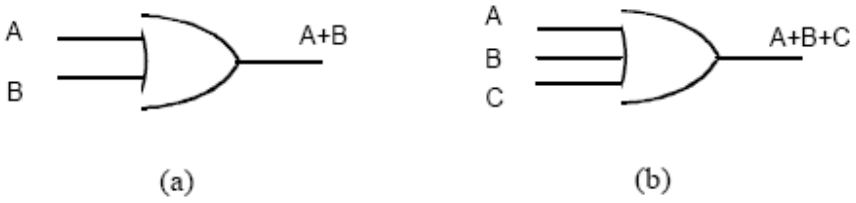
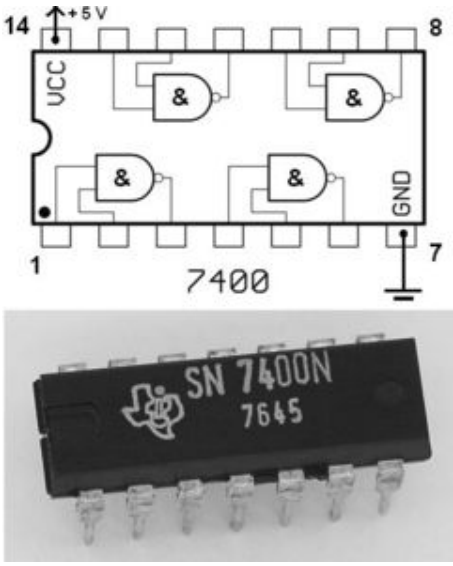


Figura 2.1 - Símbolo da porta lógica OU com 2 entradas (a) e com 3 entradas (b).



2.4.2 – Portas AND (E)

O símbolo da porta E é mostrado na Figura 2.2. À esquerda estão dispostas as entradas (no mínimo duas, obviamente) e à direita, a saída (única). As linhas que conduzem as variáveis de entrada e saída podem ser interpretadas como fios que transportam os sinais elétricos associados às variáveis. O comportamento da porta E segue estritamente a sua definição (e tabela verdade) dada anteriormente.

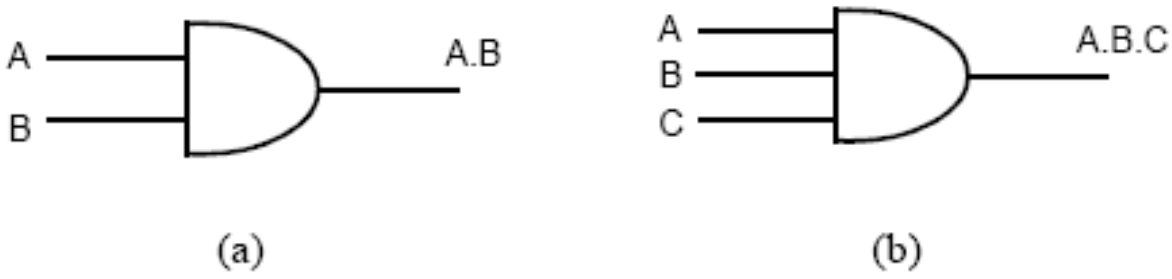


Figura 2.2 - Símbolo da porta lógica E com 2 entradas (a) e com 3 entradas (b).

2.4.3 – Inversores

A porta que simboliza a operação complementação é conhecida como inversor (ou porta inversora, ou negador). Como a operação complementação só pode ser realizada sobre uma variável por vez (ou sobre o resultado de uma sub-expressão), o inversor só possui uma entrada e, obviamente, uma saída. Caso se queira complementar uma expressão, é necessário obter-se primeiramente o seu resultado, para só então aplicar a complementação. O símbolo do inversor é mostrado na Figura 2.3.

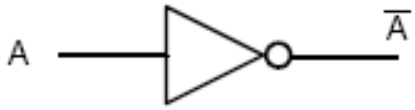


Figura 2.3 - Símbolo do inversor (também conhecido como negador ou porta inversora).

2.4.4 Exemplo de circuito lógico

Dada uma equação Booleana qualquer, é possível desenhar-se o circuito lógico que a implementa. O circuito lógico é composto das portas lógicas relacionadas às operações que são realizadas sobre as variáveis de entrada. Os resultados das operações são conduzidos por fios, os quais, no desenho, são representados por linhas simples.

Os passos a serem seguidos para se realizar o desenho do circuito lógico a partir de uma equação são praticamente os mesmos usados na avaliação da expressão. Tomemos como exemplo a equação, avaliada na seção 2.2. Inicialmente, identificamos as variáveis independentes, que no caso são X, Y e Z. Para cada uma destas, traçamos uma linha (da esquerda para a direita), representando os fios que conduzem os valores. Feito isto, deve-se seguir desenhando as portas necessárias para representar cada uma das sub-expressões, na mesma ordem tomada para a avaliação, ou seja:

1º parêntesis (dos mais internos para os mais externos);
2º operações E;
3º operações OU.

A Figura 2.4 mostra o circuito lógico para a equação W = X + Y . Z'.

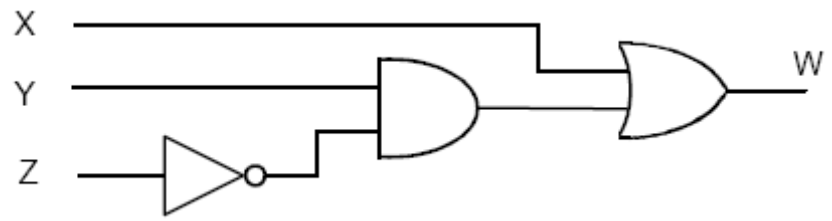


Figura 2.4 - Um circuito lógico.

2.4.5 Propriedades da Álgebra booleana

As leis da Álgebra Booleana dizem respeito ao espaço Booleano (isto é., valores que uma variável pode assumir) e operações elementares deste espaço. Já as propriedades podem ser deduzidas a partir das definições das operações. Sejam A e B duas variáveis Booleanas. Então, o espaço Booleano é definido:

se A≠0, então A=1;
se A≠1, então A=0.

As operações elementares deste espaço são operação OU, operação E e complementação, cujas definições foram dadas anteriormente. As propriedades da Álgebra Booleana são as seguintes.

Da adição lógica:

1. A + 0 = A
2. A + 1 = 1
3. A + A = A
4. A + A' = 1

Da multiplicação lógica:

5. A . 0 = 0
6. A . 1 = A

7. A . A = A
8. A . A' = 0

Da complementação:

9. (A')' = A

Comutatividade:

10. A + B = B + A
11. A . B = B . A

Associatividade:

12. A + (B + C) = (A + B)+ C
13. A.(B.C) = (A.B).C

Distributividade (da multiplicação em relação à adição):

14. A.(B + C) = A.B + A.C

2.4.6 – Teoremas de De Morgan

O primeiro teorema de De Morgan diz que a complementação de um produto (lógico) equivale à soma (lógica) das negações de cada variável do referido produto. Sob a forma de equação, teríamos:

(A . B . C ...)' = A' + B' + C' + ... (2.1)

O segundo teorema é o dual (i.e., o espelho) do primeiro, ou seja, a complementação de uma soma (lógica) equivale ao produto das negações individuais das variáveis:

(A + B + C + ...)' = A' . B' . C' . ... (2.2)

Particularizando os teoremas de De Morgan para duas variáveis, temos:

$$(A \cdot B)' = A' + B' \qquad (2.3)$$
$$(A + B)' = A' \cdot B' \qquad (2.4)$$

2.5 – Derivação de expressões booleanas

Dada uma função Booleana, descrita por sua tabela verdade, derivar uma expressão Booleana para esta função é encontrar uma equação que a descreva. Logo, a derivação de expressões Booleanas é o problema inverso da avaliação de uma expressão Booleana.

Há basicamente duas maneiras de se definir (ou descrever) uma função Booleana: descrevendo-se todas as situações das variáveis de entrada para as quais a função vale 1 ou, alternativamente, todas as situações em que a função vale 0. O primeiro método é conhecido por soma de produtos (SdP), enquanto que o segundo é chamado produto de somas (PdS). Qualquer função Booleana pode ser descrita por meio de soma de produtos ou por meio de produto de somas. Como as funções Booleanas só podem assumir um dentre dois valores (0 ou 1), basta usar-se um dos dois métodos para se encontrar uma equação para uma função.

2.5.1 – Expressões usando Soma de Produtos (SdP)

Dada uma função Booleana de n variáveis (ou seja, n entradas), haverá 2n combinações possíveis de valores. Dizemos que esse conjunto de valores que as variáveis podem assumir, juntamente com os respectivos valores da função, constituem o espaço da função. A cada combinação de entradas podemos associar um termo produto, no qual todas as variáveis da função estão presentes, e que é construído da seguinte forma: se a variável correspondente vale 0, ela deve aparecer negada; se a variável vale 1, ela deve aparecer não negada. A tabela a seguir lista os termos produto associados a cada combinação de entradas para uma função Booleana de três variáveis (A, B e C, por exemplo).

ABC	Mintermos
000	A'. B'. C'
001	A'. B'. C
010	A'. B . C'
011	A'. B . C
100	A . B' .C
101	Á . B' . C
110	A . B . C'
111	A . B . C

Cada termo produto construído conforme a regra anteriormente descrita é denominado mintermo (ou minitermo). Note que, para um dado mintermo, se substituirmos os valores das variáveis associadas, obteremos 1. Porém, se substituirmos nesse mesmo mintermo quaisquer outras combinações de valores, obteremos 0. Dessa forma, se quisermos encontrar a equação para uma função a partir de sua tabela verdade, basta montarmos um OU entre os mintermos associados aos 1s da função (também chamados mintermos 1).

Exemplo 2.1. Encontrar a equação em soma de produtos (SdP) para a função F, descrita pela seguinte tabela verdade:

A B C	F	
0 0 0	0	
0 0 1	0	
0 1 0	1	←
0 1 1	1	←
1 0 0	0	
1 0 1	1	←
1 1 0	1	←
1 1 1	0	

F é função das variáveis A, B e C. Os valores de (A,B,C) para os quais F=1 são (0,1,0), (0,1,1), (1,0,1) e (1,1,0), que estão indicados pelas setas na tabela acima. Os

mintermos associados a essas condições são A'.B.C', A'.B.C. A.B'.C, A.B.C'. Logo, a equação em soma de produtos para F será o OU entre estes produtos, ou seja:

$$F = : A'.B.C' + A'.B.C + A.B'.C + A.B.C' \qquad (2.5)$$

Para simplificar a notação, o símbolo da operação E pode ser omitido. Assim, a equação anterior pode ser reescrita de maneira mais concisa como

$$F = :A'BC' + A'BC + AB'C + ABC' \qquad (2.6)$$

2.5.2 – Produtos de somas usando maxtermos (PdS)

O método de derivação usando produto de somas é o dual (isto é, o oposto) do método de derivação em soma de produtos. A cada combinação das variáveis de entrada de uma função podemos associar um termo soma, no qual todas as variáveis da função estão presentes, e que é construído da seguinte forma: se a variável correspondente vale 1, ela deve aparecer negada; se a variável vale 0, ela deve aparecer não negada. A tabela a seguir lista os termos soma associados a cada combinação de entradas para uma função Booleana de três variáveis, A, B e C.

A B C	Maxtermos
0 0 0	A + B + C
0 0 1	A + B + C'
0 1 0	A + B'+ C
0 1 1	A + B' + C'
1 0 0	A' + B + C
1 0 1	A' + B + C'
1 1 0	A' + B' + C
1 1 1	A' + B' + C'

Exemplo 2.2. Encontrar a equação em produto de somas (PdS) para a função F, descrita pela seguinte tabela verdade:

A B C	F
0 0 0	0
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	0

Foi escolhida a mesma função do exemplo anterior, para que se possa estabelecer comparações entre os dois métodos de derivação. Os valores das variáveis de entrada (A,B,C) para os quais F=0 são (0,0,0), (0,0,1), (1,0,0) e (1,1,1). Os maxtermos associados a essas condições (ou seja, os maxtermos 0), são A + B + C , A + B + C', A' + B + C e A' + B' + C', respectivamente. Logo, a equação em produto de somas para F será o E entre estas somas, ou seja:

$$F = (A + B + C).(A + B + C').(A' + B + C).(A' + B' + C') \quad (2.7)$$

Note que a ordem de precedência de uma expressão em produto de somas é "primeiro cada soma deve ser avaliada, para só então avaliar-se o produto". Isto significa que os parênteses em torno de cada termo soma são obrigatórios! Repare também que os símbolos referentes à operação E (entre os termos soma) podem ser omitidos.

2.6 – Formas canônicas

As representações em soma de produtos e em produto de somas são denominadas formas padrão. A soma de produtos e o produto de somas, descritos nas duas seções anteriores, apresentam ainda uma característica bastante particular: em cada termo soma e em cada termo produto todas as variáveis da função estão presentes. Devido a essa característica, essas formas são chamadas canônicas.

Além das representações descritas nas seções anteriores, há representações alternativas (e mais concisas) para as expressões canônicas. Se associarmos cada combinação

das variáveis de entrada ao seu equivalente em decimal, cada mintermo pode ser representado por m_i, onde i é o decimal associado. De forma similar, cada maxtermo pode ser representado por M_i, onde i é o decimal associado. A tabela a seguir lista todos os mintermos e maxtermos de uma função de três variáveis (A, B e C).

A B C	Mintermo	Maxtermo
0 0 0	m_0	M_0
0 0 1	m_1	M_1
0 1 0	m_2	M_2
0 1 1	m_3	M_3
1 0 0	m_4	M_4
1 0 1	m_5	M_5
1 1 0	m_6	M_6
1 1 1	m_7	M_7

Voltando à função F das seções anteriores, podemos reescrever a expressão em soma de produtos, na forma canônica, como segue:

$$F = m_2 + m_3 + m_5 + m_6 \qquad (2.8)$$

Ou ainda, de maneira mais concisa:

$$F = \sum(2,3,5,6) \qquad (2.9)$$

E sua expressão em produto de somas, na forma canônica, pode ser reescrita como:

$$F = M_0 . M_1 M_4 . M_7 \qquad (2.10)$$

ou simplesmente, como:

$$F = \prod(0,1,4,7) \qquad (2.11)$$

Apesar da praticidade das representações canônicas, elas são pouco úteis para a implementação de circuitos digitais. O número de elementos (portas lógicas e conexões) de um circuito lógico depende diretamente do número de operações Booleanas (inversão, E e OU) contidas na expressão

associada. Desta forma, é normal que se deseje reduzir o número de operações contidas numa função, de modo a poder-se implementá-la com circuitos lógicos mais simples, e portanto, de menor custo. A redução do número de operações é obtida mediante a eliminação de literais da expressão, aplicando-se as propriedades da Álgebra Booleana descritas anteriormente. Um literal é uma variável negada ou não. O processo de redução de literais (ou de redução de operações, equivalentemente) é denominado simplificação.

Para exemplificar os passos básicos para a simplificação algébrica (literal) de expressões Booleanas, tomemos a expressão canônica, em soma de produtos, para a função F:

$$F = A' BC' + A'BC + AB'C + ABC' \qquad (2.12)$$

O primeiro passo é identificar pares de mintermos que se diferenciam por apenas um literal, a fim de aplicar a propriedade (14). Os mintermos A'BC' e A'BC, por exemplo, possuem os mesmos literais, exceto pela variável C: no primeiro, o literal é C' , enquanto no segundo, o literal é C. Então, com o uso da propriedade (14), pode-se fatorar esses dois mintermos, obtendo-se:

$$F = A'B(C' + C) + AB'C + ABC' \qquad (2.13)$$

Pela propriedade (4), tem-se que C' + C = 1. Substituindo na equação anterior, tem-se:

$$F = A'B .1 + AB'C + ABC' \qquad (2.14)$$

Como foi visto anteriormente, A'B.1 = A'B. Substituindo nesta equação, obtém-se:

$$F = A'B + AB'C + ABC' \qquad (2.15)$$

Assim, pela manipulação algébrica, obtivemos uma expressão em soma de produtos que é mais simples em relação a sua expressão em soma de produtos na forma canônica, pois o número de operações e também de literais foram reduzidos.

Entretanto, o mintermo A'BC' também poderia ter sido agrupado com o mintermo ABC', pois ambos possuem os mesmos literais, exceto pela variável A (A' no primeiro e A no segundo). Naturalmente, os passos a serem seguidos seriam os mesmos descritos anteriormente. E a equação resultante seria um pouco diferente, mas com o mesmo número de operações, sendo portanto, de mesma complexidade. Na verdade, o melhor seria se pudéssemos agrupar o mintermo A'BC' com o mintermo ABC' e ao mesmo tempo com o mintermo A'BC. Felizmente, a propriedade (3) da Álgebra Booleana diz que o OU entre duas ou mais variáveis Booleanas iguais é igual a própria variável Booleana em questão. Estendendo esta propriedade, pode-se dizer que o OU entre duas ou mais funções (inclusive produtos) Booleanas iguais equivale à própria função Booleana em questão. Desta forma, pode-se expandir o mintermo A'BC' para

$$A'BC' = A'BC' + A'BC' \qquad (2.16)$$

que é uma manipulação algébrica decorrente da propriedade (3). Retomando a equação 2.12 e utilizando 2.16, segue-se que

$$F = A'BC' + A'BC + AB'C + ABC' + A'BC' \qquad (2.17)$$

Então, a propriedade (3) garante que as expressões 2.12 e 2.17 são equivalentes, embora o mintermo A'BC' apareça duplicado. E pelo fato de aparecer duas vezes, pode-se usar uma cópia de A'BC' para simplificar com ABC' e outra para simplificar com A'BC. Os passos da simplificação são os mesmos já descritos: pela propriedade (14), segue:

$$F = A'B(C + C') + AB'C + (A + A')BC' \qquad (2.18)$$

e pela propriedade (6), vem:

$$F = A'B.1 + AB'C + 1.\ BC' \qquad (2.19)$$

Finalmente, pela propriedade (4), tem-se:

$$F = A'B + AB'C + BC' \qquad (2.20)$$

Repare que o mintermo AB'C não pôde ser agrupado com nenhum outro mintermo. Note também que foram feitas

todas as simplificações possíveis, uma vez que foram agrupados e simplificados todos os pares de mintermos que se diferenciam apenas por uma variável. Logo, a expressão 2.20 representa a simplificação máxima possível sob a forma de soma de produtos. E por esse motivo, ela é dita equação mínima em soma de produtos da função F. Quanto a expressão 2.15 é uma soma de produtos simplificada (porém, não-mínima). Logo, toda equação mínima é simplificada, porém, nem toda equação que foi simplificada é necessariamente mínima.

Embora a equação mínima em soma de produtos apresente menor número de operações Booleanas que a representação na forma canônica, as vezes pode ser possível reduzir-se ainda mais o número de operações, fatorando-se literais. Por exemplo, na expressão 2.20 pode-se fatorar o primeiro e o terceiro mintermos como segue:

$$F = B(A' + C') + AB'C \qquad (2.21)$$

A expressão 2.21, obtida pela fatoração de 2.20, não é nem do tipo soma de produtos, nem produto de somas, pois há um termo que não é nem produto, nem soma. Diz-se que a expressão está na forma fatorada. No caso de 2.21, a fatoração não resultou em redução do número de operações.

No que se refere a terminologia, as formas soma de produtos e produto de somas são ditas formas padrão. A forma fatorada é dita não-padrão. As formas canônicas são, pois, casos especiais de formas padrão, nas quais os termos são mintermos ou maxtermos. A fim de diferenciar somas de produtos canônicas de somas de produtos simplificadas, usaremos a expressão “soma de mintermos”. De maneira similar, usaremos a expressão “produto de maxtermos” para diferenciar produtos de somas canônicos de produtos de somas simplificados.

2.7 – Circuitos lógicos para formas canônicas

As regras gerais para se realizar o desenho de circuitos lógicos já foram apresentadas anteriormente As regras a

seguir devem ser observadas, a fim de facilitar a compreensão do desenho:

- as variáveis de entrada devem ser identificadas preferencialmente à esquerda, junto aos respectivos fios;
- inversores devem ser providos para as variáveis que aparecem negadas na equação;
- as portas que implementam as operações Booleanas que aparecem na equação normalmente são posicionadas da esquerda para a direita, seguindo a ordem de avaliação dos operadores

No caso de equações na forma soma de produtos (canônica ou simplificada), há um primeiro nível (desconsiderando-se possíveis inversores), constituído somente por portas E, onde cada porta E implementa um dos produtos da equação. Há ainda um segundo nível, constituído por uma porta OU, responsável pela "soma" lógica dos produtos. A Figura 2.5 mostra um possível circuito lógico para a equação 2.12.

Repare que em todas as interseções de fios em que há conexão física, deve haver um ponto (suficientemente grande), como se fosse uma "solda". Logo, quando não há o referido ponto na interseção de fios, significa que tais fios estão "eletricamente isolados".

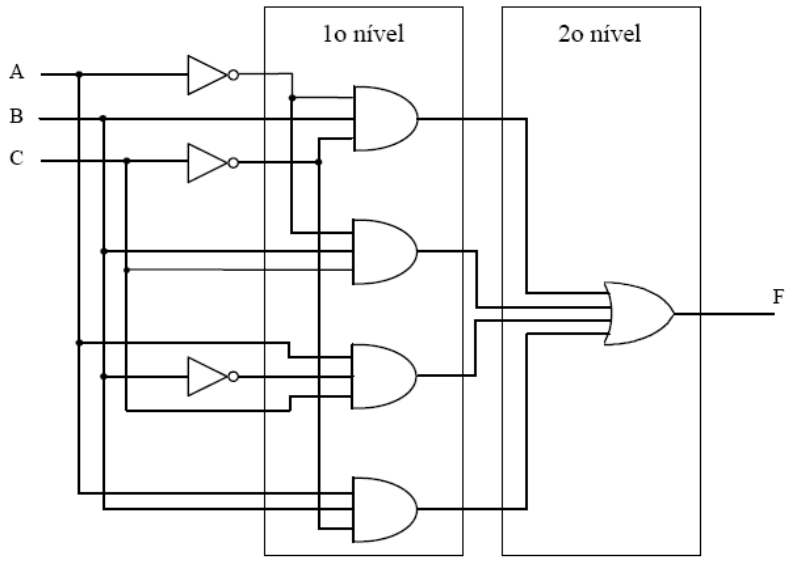


Figura 2.5 - Um circuito lógico para soma de produtos.

O circuito da Figura 2.5 pode ainda ser desenhado utilizando-se uma notação simplificada para os inversores das entradas. Ao invés de se desenhar um inversor para cada variável que aparece negada na equação, coloca-se um círculo junto a cada entrada de cada porta na qual há uma variável negada. Veja a Figura 2.6 abaixo.

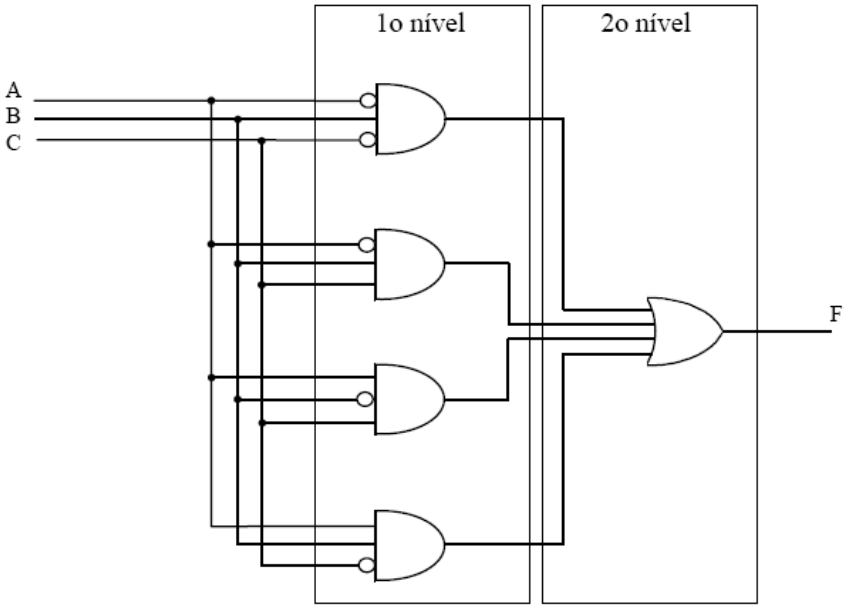


Figura 2.6 - Um circuito lógico para soma de produtos - outra possível representação.

No caso de equações na forma produto de somas (canônica ou simplificada), o primeiro nível é constituído por portas OU, sendo cada uma responsável por uma das “somas” lógicas da equação. O segundo nível, por sua vez, é constituído por uma porta E, que realiza o produto lógico das parcelas. A Figura 2.7 mostra um circuito lógico para a equação 2.7.

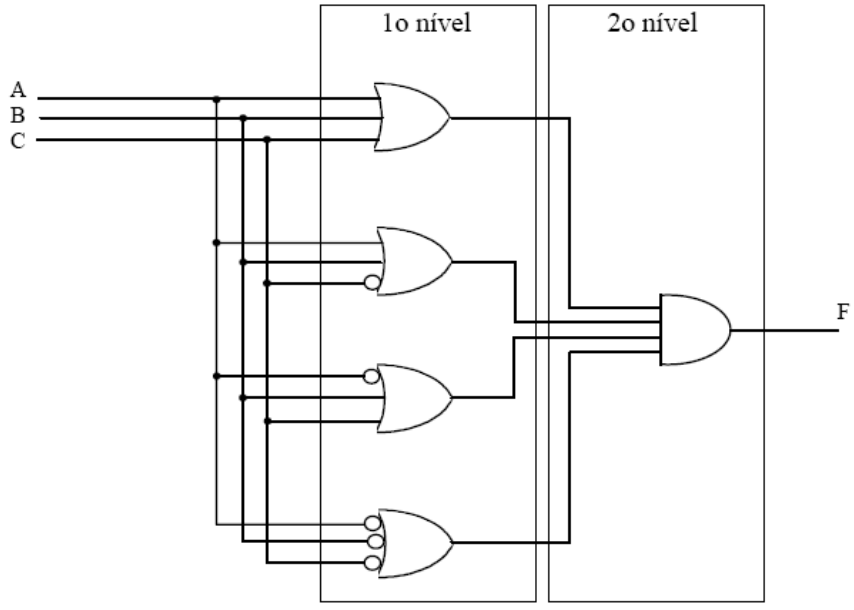


Figura 2.7 - Um circuito lógico para produto de somas.

Pelo fato de apresentarem apenas dois níveis de portas (dois níveis lógicos), circuitos para equações representadas nas formas padrão, canônicas ou simplificadas, são ditos circuitos em dois níveis (ou lógica a dois níveis).

Acomplexidade relativa de uma porta pode ser medida pelo número de entradas que ela apresenta. A Figura 2.8 mostra o circuito lógico para a equação 2.20, que é a forma mínima

para a função da equação 2.12. Note que este circuito é de menor complexidade que o circuito da Figura 2.6. A complexidade relativa de um circuito lógico pode ser calculada somando-se o número de entradas das portas. Nos circuitos das Figuras 2.6 e 2.7 há 4 portas de 3 entradas e 1 porta de 4 entradas. Então, a complexidade relativa será 4x3+1x4=16. No circuito da Figura 2.8 há 2 portas de 2 entradas e 2 portas de 3 entradas. Sua complexidade relativa será 2x2+2x3=10. Claramente, o circuito da Figura 2.8 é de menor complexidade que os circuitos das Figuras 2.6 e 2.7. Estes dois circuitos são de mesma complexidade relativa. No cálculo da complexidade relativa, as inversões normalmente não são levadas em conta.

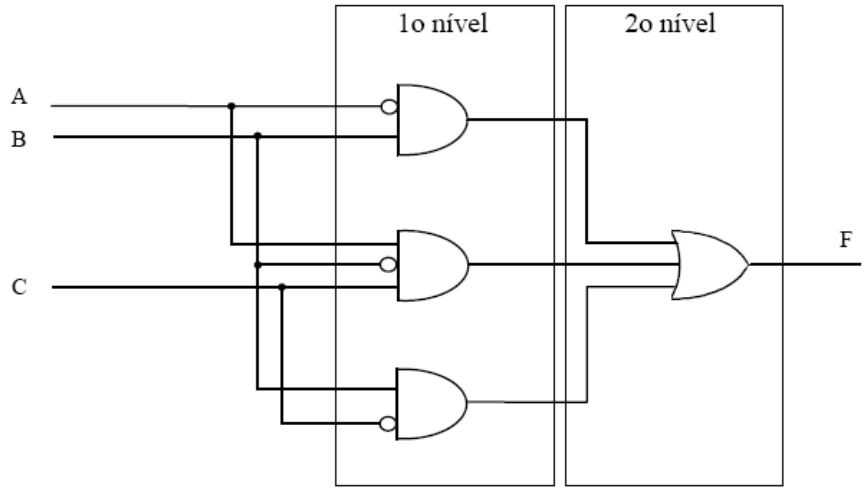


Figura 2.8 - Circuito lógico para a equação 2.20.

Circuitos para formas fatoradas podem ser vistos como o caso mais genérico. Em geral, as formas fatoradas conduzem a circuitos cuja quantidade de níveis lógicos é maior do que dois. Por isso, circuitos lógicos para formas fatoradas

são denominados circuitos multinível (lógica multinível). Às vezes uma forma fatorada pode apresentar menor número de operações do que a respectiva forma padrão. Quando isso ocorre, o circuito associado à forma fatorada também será de menor complexidade relativa. Entretanto, se não ocorrer redução no número de operações, mesmo assim é possível que o circuito para a forma fatorada seja de menor complexidade relativa, pois o conceito de complexidade relativa também inclui o número de entradas de cada porta. Então, a maneira mais segura de saber se o circuito associado à forma fatorada é de menor complexidade ou não é desenhá-lo e somar o número de entradas. A Figura 2.9 mostra o circuito para a equação 2.21, obtida a partir da equação 2.20 fatorando-se o literal B. Note que o número de operações Booleanas destas equações é o mesmo: 4. No entanto, a complexidade do circuito da forma fatorada é 3x2+1x3=9, portanto menor do que a complexidade do circuito da Figura 2.8.

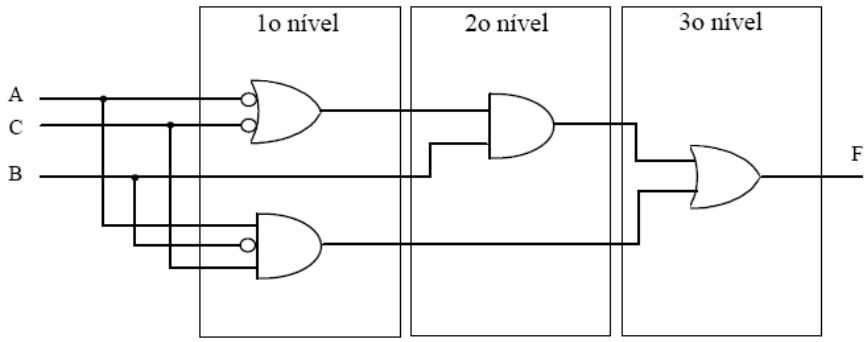


Figura 2.9 - Circuito lógico multinível, associado à equação 2.21, a qual está na forma fatorada.

2.8 – Simplificação de funções booleanas usando mapas de Karnaugh

O método de simplificação apresentado até aqui tem aplicação limitada, principalmente quando se tem um número grande de variáveis. Um método alternativo para estes casos foi inventado por Maurice Karnaugh, conhecido como mapas de Karnaugh ou mapas-K, consiste na simplificação baseado na identificação visual de grupos de mintermos passíveis de serem simplificados. No entanto, para que se possa identificar tais grupos, é necessário que os mintermos

sejam dispostos de maneira conveniente, o que será explicado nas próximas sub-seções. Todo o processo se baseia na simplificação de mintermos adjacentes.

Definição. Em uma expressão booleana na forma de soma de produtos, dois ou mais mintermos são adjacentes se existir, em todos eles, uma ou mais variáveis em comum. Estas ocorrências comuns terão que ser obrigatoriamente na mesma forma, ou seja ou todas na forma natural ou todas na forma complementar. Por exemplo, na expressão A'B + A'B', os mintermos A'B e A'B' são adjacentes porque a variável A' ocorre em ambos mintermos. Esta ocorrência implica em que a variável B pode ser eliminada facilmente utilizando os teoremas da Lógica Booleana, da seguinte forma: A'B + A'B' = A'(B + B') = A'. Neste caso, apenas uma variável foi eliminada, mas podem acontecer casos em que mais de uma variável pode ser eliminada.

Como outro exemplo, na expressão A'B'C'D' + A'B'CD' + AB'C'D' + AB'CD' todos os mintermos são adjacentes porque as variáveis B' e D' ocorrem em todos eles. Neste caso, A'B'C'D' + A'B'CD' + AB'C'D' + AB'CD' = (A'C' + A'C + AC' + AC)B'D' = ((A'(C' + C) + A(C' + C))B'D' = (A' + A)B'D' =B'D'. Ou seja, duas variáveis são eliminadas. A quantidade de variáveis que serão eliminadas na simplificação é exatamente a quantidade de mintermos adjacentes, sem considerar mintermos repetidos.

2.8.1 – Mapas de Karnaugh para duas variáveis

Para expressões booleanas com apenas duas variáveis, o mapa de Karnaugh é bastante simples, porque uma expressão booleana deste tipo só apresenta os mintermos m_0 = A'B', m_1 = A'B, m_2 = AB' e m_3.= AB. Cada mintermo tem seu local fixo, onde ele apresenta um valor 1 ou 0. Isto pode ser verificado na figura a seguir

	$\bar{B}$	B
$\bar{A}$	$\bar{A}\bar{B}$	$\bar{A}B$
A	$A\bar{B}$	AB

ou

	$\bar{B}$	B
$\bar{A}$	m_0	m_1
A	m_2	m_3

Maurice Karnaugh (4 de outubro de 1924) foi um físico americano que se tornou famoso pela criação dos mapas de Karnaugh utilizados na Álgebra Booleana.

Ele estudou Matemática e Física no *City College of New York* (1924-1928) e foi transferido para a Universidade de Yale para completar seu Bacharelado (1949), seu Mestrado (1950) e seu PhD em Física com uma Tese intitulada "*The Theory of Magnetic Resonance and Lambda-Type Doubling in Nitric-Oxide* "(1952).

Karnaugh trabalhou na Bell Labs (1952-1966) desenvolvendo os mapas de Karnaugh (1954) e também desenvolveu patentes para a PCM na área de Codificação de Circuitos Lógico-Magnéticos. Depois ele trabalhou na *Federal Systems Division* da IBM em Gaithersburg (1966-70) e na

Deve ser observado que o mintermo m_0 é adjacente ao mintemo m_1 e ao mintermo m_2 ao mesmo tempo, mas não é adjacente ao mintermo m_3. Os mintermos m_1 e m_2 também não são adjacentes.

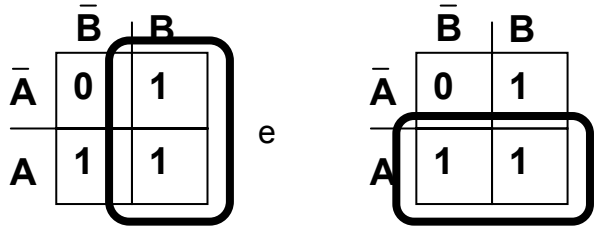
A partir de uma tabela verdade ou de uma expressão booleana em sua forma canônica, o primeiro passo para a simplificação desta expressão usando mapas-K consiste na plotagem no mapa dos valores de todos os mintermos, ou seja, 0 ou 1. O passo seguinte consiste no agrupamento de mintermos adjacentes. Este processo é o mesmo para expressões booleanas com qualquer número de variáveis. A diferença consiste apenas nas formas como os mapas são apresentados.

Exemplo. Simplificar a expressão booleana f(A,B) = A'B + AB' + AB.

Plotando os valores dos mintermos no mapa-K para duas variáveis temos:

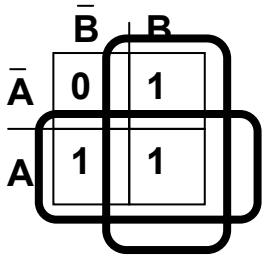
	$\bar{B}$	B
$\bar{A}$	0	1
A	1	1

A etapa seguinte consiste em agrupar os mintermos de valor 1 e que sejam adjacentes. Neste caso,



Todos os mintermos de valor 1 devem ser cobertos, mesmo que não tenham qualquer outro mintermo adjacente. No exemplo, deve-se observar que o mintermo m3 é adjacente ao mintermo m1 e m2 e, por este motivo, ele é agrupado com cada um destes. Isto é justificado pela proprieda-

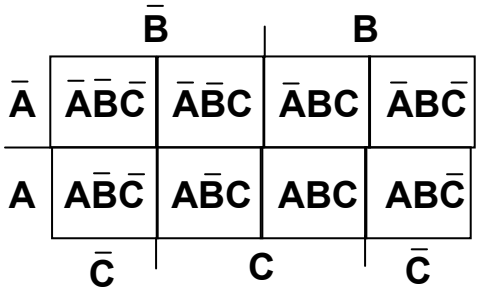
de da Álgebra Booleana de que A = A + A, ou seja, um mintermo pode ser duplicado para que a simplificação seja a mais abrangente possível. Assim, o mapa final se tornará:



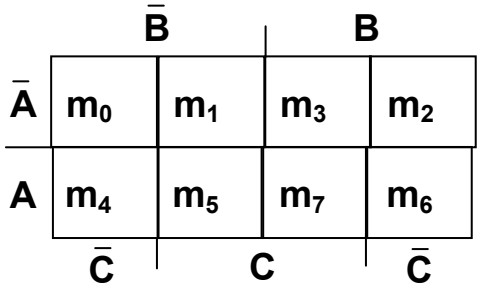
$f(A,B) = A + B$

2.8.2 – Mapas de Karnaugh para três variáveis

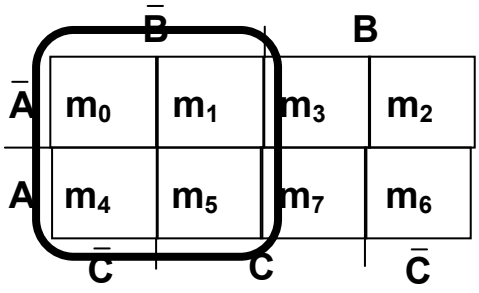
Para expressões booleanas com 3 variáveis, a aparência do mapa-K é a seguinte.



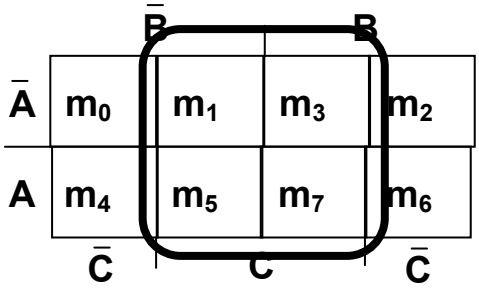
ou em forma de mintermos:



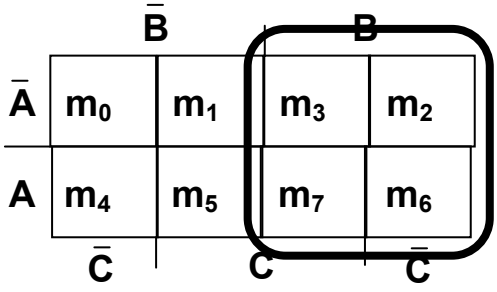
Os agrupamentos máximos são 4 mintermos, mas podem ser de 2 mintermos também. Os mintermos sem adjacentes serão considerados isolados. Neste caso, os agrupamentos possíveis de 4 mintermos são:



que é igual a B', ou

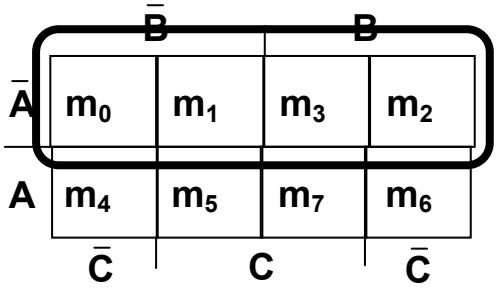


que é igual a C, ou ainda

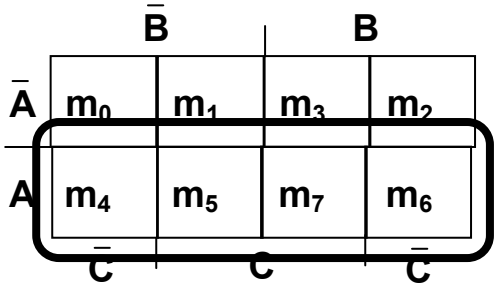


que é igual a B.

Podem ainda acontecer os seguintes agrupamentos de 4 mintermos:

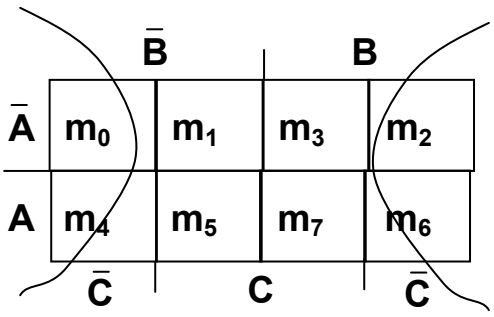


que é igual a A', ou ainda pode acontecer o agrupamento a seguir que é igual a A.



Deve ser observado ainda neste caso que o mintermo m_0 é adjacente ao mintermo m_4, ao mintermo m_1 e também ao mintermo m_2. Isto porque o mintermo m_0 = A'B'C' e o mintermo m_2 = A'BC', ou seja eles podem ser simplificados caso aconteçam em uma soma de produtos. Imagine como se o mapa fosse uma folha de papel que ao se dobrar formando um cone onde os mintermos m_0 e m_4 ficariam adjacentes aos mintermos m_2 e m_6 respectivamente. Na figura a seguir podemos visualisar este caso, em que o agrupamento resultante é C'.

Vamos verificar um exemplo.



Exemplo. Simplificar a função F(A,B,C) = A'BC + A'BC' + AB'C + ABC'

O primeiro passo é construir uma tabela para F, usando a nova disposição dos mintermos.

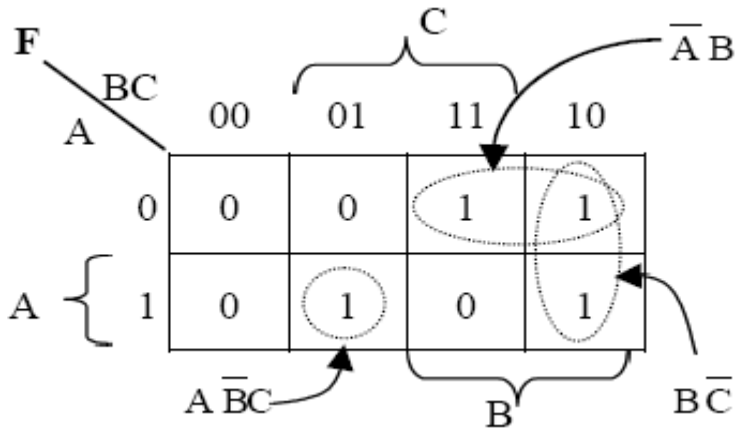
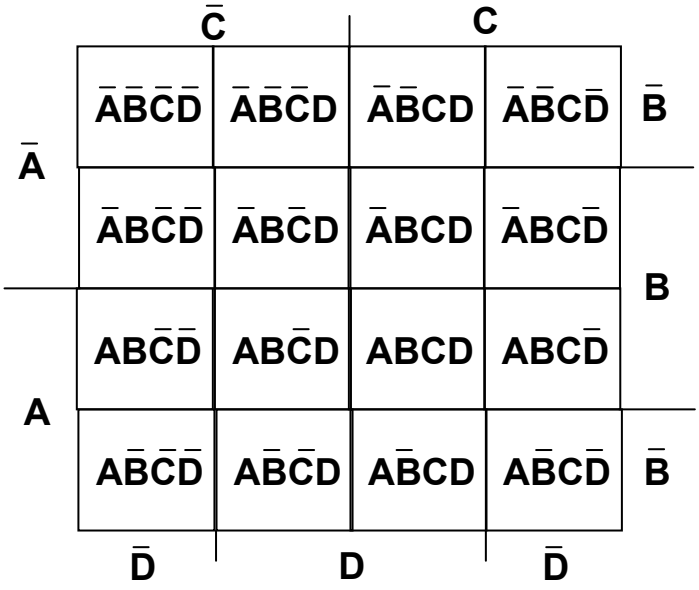


Figura 2.13 - Grupos de mintermos-1 adjacentes e termos produto para uma função de 3 variáveis.

Após, deve-se identificar todos os grupos de mintermos-1 adjacentes entre si. Cada grupo de mintermos-1 originará um produto, conforme indicado na Figura 2.13. A equação em soma de produtos simplificada será o OU entre os produtos encontrados:

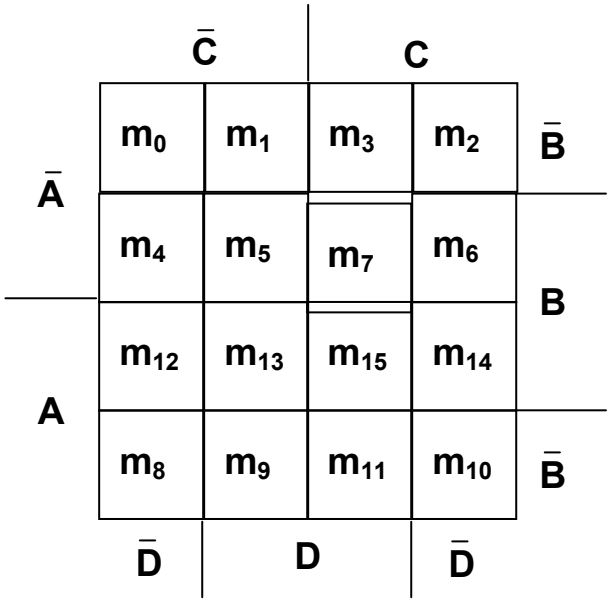
$$F = A'B + AB'C + BC'$$

2.8.3 – Mapas de Karnaugh para quatro variáveis



Os mapas de Karnaugh para 4 variáveis apresentam o aspecto mostrado na figura a seguir. Com 4 variáveis podemos ter até 16 mintermos. Neste caso, pode-se agrupar os

16 mintermos em um único grupo, ou seja, a expressão booleana é igual a 1, ou pode-se fazer agrupamentos de 8 mintermos, de 4 mintermos ou de 2 mintermos. Os agrupamentos de 8 mintermos eliminam 3 variáveis, os de 4 eliminam 2 e os de 2 eleiminam 1 variável. Ou em termos de mintermos:



Deve-se ter cuidado com termos adjacentes que não são obviamente claros neste caso. O procedimento é similar aos já vistos até aqui.

O procedimento básico para se determinar a melhor cobertura (também chamada cobertura mínima) para uma expressão em soma de produtos é o seguinte:

- Identificar os agrupamentos de mintermos com maior número de elementos possível, iniciando com o tamanho 2n, onde n é o número de variáveis da função. Caso algum mintermo fique isolado (isto é, não há nenhum outro mintermo adjacente a ele), então ele constituirá um agrupamento de um elemento;

- Identifiicar o menor conjunto de agrupamentos de modo que cada mintermo pertença a pelo menos um agrupamento. Isto significa que todo mintermos deve ser coberto pelo menos uma vez.

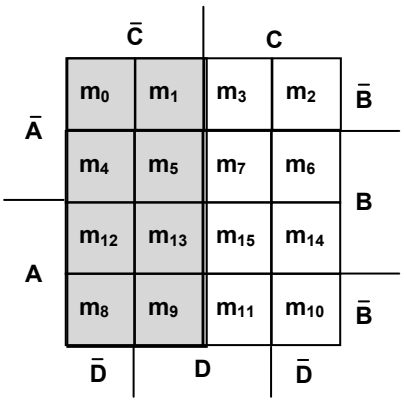
Observações:

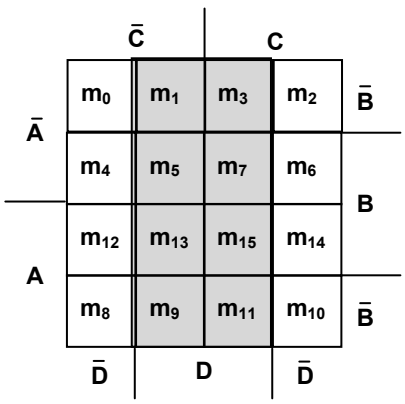
1. Cada mintermo pode ser coberto por mais de um agrupamento, caso isso resulte em uma simplificação maior;

2. Um último teste para verificar se a expressão obtida é realmente a mínima consiste em verificar se algum agrupamento pode ser removido, sem deixar algum mintermo descoberto. Um agrupamento que poder ser removido sem descobrir mintermos é dito subcubo não-essencial. Logo, todo agrupamento ou subcubo que não pode ser removido é dito essencial.;

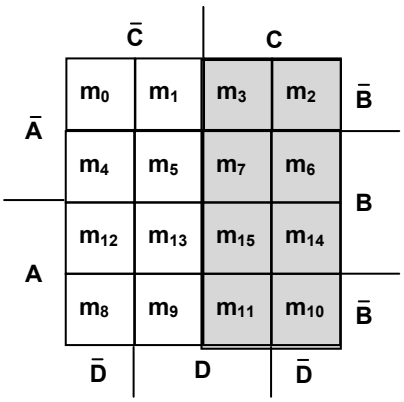
3. Pode haver mais de uma expressão mínima para uma mesma função Booleana;

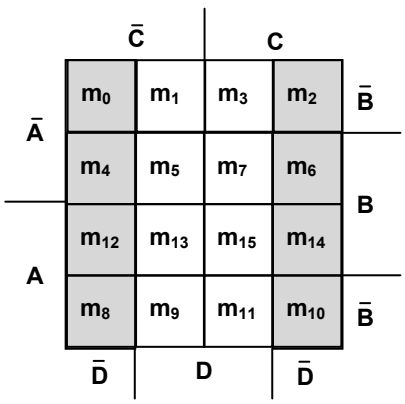
4. A expressão mínima é aquela de menor complexidade. E a complexidade é medida pelo número de literais de uma função, ou seja, pela quantidade de suas variáveis.

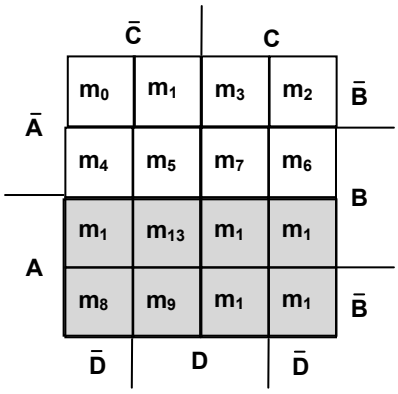


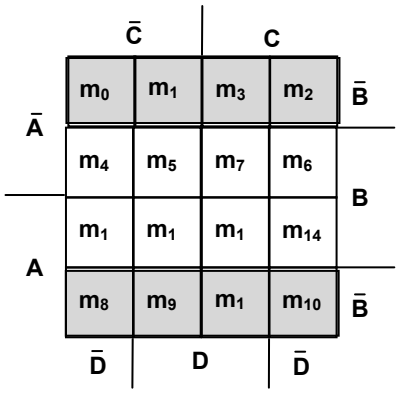


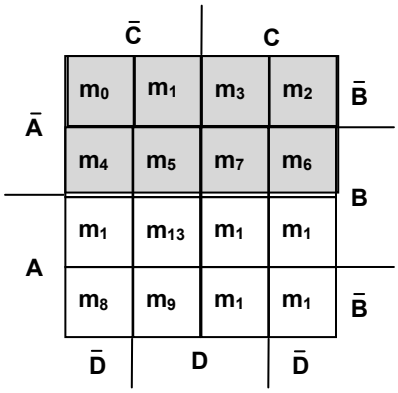
As áreas onde de mintermos adjacentes podem conter até 16 mintermos. Neste caso, o valor da função é 1. Estas áreas também podem conter 8 mintermos adjacentes, que são os seguintes, mostrados na figura a seguir.

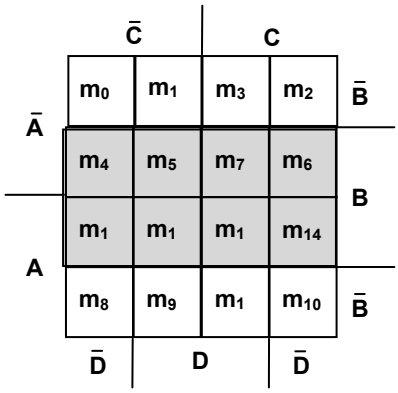






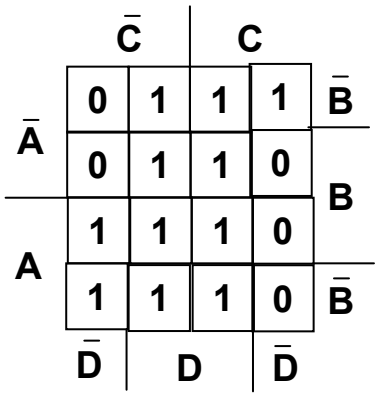




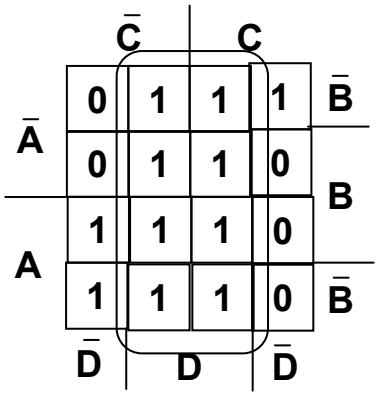


Exemplo. Utilizando mapas-K, simplificar a seguinte função booleana: F(A,B,C,D) = A'B'C'D + A'B'CD' + A'B'CD + A'BC'D + A'BCD + AB'C'D' + AB'C'D + AB'CD + ABC'D' + ABC'D + ABCD.

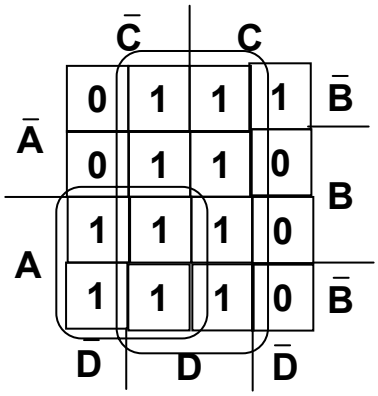
Plotando no mapa-K, temos:



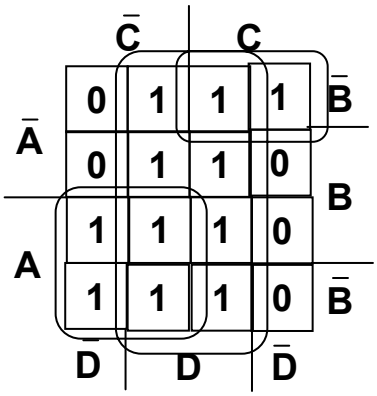
Neste caso, verificamos que o maior agrupamento possível é de 8 mintermos para construírem o agrupamento D. Isto pode ser verificado na figura a seguir:



Os mintermos m_8 e m_{12} podem ser agrupados para formar uma dupla. No entanto, observa-se também que eles podem ser agrupados com os mintermos m_9 e m_{13} para formarem a quadra AC', Esta forma de agrupamento é preferível porque anula-se mais uma variável na expressão final. Neste caso teremos o agrupamento mostrado na figura a seguir.



Finalmente, o mintermo m_2 é o último a ser considerado. Verificamos que ele está isolado mas ele é adjacente ao mintermo m_3 e portanto deve ser agrupado a este para formar o subcupo A'B'C, apesar do mintermo m_3 já estar agrupado em outro subcubo. A figura final é a seguinte:

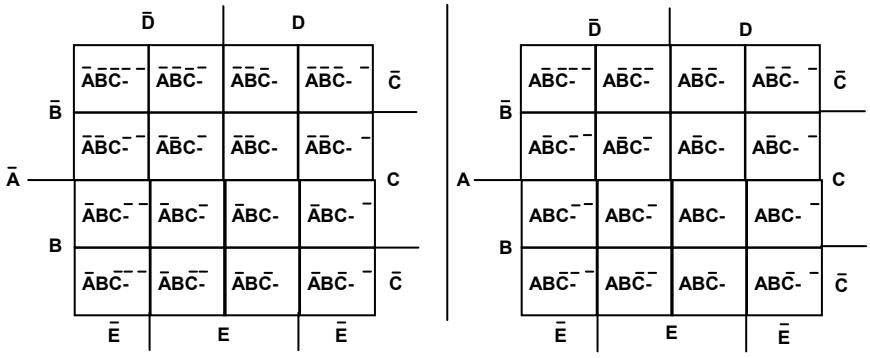


Neste caso, a função simplificada é F(A,B,C,D) = D + AC' + A'B'C.

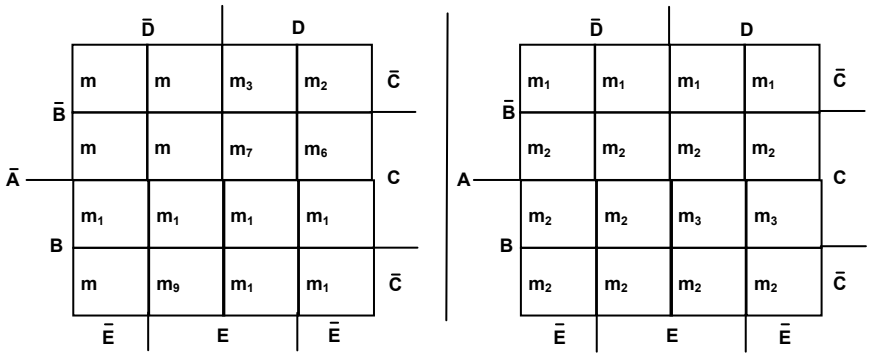
Exercício. Simplificar a função F(A,B,C,D) = Σ(2,3,5,7,9,10,11), utilizando mapas de Karnaugh.

2.8.4 – Mapas de Karnaugh para cinco variáveis

Os mapas de Karnaugh para 5 ou mais variáveis apresentam a necessidade, por parte do usuário, de que ele tenha uma visão espacial acurada para que possa perceber os mintermos adjacentes, uma vez que os mapas são bem mais complexos que os anteriores. Por exemplo, para funções com 5 variáveis o mapa-K tem a seguinte aparência:



ou em termos de mintermos,



A dificuldade surge porque o mapa do lado esquerdo deverá ser posto sobre o mapa do lado direito para determinar os mintermos adjacentes.

Mapas com mais de 5 variáveis também podem ser utilizados no entanto requer do usuário mais cuidado ainda.

2.8.5 – Mapas de Karnaugh para produto de somas

Conforme já mencionado anteriormente, também é possível obter-se uma expressão mínima em produto de somas a partir do mapa de Karnaugh da função Booleana. Para tanto, deve-se identificar os subcubos de mintermos-0, ao invés de subcubos de mintermos-1. Cada subcubo de mintermo-0 irá originar um termo soma, possivelmente já simplificado, o qual recebe o nome de "*implicado".*

Os passos para a obtenção de uma cobertura mínima são os mesmos já descritos para a obtenção da expressão em soma de produtos.

Exercício. Determinar a equação mínima em produto de somas para a função Booleana.

S2(W,X, Y, Z) = ∑(0,1,2,5,8,9,10)

Observação importante: repare que a função foi especificada pela descrição de seus mintermos-1. Mas como foi solicitada a expressão em produto de somas, uma vez montado o mapa de Karnaugh usando a informação fornecida, passaremos a identificar os subcubos de mintermos-0.

Exercício. Determinar a expressão mínima em soma de produtos e a expressão mínima em produto de somas para a função Booleana dada a seguir. Desenhar o circuito lógico para cada expressão obtida.

S3(A,B,C,D) = ∏(1,2,3,6,7,8,9,12,14)

Obs: existe mais de uma cobertura mínima possível para essa função.

2.9 – SAIBA MAIS

Existem muitos bons textos e alguns deles estão listados na Bibliografia colocada ao final da Unidade 2. Outros estão na Internet à disposição . Estes estão listados a seguir.

2.10 – WEB-BIBLIOGRAFIA

www.ufpi.br/uapi (A Página da Universidade Aberta do Piauí - UAPI)

www.uab.gov.br (O Site da Universidade Aberta do Brasil-UAB)

www.seed.mec.gov.br (A Homepage da Secretaria de Educação a Distância do MEC - SEED)

www.abed.org.br (O site da Associação Brasileira de Educação a Distância - ABED)

2.11 – REFERÊNCIAS BIBLIOGRÁFICAS

GAJSKI, Daniel D. Principles of Digital Design, New Jersey: Prentice Hall, 1997 (ISBN 0-13-301144-5)

MANO, M. Morris; Computer Engineering: Hardware Design. New Jersey: Prentice Hall, 1988 (ISBN 0-13-162926-3)

BROWN, Stephen; VRANESIC, Zvonko. Fundamentals of Digital Logic with VHDL Design, McGraw-Hill Higher Education (a McGraw-Hill Company), 2000 (ISBN texto: 0-07-012591-0 CD parte da coleção: 0-07-235596-4)

ERCEGOVAC, Milos; LANG, Tomás; MORENO, Jaime H. Introdução aos Sistemas Digitais. Porto Alegre: Bookman, 2000 (ISBN: 85-7307-698-4)

KATZ, Randy H. Contemporary Logic Design. The Benjamin/Cummings Publishing Company, Inc. , 1994 (ISBN: 0-8053-2703-7)

IDOETA, Ivan V. et CAPUANO, Francisco G. Elementos de Eletrônica Digital. 40ª. Edição. Editora Érica Ltda. São Paulo, 2008.

UYEMURA, John. Sistemas digitais: Uma Abordagem Integrada. Pioneira Thompson Learning Ltda. 2002.

Unidade 3

CIRCUITOS COMBINACIONAIS

Resumo

O objetivo principal desta unidade é apresentar os principais conceitos e estruturas dos circuitos combinacionais, onde as saídas dependem unicamente dos valores das entradas.

Estes circuitos apresentam um comportamento distinto dos circuitos seqüenciais que serão vistos na próxima unidade. Entre os circuitos combinacionais a serem vistos se dstacam os circuitos aritméticos que são responsáveis pelas operações como somas, subtrações e multiplicações que são muito comuns nos computadores.

A forma de apresentação utilizada é de acordo com o exigido para o ensino à distância, ou seja, tendo em vista sempre esta nova modalidade de ensino.

SUMÁRIO

UNIDADE 3 – CIRCUITOS COMBINACIONAIS

3 Circuitos combinacionais 68
3.1 Introdução 68
3.2 Análise de circuitos combinacionais 69
3.3 Projeto de circuitos combinacionais 70
3.4 Interconexão de Circuitos combinacionais 71
3.4.1 Decodificadores 72
3.4.2 Seletores 75
3.5 Circuitos aritméticos 76
3.5.1 Meio somador e somador completo 77
3.5.2 O somador paralelo 81
3.5.3 O somador/subtrator 84
3.5.4 O multiplicador 85
3.6 SAIBA MAIS 88
3.7 WEB-BIBLIOGRAFIA 88
3.8 REFERÊNCIAS BIBLIOGRÁFICAS 88

3 – Circuitos combinacionais

3.1 – Introdução

Os circuitos lógicos dos sistemas digitais podem ser de dois tipos: circuitos combinacionais ou circuitos seqüenciais.

Um circuito combinacional é constituído por um conjunto de portas lógicas as quais determinam os valores das saídas diretamente a partir dos valores atuais das entradas. Pode-se dizer que um circuito combinacional realiza uma operação de processamento de informação a qual pode ser especificada por meio de um conjunto de equações Booleanas. No caso, cada combinação de valores de entrada pode ser vista como uma informação diferente e cada conjunto de valores de saída representa o resultado da operação. A Figura 3.1 mostra o diagrama de blocos genérico de um circuito combinacional.

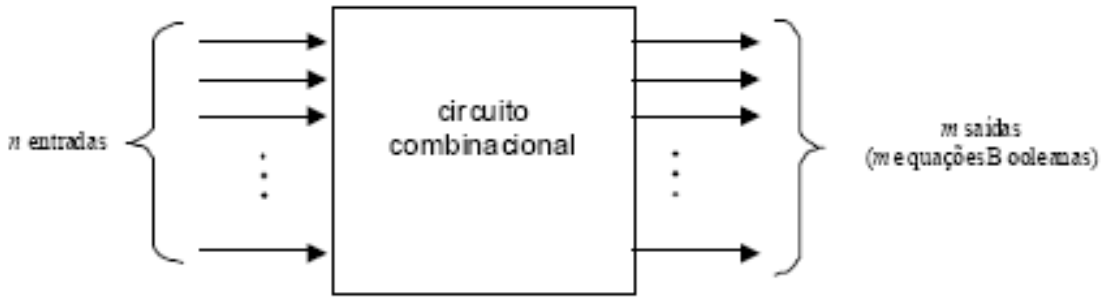


Figura 3.1 - Diagrama genérico de um circuito combinacional.

Um circuito seqüencial, por sua vez, emprega elementos de armazenamento denominados *latches* e *flip-flops*, além de portas lógicas. Os valores das saídas do circuito dependem dos valores das entradas e dos estados dos *latches* ou *flip-flops* utilizados. Como os estados dos *latches* e *flip-flops* é função dos valores anteriores das entradas, diz-se que as saídas de um circuito seqüencial dependem dos valores das entradas e do histórico do próprio circuito. Logo, o comportamento de um circuito seqüencial é especificado pela seqüência temporal das entradas e de seus estados internos. A Figura 3.2 esboça um diagrama de blocos genérico para circuitos seqüenciais conhecido como modelo de

Mealy. Circuitos seqüenciais serão objeto de estudo da próxima Unidade.

3.2 – Análise de circuitos combinacionais

O objetivo da análise de um circuito combinacional é determinar seu comportamento. Então, dado o diagrama de um circuito, deseja-se encontrar as equações que descrevem suas saídas. Uma vez encontradas tais equações, pode-se obter a tabela verdade, caso esta seja necessária. É importante certificar-se que o circuito seja realmente combinacional e não seqüencial. Um modo prático é verificar se existe algum caminho (ou ligação) entre saída e entrada do circuito. Caso não exista, o circuito é combinacional.

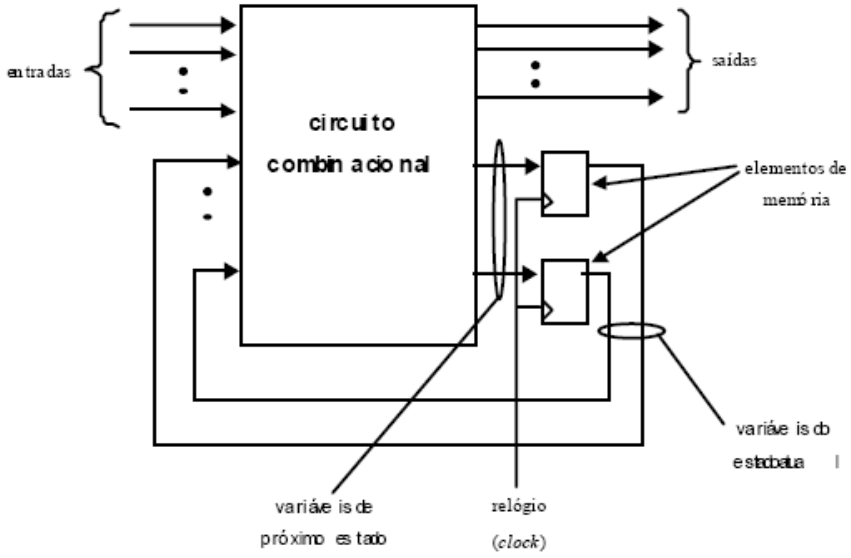


Figura 3.2 - Diagrama genérico de um circuito seqüencial segundo o modelo de Mealy.

O procedimento básico para se determinarem as equações que descrevem as saídas de um circuito combinacional é o seguinte:

1. Dar um nome para as variáveis associadas a cada saída de cada porta do circuito, exceto aquelas saídas que já possuem nome (como por exemplo, as saídas do circuito);

2. A partir da esquerda, e seguindo a ordem de precedência determinada pelas ligações, determinar as equações as-

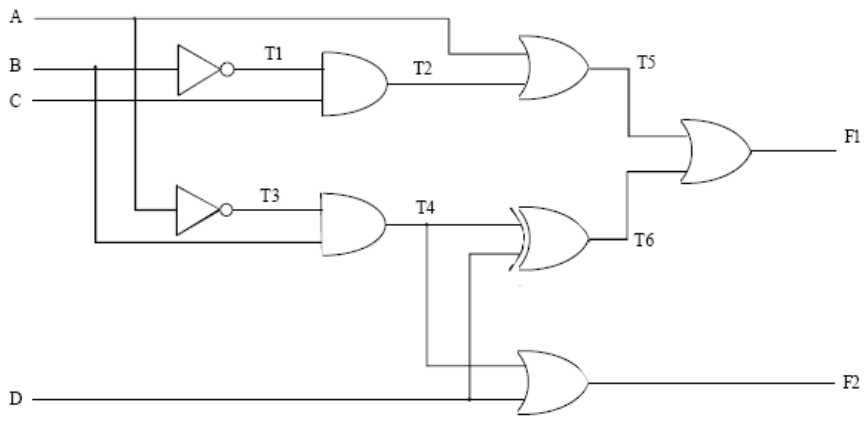
sociadas a cada variável, até que as equações de todas as saídas tenham sido encontradas.

Uma vez determinadas as equações das saídas, a montagem da tabela verdade será direta, havendo uma coluna para cada saída.

Exemplo 3.1. Determinar as equações das saídas F1 e F2 do circuito que segue.

Vamos chamar as variáveis associadas às saídas das portas de T1, T2, T3 etc. Há somente duas portas cujas saídas já tem nome, que são justamente as saídas do circuito: F1 e F2. Listando as equações para essas variáveis, segue:

T1 =
T2 =
T3 =
T4 =
T5 =
T6 =
F1 =



3.3 – Projeto de circuitos combinacionais

O projeto de um circuito combinacional inicia na especificação do problema e culmina no diagrama do circuito (ou no conjunto de equações que o descrevem). Um procedimento genérico para o projeto envolve os seguintes passos:

1. Escolher um símbolo para cada variável de entrada e para

cada variável de saída;

2. A partir da especificação do problema, determinar a tabela verdade (caso ela já não faça parte da especificação do problema);

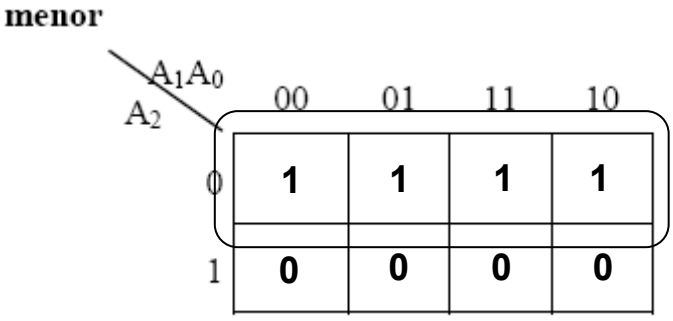
3. Obter as equações simplificadas;

4. Mapear o circuito para a biblioteca de portas disponível (se for o caso);

5. Desenhar o circuito final.

Exemplo 3.1. Projetar um circuito que recebe um inteiro binário de 3 bits e determina se este número é menor ou igual a 3. Usar somente portas NAND de duas entradas e inversores.

Chamemos a função que representa a saída do circuito de "menor". Denominando de $A_2A_1A_0$ aos 3 bits que compõem o número, podemos montar o mapa de Karnaugh para essa função. Os valores para os quais "menor" deve valer 1 são $A_2A_1A_0$={(0,0,0);(0,0,1);(0,1,0);(0,1,1)}. Para as demais combinações de entradas, "menor" deverá ser igual a 0.



menor= A'

3.4 – Interconexão de Circuitos combinacionais

Os circuitos combinacionais são os responsáveis pelas operações lógicas e aritméticas dentro de um sistema digital (vale lembrar que um computador é um sistema digital). A-

lém das operações lógicas e aritméticas como adição, subtração complementação, existem ainda outras funções necessárias para a realização de conexões entre os diversos operadores. Dentre essas funções estão a multiplexação e a decodificação. Os elementos que realizam essas últimas operações são denominados multiplexadores e decodificadores, respectivamente, e são também circuitos combinacionais. A seguir, veremos como tais circuitos são constituídos.

3.4.1 – Decodificadores

Um decodificador é um circuito combinacional usado para ativar ou habilitar um (e somente um) dentre m componentes. É assumido que cada componente possui um índice entre 0 e m-1, representado por um endereço em binário.

Um decodificador n : m (lê-se n por m) possui n entradas e m saídas, com m ≤2n.

No caso de um decodificador 3:8, serão 8 saídas, onde cada saída pode ser encarada como um endereço diferente. Para ativar uma dentre 8 saídas são necessárias 3 variáveis de entrada (daí 3:8). Cada combinação das variáveis de entrada seleciona uma e somente uma dentre as 8 saídas, de modo que cada saída somente será selecionada por uma das 8 combinações. Desta forma, é natural que se associe a cada saída um índice decimal que represente a combinação de entradas responsável pela sua ativação. Assumindo-se ativação em lógica direta, isto é, que uma saída está ativada se ela vale 1, então a tabela verdade para um decodificador 3:8 será:

endereço	Entradas (sinais de controle)			saídas							
	A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
2	0	1	0	0	0	1	0	0	0	0	0
3	0	1	1	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0	0
5	1	0	1	0	0	0	0	0	1	0	0
6	1	1	0	0	0	0	0	0	0	1	0
7	1	1	1	0	0	0	0	0	0	0	1

Note que cada saída só vale 1 para uma determinada combinação das variáveis de entrada. Além disso, cada combinação de entrada só ativa uma dentre todas as 8 saídas.

O circuito de um decodificador 3:8 terá, portanto, 8 saídas, sendo cada saída um dentre os 8 mintermos possíveis para uma função Booleana de 3 variáveis. A Figura 3.3a mostra o símbolo para o decodificador 3:8, enquanto a Figura 3.3b mostra um circuito possível para o mesmo decodificador, utilizando portas E de 3 entradas e inversores.

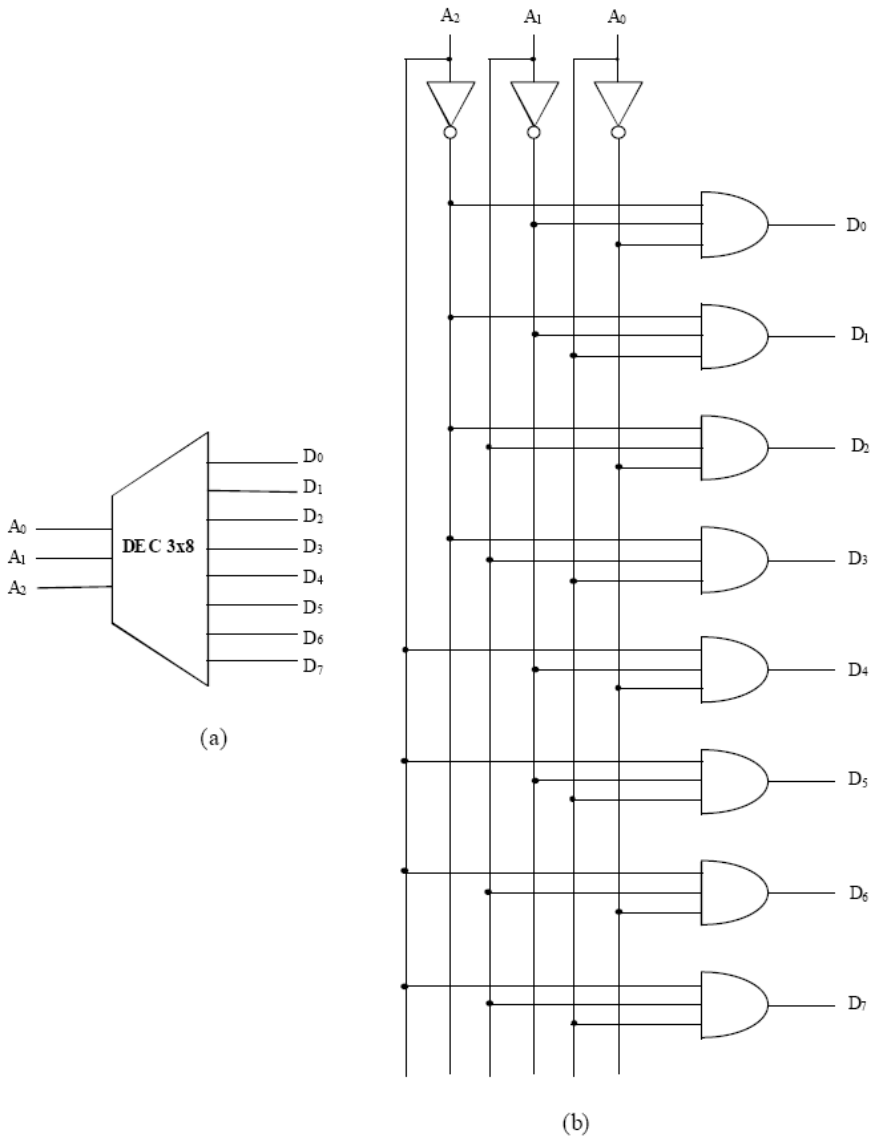


Figura 3.3- Símbolo (a) e diagrama (b) de um decodificador 3:8.

Um decodificador pode possuir uma entrada de habilitação. Esta entrada tem a função de habilitar ou desabilitar seu funcionamento. Assim, se esta entrada valer 0, nenhuma saída estará ativada, independente dos valores das demais entradas. Por outro lado, se a entrada de habilitação valer 1, o decodificador estará ativando uma das saídas.

Neste exemplo, foi considerado que a habilitação do decodificador se dá com lógica direta, isto é, quando a entrada de habilitação valer 1. A lógica de habilitação poderia ser negada, ou seja, habilita se a entrada de habilitação valer 0 e não habilita, caso contrário.

A tabela verdade de um decodificador 2:4 com ativação e habilitação em lógica direta é a seguinte:

endereço	entradas (sinais de controle)			saídas			
	E	A_1	A_0	D_0	D_1	D_2	D_3
-	0	0	0	0	0	0	0
-	0	0	1	0	0	0	0
-	0	1	0	0	0	0	0
-	0	1	1	0	0	0	0
0	1	0	0	1	0	0	0
1	1	0	1	0	1	0	0
2	1	1	0	0	0	1	0
3	1	1	1	0	0	0	1

saídas desabilitadas

Como pode-se verificar, nas primeiras 4 linhas o sinal de habilitação (E) vale zero, o que desativa as saídas, independentemente dos valores das demais entradas (A_1 e A_0). Desta forma, podemos re-escrever esta tabela de maneira mais compacta, indicando numa única linha que, quando E=0, os valores das entradas A_1 e A_0 não interessam (*don't cares* de entrada):

endereço	entradas (sinais de controle)			saídas			
	E	A_1	A_0	D_0	D_1	D_2	D_3
-	0	X	X	0	0	0	0
0	1	0	0	1	0	0	0
1	1	0	1	0	1	0	0
2	1	1	0	0	0	1	0
3	1	1	1	0	0	0	1

saídas desabilitadas

A Figura 3.4a mostra o símbolo para esse decodificador e a Figura 3.4b mostra uma possível implementação (circuito lógico).

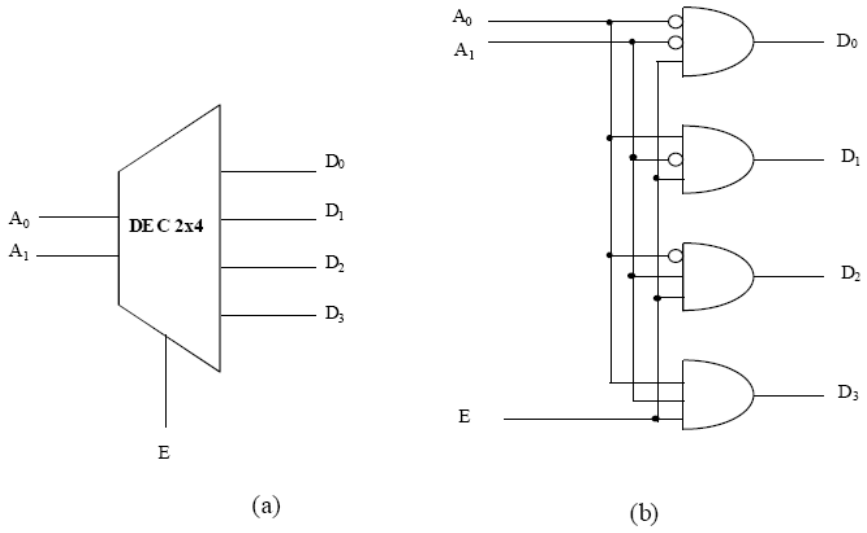


Figura 3.4: símbolo (a) e diagrama (b) de um decodificador 2x4 com entrada de habilitação.

3.4.2 – Seletores

Um seletor (também conhecido como multiplexador) é um circuito combinacional usado para selecionar uma dentre um conjunto de m fontes de informação disponíveis. Um seletor que possui n entradas para realizar a seleção é capaz de selecionar uma dentre 2^n entradas. Logo, m deve ser menor ou igual a 2^n.

Dado o conjunto de entradas A_0, A_1, A_2 e A_3, e as variáveis de seleção S_0 e S_1, a tabela verdade para um seletor 4-1 será:

endereço	variáveis de seleção		saída
	S_1	S_0	Y
0	0	0	A_0
1	0	1	A_1
2	1	0	A_2
3	1	1	A_3

Pela tabela verdade acima percebe-se que a saída Y pode ser implementada por um circuito em soma de produtos, onde em cada produto estarão presentes as variáveis S_0 e S_1 e uma dentre as variáveis de entrada A_0, A_1, A_2 e A_3:

$$Y = \overline{S_0} \cdot \overline{S_1} \cdot A_0 + \overline{S_0} \cdot S_1 \cdot A_1 + S_0 \cdot \overline{S_1} \cdot A_2 + S_0 \cdot S_1 \cdot A_3$$

A Figura 3.5a mostra o símbolo para tal seletor e a Figura 3.5b mostra um possível circuito em soma de produtos.

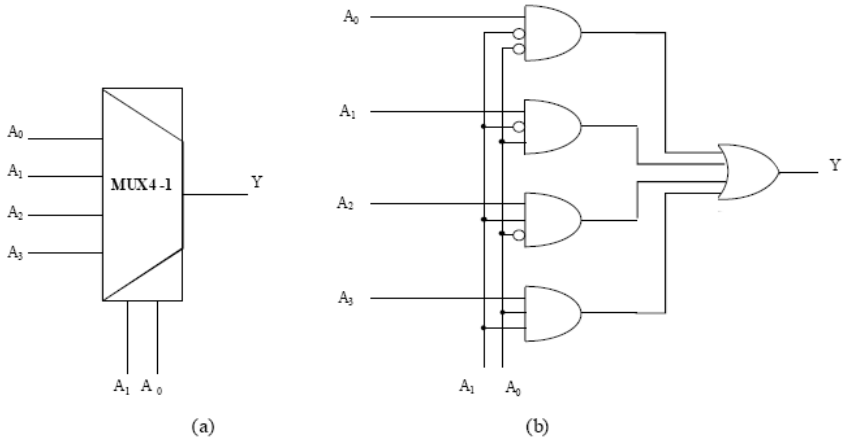


Figura 3.5 - Símbolo (a) e diagrama (b) de um seletor 4-1.

3.5 – Circuitos aritméticos

Um circuito combinacional aritmético implementa operações aritméticas como adição, subtração, multiplicação e divisão com números binários. A operação aritmética mais simples é a adição de dois dígitos binários, que consiste de quatro possíveis operações elementares: 0+0=0, 0+1=1, 1+0=1 e 1+1=10. As três primeiras operações produzem um dígito de soma. Entretanto, quando ambos os operandos são iguais a 1, são necessários dois dígitos para expressar seu resultado. Neste caso, o transporte (vai-um ou carry, em inglês) é somado ao próximo par mais significativo de bits. Um circuito combinacional que implementa a adição de dois bits é chamado meio-somador (half adder, em inglês). Um circuito que implementa a adição de três bits (dois bits significativos e um carry) é chamado de somador completo (full adder, em inglês). Estes nomes decorrem do fato de que com dois meio-somadores pode-se implementar um somador comple-

to. O somador completo é um circuito aritmético básico a partir do qual todos os outros circuitos aritméticos são construídos.

3.5.1 – Meio somador e somador completo

A operação aritmética mais simples é a adição de dois dígitos binários (bits), a qual pode ser vista como a adição de dois números binários de um bit cada. Considerando-se todas as 4 combinações de valores que podem ocorrer, os resultados possíveis dessa adição são:

0+0=0
0+1=1
1+0=1
1 + 1 = 10

Repare que no último caso acima, o resultado da adição é o valor 2, que em binário necessita de dois dígitos para ser representado (10). Ora, um circuito aritmético para realizar a adição de dois bits deve operar corretamente para qualquer combinação de valores de entrada. Isso significa que o circuito para a adição de dois bits deve possuir duas entradas e duas saídas, conforme ilustrado na Figura 3.6.

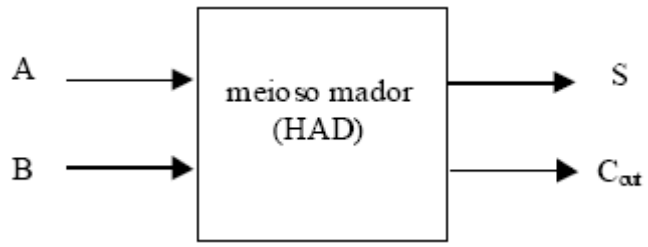


Figura 3.6 - Esquema das entradas e saídas de um meio somador (*half adder* ou HAD).

Denomina-se meia-soma a operação de adição de dois bits. O circuito mostrado na Figura 3.6 é denominado meio somador. As duas entradas, A e B, representam os dois bits a serem adicionados. A saída S representa o dígito menos significativo do resultado, enquanto que a saída Cout representa o dígito mais significativo do resultado, o qual também é conhecido por transporte de saída (*carry*), uma vez que ele assume valor 1 somente quando o resultado da soma de A e B não pode ser representado num único dígito.

A fim de se projetar o circuito do meio somador, devemos montar uma tabela verdade para as saídas S e Cout utilizando-se os valores que resultam da adição de dois dígitos binários, como segue:

A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Note que a saída S nada mais é do que o XOR entre A e B (S =A'.B + A. B' = A⊕B). Já a saída Cout é o E entre A e B (Cout = A . B). Então, um circuito para o meio somador usa apenas uma porta XOR de duas entradas e uma porta E de duas entradas, conforme mostrado na Figura 3.7.

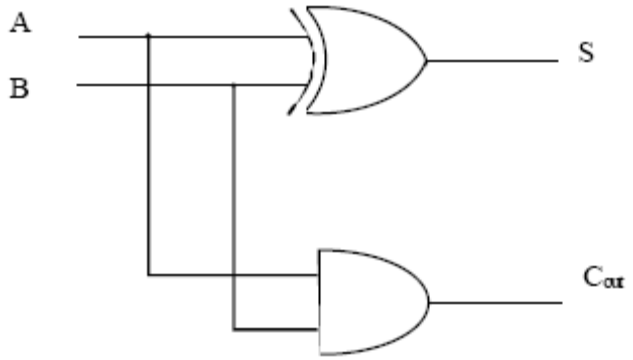


Figura 3.7 - Circuito para o meio somador (half adder ou HAD).

Entretanto, quando ao somarmos dois números binários que possuem mais de um dígito cada ocorrer transporte diferente de zero para a soma de um par de dígitos intermediários, a soma do par seguinte deverá considerar esse transporte proveniente do par anterior, conforme ilustra o exemplo a seguir (Figura 3.8).

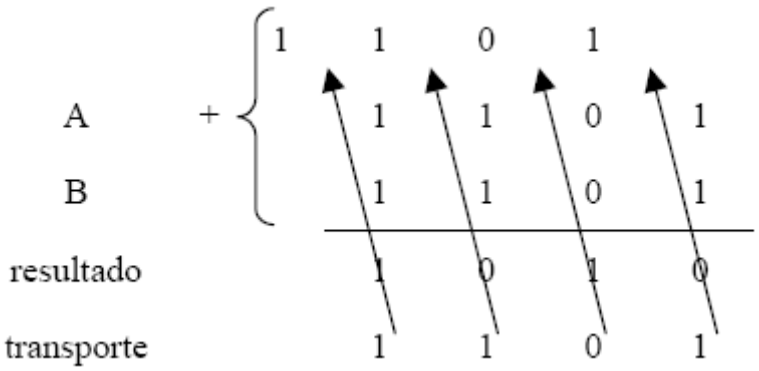
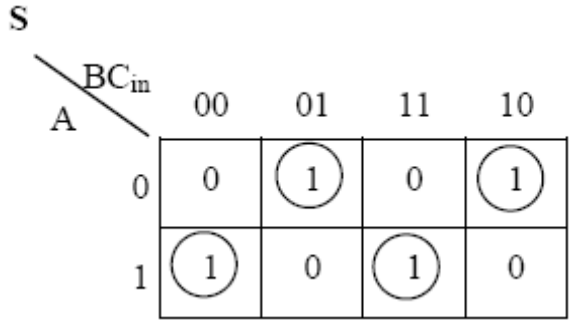


Figura 3.8 - Exemplo de adição de dois números binários com mais de um dígito.

O exemplo mostrado na Figura 3.8 ilustra bem o fato de, para cada posição exceto a menos significativa, o resultado é obtido mediante a adição de três bits: um pertencente ao número A, um pertencente ao número B e um terceiro que é o transporte proveniente do resultado da adição entre os bits da posição anterior.

O circuito capaz de realizar a soma de três bits (A, B e Cin), gerando o resultado em dois bits (S e Cout) é denominado somador completo (full adder, em inglês). Apesar da entrada Cin normalmente receber o transporte proveniente da soma imediantamente anterior (carry in, em inglês), a rigor as três entradas são absolutamente equivalentes sob o ponto de vista funcional. A tabela verdade para a soma completa é mostrada a seguir, juntamente com o mapa de Karnaugh e as equações mínimas resultantes para S e Cout. A Figura 3.9 mostra um circuito para o somador completo.

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



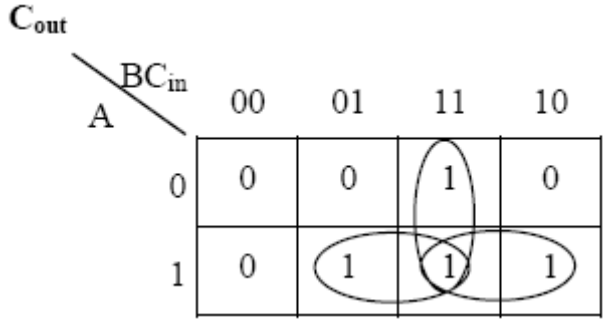
Conforme pode-se ver pelo mapa de Karnaugh acima, a expressão mínima em soma de produtos para S contém todos os mintermos da função:

$$S = \overline{A} \cdot \overline{B} \cdot C_{in} + \overline{A} \cdot B \cdot \overline{C_{in}} + A \cdot \overline{B} \cdot \overline{C_{in}} + A \cdot B \cdot C_{in}$$

O circuito que implementa a saída S do somador completo pode ser derivado a partir da equação em soma de produtos acima. No entanto, pode-se ainda manipular tal equação conforme segue:

$$S = \overline{A} \cdot (\overline{B} \cdot C_{in} + B \cdot \overline{C_{in}}) + A \cdot (\overline{B} \cdot \overline{C_{in}} + B \cdot C_{in}) = \overline{A} \cdot (B \oplus C_{in}) + A \cdot \overline{(B \oplus C_{in})} = A \oplus B \oplus C_{in}$$

Logo, o circuito para a saída S do somador completo pode também ser representado com duas portas XOR, conforme mostra a Figura 3.9.



A saída C_{out} tem como expressão mínima em soma de produtos:

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

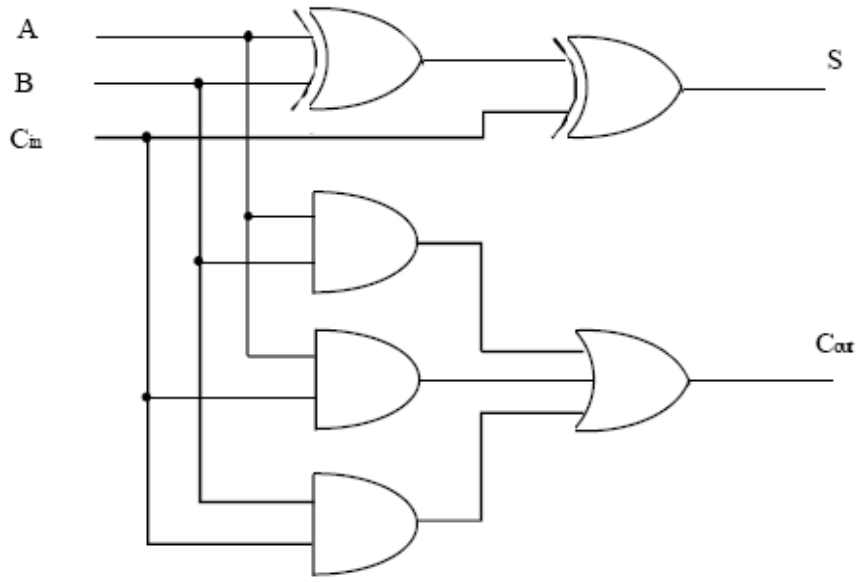


Figura 3.9 - Circuito para o somador completo (full adder ou FAD).

EXERCÍCIOS

1. Um cofre tem 5 cadeados com chaves (x, y, z, v, w) e 5 executivos (A, B, C, D, E) são possuidores de chaves para estes cadeados, conforme o quadro abaixo:
   - o executivo A tem as chaves x e v;
   - o executivo B tem as chaves v e y;
   - o executivo C tem as chaves w e y;
   - o executivo D tem as chaves x e z;
   - o executivo D tem as chaves v e z.

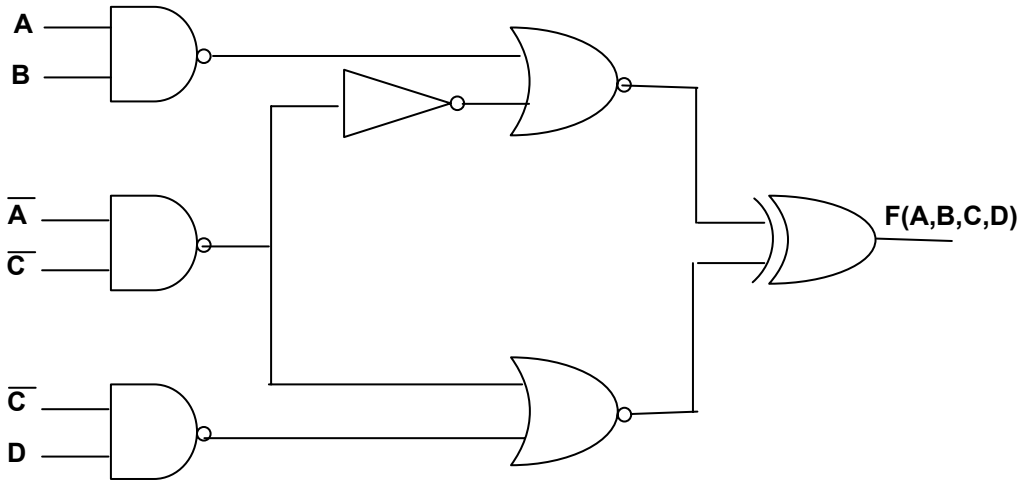
a) Determine o número mínimo de executivos necessários para que o cofre possa ser aberto (sem massarico);

b) Ache todas as combinações de executivos que abrem o cofre;

c) Qual o executivo indispensável para o cofre seja aberto?

2. Ao longo de uma parede existem 3 portas. Ao lado de cada porta existe um interruptor. Os três interruptores (A, B, C) controlam uma lâmpada. Encontre a tabela verdade de f(A, B, C) que indica quando a lâmpada está acesa.

3. Dado o diagrama abaixo,



a) Encontre a forma em mintermos da função F;
b) Ache as forma em maxtermos da função F.

3.5.2 O somador paralelo

Utilizando-se n somadores completos, pode-se realizar um somador capaz de operar dois números binários de n bits. Particularmente, o dígito de ordem i do resultado, Si, será obtido pela adição de A_i, B_i e C_i, onde C_i é o transporte proveniente do dígito anterior. O somador de índice i recebe como entradas A_i, B_i e C_i, gerando a soma Si e o valor de transporte C_{i+1}, o qual será entrada para o somador completo do dígito seguinte (i+1). A Figura 3.10 mostra uma representação de bloco possível para o somador completo da Figura 3.9. A Figura 3.11 mostra um circuito somador paralelo para números binários com 4 bits.

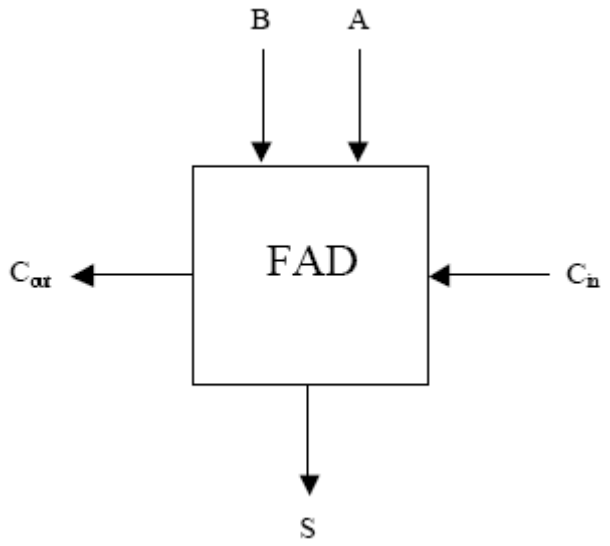


Figura 3.10 - Representação de bloco para o somador completo (*full adder* ou FAD).

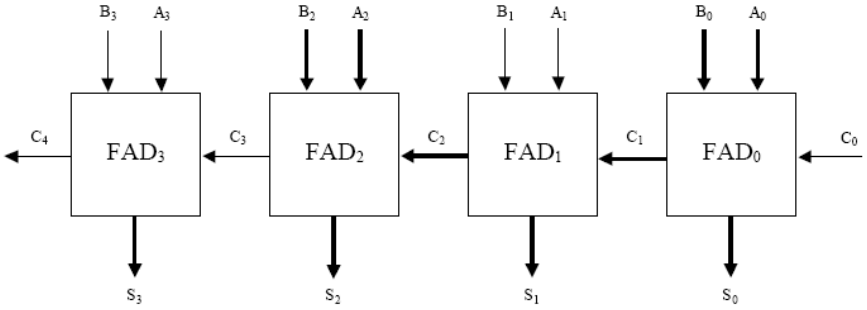


Figura 3.11- Somador paralelo de 4 bits.

Repare que o somador completo de índice zero, FAD_0, também possui uma entrada C_{in}, aqui denominada C_0. Como a priori não existe um valor de transporte a ser somado aos dígitos menos significativos, A_0 e B_0, esta entrada deverá estar constantemente ligada a zero. Já a saída de transporte Cout do dígito mais significativo, C_4 no caso do somador de 4 bits, serve para indicar se o resultado da adição entre A e B pode ser representado em 4 bits. Caso o resultado não pode ser representado em 4 bits, C_4 irá exibir o valor 1.

Repare também que, uma vez que um novo par de valores A e B é fornecido ao circuito somador, as últimas 2 saídas que se estabilizam são S_3 e C_4 , uma vez que estas dependem de C_3, que por sua vez depende da estabilização de C_2 e assim por diante. Desta forma, pode-se aproximar o atraso deste somador como sendo proporcional ao número de estágios (=número de somadores completos cascateados). Com efeito, a propagação do transporte ou *carry* ao

longo da cadeia de somadores é o ponto fraco deste tipo de somador.

Existem outros tipos de somadores capazes de operar mais rapidamente, mas que por razões de tempo não serão estudados nesta disciplina.

A construção de um somador para operar dois números binários de n bits requer o uso de n somadores completos, conectados segundo a mesma topologia mostrada na Figura 3.11.

É importante ressaltar que tal somador pode operar dois números inteiros quaisquer, positivos ou negativos, desde que ambos estejam representados em complemento de 2.

3.5.3 – O somador/subtrator

A subtração de dois números inteiros em binário pode ser feita utilizando-se a seguinte fórmula:

$$A - B = A + \overline{B} + 1$$

onde todas as operações são aritméticas, exceto $\overline{B}$, que representa a complementação de B, bit a bit.

A Figura 3.12 mostra um circuito somador/subtrator de 4 bits. Esse circuito é originado do somador paralelo de 4 bits, porém com a adição de portas xor nas entradas associadas a B, de modo a permitir a negação individual de cada bit de B. A tabela que segue mostra o funcionamento deste circuito, em função dos sinais de controle sel_1 e sel_2. Note que sel_1 coincide com C_0.

A exemplo do que ocorre com o somador paralelo apresentado na seção anterior, também o somador/subtrator pode operar dois números inteiros quaisquer, positivos ou negativos, desde que tais números estejam representados em complemento de dois. Caso os dois números a serem operados estivessem representados em sinal-magnitude, por exemplo, seria necessário existir um circuito para testar o

sinal de cada número e comparar as magnitudes, para só então realizar a soma ou a subtração. Como isso representaria a necessidade de um hardware mais complexo, e possivelmente mais caro e mais lento, a representação em complemento de dois é dominantemente utilizada nos computadores atuais.

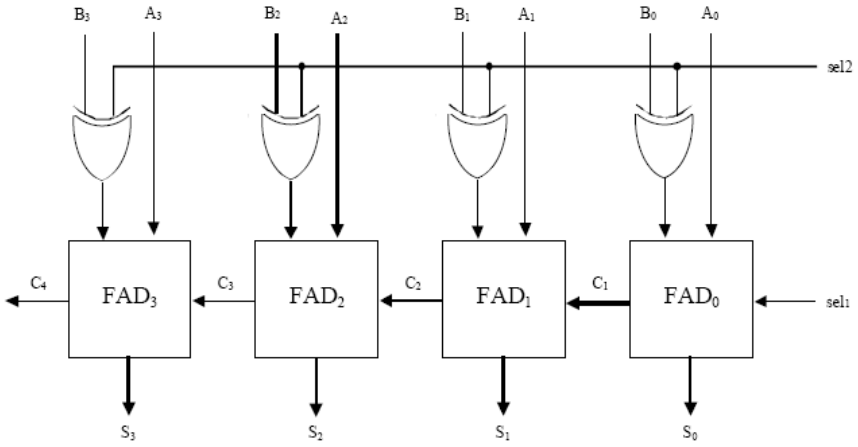


Figura 3.12 - Somador/subtrator de 4 bits.

Operações possíveis para o somador/subtrator da Figura 3.12.

sel_2	sel_1	operação	descrição
0	0	$S = A + B + 0$	adiciona A e B ($S = A + B$)
0	1	$S = A + B + 1$	adiciona A e B incrementado ($S = A + B + 1$)
1	0	$S = A + \overline{B} + 0$	subtrai B decrementado de A ($S = A - B - 1$)
1	1	$S = A + \overline{B} + 1$	subtrai B de A ($S = A - B$)

3.5.4 – O multiplicador

A multiplicação de números binários é realizada da mesma maneira como a de números decimais. O multiplicando é multiplicado por cada bit do multiplicador, começando do bit menos significativo. Cada uma destas multiplicações formam um produto parcial. Os sucessivos produtos parciais são deslocados uma posição para a esquerda. O produto final é obtido a partir da soma dos produtos parciais.

Para entender como um multiplicador binário pode ser implementado com um circuito combinacional, considere a multiplicação de dois números de dois bits mostrada na Figura abaixo:

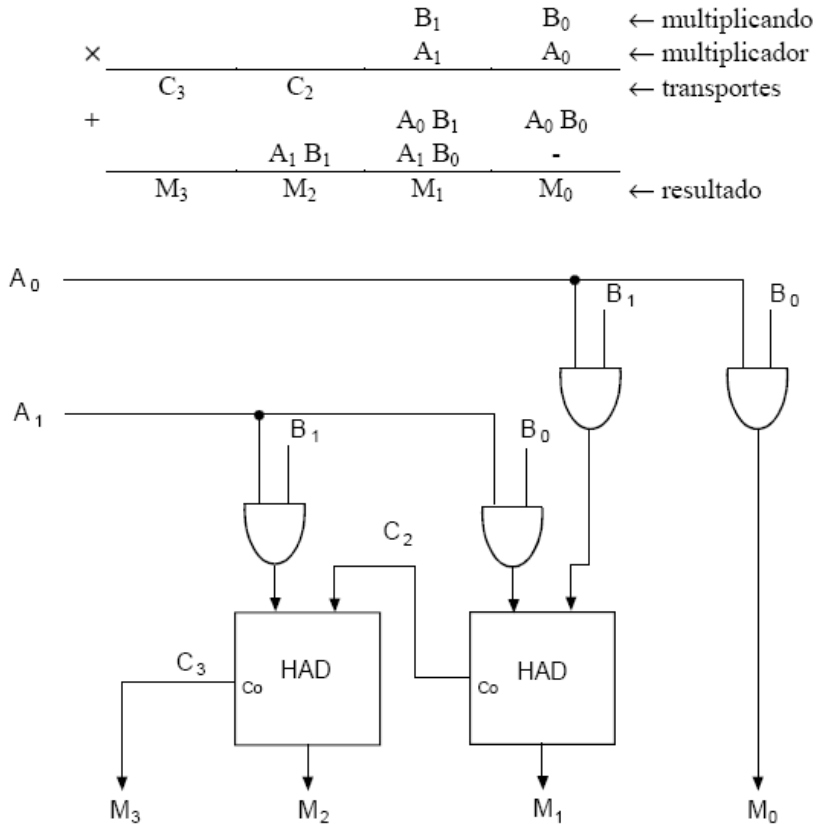


Figura 3.13 – Multiplicador de 2 bits.

Os bits do multiplicando são B_1 e B_0, os bits do multiplicador são A_1 e A_0 e o produto é $M_3M_2M_1M_0$. O primeiro produto parcial é formado pela multiplicação de B_1B_0 por A_0. A multiplicação de dois bits, tais como A_0 e B_0, produz um 1 se ambos os bits são 1, do contrário ela produz um 0. Isto é idêntico à operação E. Assim, o produto parcial pode ser implementado com portas E como mostrado no circuito da Figura 3.13. O segundo produto parcial é formado pela multiplicação de B_1B_0 por A_1 e é deslocado uma posição para a esquerda. Os dois produtos parciais são somados com dois circuitos meio-somadores. Usualmente tem-se mais bits nos produtos parciais, fazendo-se necessário o uso de somadores completos para produzir a soma dos produtos parciais.

Um circuito multiplicador binário combinacional com mais bits pode ser construído de maneira semelhante. Um

bit do multiplicador é operado por um E com cada bit do multiplicando em tantos níveis quanto existam bits no multiplicador. A saída binária em cada nível de portas E é somada em paralelo com o produto parcial do nível anterior para formar um novo produto parcial. O último nível produz o resultado.

Para j bits no multiplicador e k bits no multiplicando, serão necessários j×k portas E e (j-1) somadores de k bits para gerar um produto de j+k bits.

EXERCÍCIOS

Exercício 3.1 **-** Projetar um decodificador 3:8 com ativação em lógica negada (isto é, para cada saída D_i, se D_i=0, D_i está ativada, se D_i=1, a D_i está desativada).

Exercício 3.2 **-** Projetar um decodificador 2:4 com entrada de habilitação. Tanto a habilitação como a ativação das saídas deve se dar em lógica negada.

Exercício 3.3 **-** Reprojete o decodificador do exercício anterior utilizando somente portas NAND de 2 entradas.

Exercício 3.4 **-** Projete um decodificador 3:8 utilizando um inversor e 2 decodificadores 2:4 com entrada de habilitação.

Exercício 3.5 **-** Projetar um seletor 8-1 a partir de seletor 4-1.

Exercício 3.6 **-** Projetar um seletor 4-1, onde cada entrada é composta por um conjunto de 2 bits. Usar o símbolo para representar cada seletor 4-1, ao invés de desenhar o circuito detalhado.

Exercício 3.7 **-** Desenhe o circuito lógico de um multiplicador de quatro bits.

3.6 – SAIBA MAIS

Existem muitos bons textos e alguns deles estão listados na Bibliografia colocada ao final da Unidade 2. Outros estão na Internet à disposição . Estes estão listados a seguir.

3.7 – WEB-BIBLIOGRAFIA

www.ufpi.br/uapi (A Página da Universidade Aberta do Piauí - UAPI)

www.uab.gov.br (O Site da Universidade Aberta do Brasil-UAB)

www.seed.mec.gov.br (A Homepage da Secretaria de Educação a Distância do MEC - SEED)

www.abed.org.br (O site da Associação Brasileira de Educação a Distância - ABED)

3.8 REFERÊNCIAS BIBLIOGRÁFICAS

GAJSKI, Daniel D. Principles of Digital Design, New Jersey: Prentice Hall, 1997 (ISBN 0-13-301144-5)

MANO, M. Morris; Computer Engineering: Hardware Design. New Jersey: Prentice Hall, 1988 (ISBN 0-13-162926-3)

TAUB, H. Circuitos Digitais e Microprocessadores. McGraw-Hill, 1982.

BROWN, Stephen; VRANESIC, Zvonko. Fundamentals of Digital Logic with VHDL Design, McGraw-Hill Higher Education (a McGraw-Hill Company), 2000 (ISBN texto: 0-07-012591-0 CD parte da coleção: 0-07-235596-4)

ERCEGOVAC, Milos; LANG, Tomás; MORENO, Jaime H. Introdução aos Sistemas Digitais. Porto Alegre: Bookman, 2000 (ISBN: 85-7307-698-4)

KATZ, Randy H. Contemporary Logic Design. The Benjamin/Cummings Publishing Company, Inc. , 1994 (ISBN: 0-8053-2703-7)

Unidade 4

CIRCUITOS SEQUENCIAIS

Resumo

O objetivo principal desta unidade é apresentar os circuitos seqüenciais após serem apresentados os circuitos combinacionais na Unidade anterior. Ao contrário dos combinacionais, os circuitos seqüenciais não dependem unicamente dos valores de entrada. Eles dependem também dos valores anteriores que devem estar armazenados em algum circuito para que possam ter alguma utilidade, ou seja, devem ser armazenados em algum tipo de memória. Entre os circuitos seqüenciais estão os latches e os flip-flpos que são os elementos principais na construção dos diversos tipos de memórias que os computadores utilizam.

A forma de apresentação utilizada é de acordo com o exigido para o ensino à distância, ou seja, tendo em vista sempre esta nova modalidade de ensino.

SUMÁRIO

4 Circuitos seqüenciais.....90
4.1 Introdução.....91
4.2 Fundamentação teórica.....91
4.3 Latches.....96
4.3.1 O *latch* RS.....96
4.3.2 O latch RS controlado.....102
4.3.3 O *latch* D.....104
4.3.4 *Latches* com lógica de ativação complementar 106
4.4 Flip-flops.....108
4.4.1 *Flip-flop* D mestre–escravo.....109
4.4.2 Flip-flops disparados pela borda.....111
4.4.3 Flip-flops disparados pela borda descendente.. 114
4.4.4 *Set* e *reset* assíncronos.....115
4.5 SAIBA MAIS.....**Erro! Indicador não definido.**
4.6 WEB-BIBLIOGRAFIA.....117
4.7 REFERÊNCIAS BIBLIOGRÁFICAS.....117

4 – Circuitos seqüenciais

4.1 – Introdução

Conforme já citado no capítulo 3, os circuitos lógicos dos sistemas digitais podem ser de dois tipos: circuitos combinacionais ou circuitos seqüenciais. Um circuito combinacional é constituído de um conjunto de portas lógicas, as quais determinam os valores das saídas diretamente a partir dos valores atuais das entradas. Estes tipos de circuitos foram objeto de estudo no Capítulo 3 deste estudo. O objeto de estudo do Capítulo atual são os circuitos seqüenciais.

4.2 – Fundamentação teórica

A Figura 4.1 mostra o diagrama de blocos de um circuito seqüencial. Um circuito seqüencial é composto por um circuito combinacional e elementos de memória. As entradas e as saídas do circuito seqüencial estão conectadas somente ao circuito combinacional. Os elementos de memória são circuitos capazes de armazenar informação codificada em binário. Algumas das saídas do circuito combinacional são entradas para os elementos de memória, recebendo o nome de variáveis do próximo estado. Já as saídas dos elementos de memória constituem parte das entradas para o circuito combinacional e recebem o nome de variáveis do estado atual. As conexões entre o circuito combinacional e os elementos de memória conFiguram o que se costuma chamar laço de realimentação, pois a saída de um bloco é entrada para o outro e vice-versa.

A informação armazenada nos elementos de memória num dado instante determina o estado em que se encontra o circuito seqüencial. O circuito seqüencial recebe informação binária das entradas que, juntamente com a informação do estado atual, determinam os valores das saídas e os valores do próximo estado (vide Figura 4.1). Desta forma, fica evidente que as saídas de um circuito seqüencial dependem não apenas das entradas, mas também do estado atual, armazenado nos elementos de memória. E o mesmo pode ser dito para as variáveis de próximo estado. Em função deste comportamento seqüencial, um circuito seqüencial é

especificado pela seqüência temporal de entradas, saídas e estados internos.

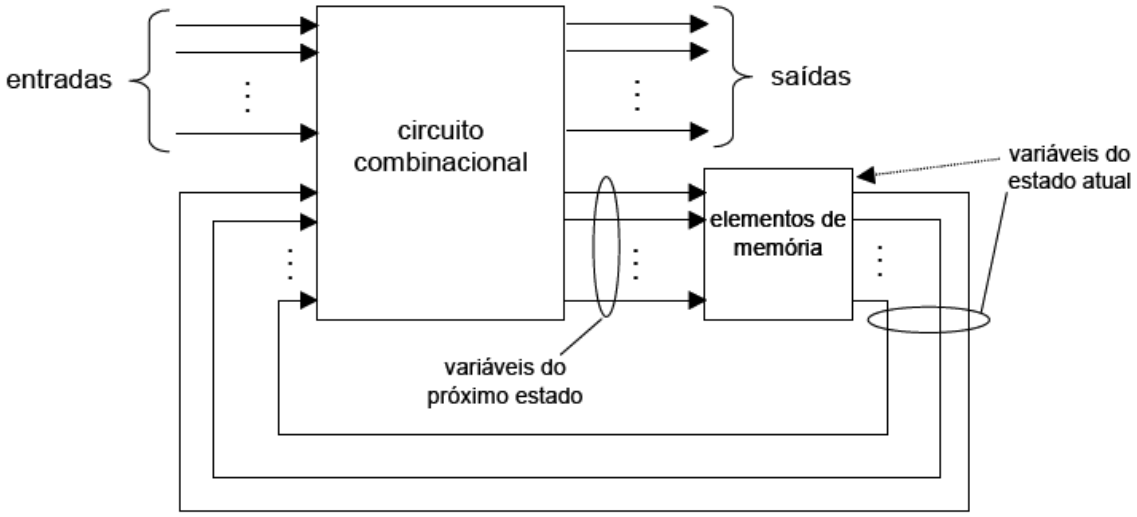


Figura 4.1 - Diagrama de blocos de um circuito seqüencial.

Os circuitos seqüenciais podem ser divididos em dois tipos, conforme o comportamento temporal dos seus sinais: síncronos e assíncronos.

O comportamento de um circuito seqüencial assíncrono depende da ordem segundo a qual as entradas mudam e o estado do circuito pode se alterar a qualquer tempo, como conseqüência de uma mudança de suas entradas. Os elementos de memória utilizados nos circuitos seqüenciais assíncronos apresentam uma capacidade de armazenamento que está associada diretamente ao atraso de propagação dos circuitos que os compõem. Em outras palavras, o tempo que esses circuitos levam para propagar uma mudança de suas entradas até suas saídas pode ser encarado como o tempo durante o qual eles retêm os valores aplicados antes da mudança, e esse fenômeno coincide com o conceito de memória, para os circuitos digitais. Nos circuitos seqüenciais assíncronos, os elementos de memória são compostos por portas lógicas que provêem um atraso de propagação com valor adequado para o funcionamento do circuito. Então, um circuito seqüencial assíncrono pode ser visto como um circuito combinacional com realimentação. O projeto de circuitos com realimentação apresenta grandes dificuldades, uma

vez que seu funcionamento correto é dependente das características temporais dos componentes (portas lógicas e fios). A principal dificuldade provém do fato de que os componentes apresentam atrasos que não são fixos, podendo ser diferentes mesmo para exemplares com mesma função e de um mesmo fabricante. Desta forma, os circuitos seqüenciais assíncronos têm sido evitados, sempre que possível, em favor do uso de circuitos seqüenciais síncronos.

Um circuito seqüencial síncrono utiliza um sinal especial denominado de relógio (clock, em inglês) o qual tem a função de cadenciar uma eventual troca de estado. A Figura 4.2 mostra um exemplo de sinal de relógio. A forma de onda de um sinal de relógio é dita monótona, pois não se altera ao longo do tempo. Nela podem ser identificados a borda de subida, a borda de descida, o nível lógico zero e o nível lógico um. O tempo que decorre para o sinal se repetir é denominado período e é representado por T. Por exemplo, o tempo entre duas bordas de subida sucessivas é igual a T. Da mesma forma, o tempo entre duas bordas de descida sucessivas é igual a T.

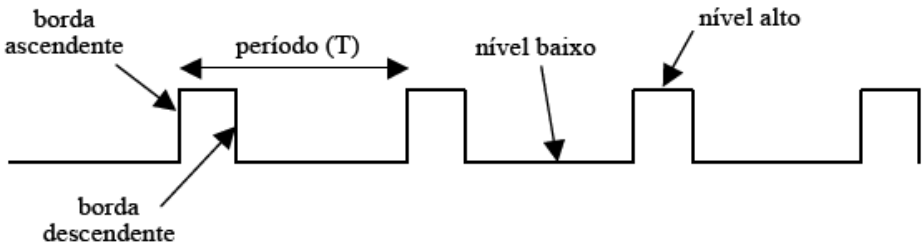


Figura 4.2 - Exemplo de sinal de relógio (clock).

A freqüência de um sinal de relógio, representada por f, é definida como sendo o inverso do período, ou seja:

$$f = \frac{1}{T} \qquad (4.1)$$

Para medir-se o período, usa-se os múltiplos do segundo: ms (milissegundo = 10-3s), μs (microssegundo = 10-

6s), ns (nanossegundo = 10-9s) e ps (picossegundo = 10-12s). Para medir-se a freqüência, usa-se os múltiplos do hertz: kHz (quilohertz = 10+3Hz), MHz (megahertz =10+6Hz) e GHz (gigahertz = 10+9Hz). Um hertz equivale a 1/s (i.e., o hertz é o inverso do segundo).

Exemplo 4.1: um circuito digital síncrono é cadenciado pelo uso de um sinal de relógio de 200 MHz. Qual é o maior atraso permitido para um circuito combinacional qualquer dentro deste circuito?

Ora, se esse circuito deve trabalhar à freqüência de 200 MHz, então, cada um de seus blocos combinacionais deve ter um atraso inferior ao período do relógio, o qual pode ser calculado por:

$$T = \frac{1}{200x10^{6}\,Hz} = 0{,}005x10^{-6}\,s = 5x10^{-9}\,s = 5ns$$

Num circuito seqüencial síncrono, o sinal de relógio determina quando os elementos de memória irão amostrar os valores nas suas entradas. Conforme o tipo de circuito utilizado como elemento de memória, esta amostragem das entradas pode ser sincronizada pela borda ascendente ou pela borda descendente do relógio. Seja qual for o tipo de sincronização, o tempo que transcorre entre duas amostragens sucessivas equivale a T, o período do relógio. Isto implica que, qualquer mudança no estado de um circuito seqüencial síncrono irá ocorrer somente após a borda do sinal de relógio na qual seus elementos de memória são disparados. A Figura 4.3 mostra o diagrama de blocos de um circuito seqüencial síncrono.

Os elementos de memória utilizados nos circuitos seqüenciais síncronos são denominados flip-flops. Um flip-flop é um circuito digital que possui duas entradas e duas saídas e é capaz de armazenar um bit de informação. As duas entradas não são intercambiáveis: uma é reservada ao sinal de controle (relógio) e a outra recebe o dado (bit) a ser ar-

mazenado. As saídas correspondem ao dado (bit) armazenado e ao seu complemento. O sinal de relógio determina o instante em que o flip-flop amostra o valor do dado, podendo corresponder a uma borda de subida ou a uma borda de descida, dependendo de como o flip-flop é constituído. O diagrama da Figura 4.3 mostra que o valor de cada variável de estado é armazenado num flip-flop específico. Os valores que representam o próximo estado só são amostrados na borda ativa do relógio. Logo, o estado atual fica armazenado no conjunto de flip-flops até que uma nova borda do relógio chegue, quando então o próximo estado passa a ser o estado atual e um novo próximo estado será gerado pelo circuito combinacional.

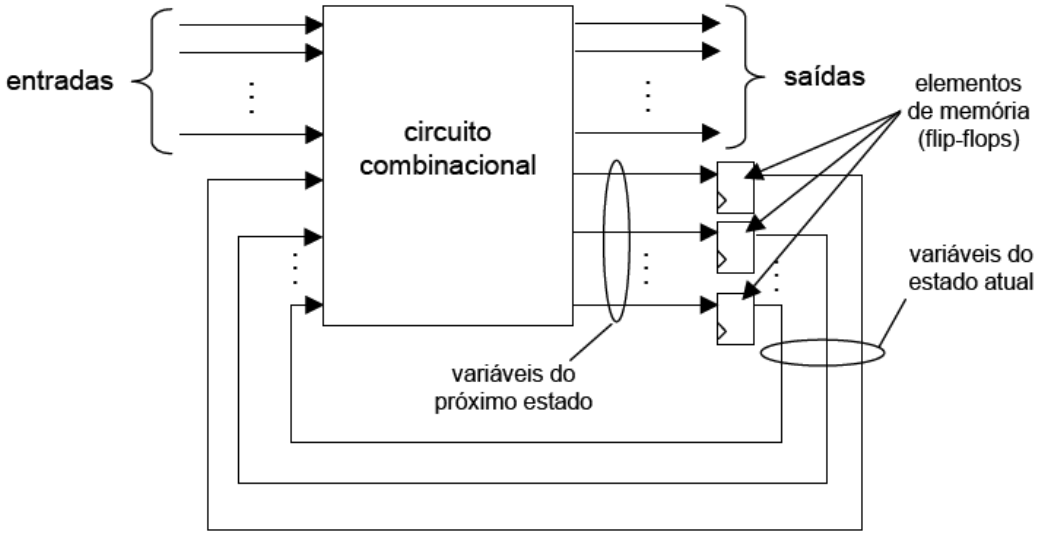


Figura 4.3 - Diagrama de blocos de um circuito seqüencial síncrono.

Desde que devidamente alimentado com energia, um flip-flop pode manter indefinidamente um estado, até que os sinais de entrada assumam uma conFiguração tal que o façam mudar de estado. Essa conFiguração depende de como o flip-flop é constituído. O estado em que um flip-flop se encontra usualmente é associado ao valor binário que ele está armazenando. Desta forma, num dado instante, um flip-flop estará armazenando ou o valor lógico 1 (um) ou o valor lógico 0 (zero), pois esses são os dois valores possíveis para uma variável Booleana.

4.3 – Latches

Os vários *flip-flops* existentes se diferenciam pelo número de entradas que possuem e na maneira pela qual tais entradas afetam o estado em que o flip-flop se encontra. Os tipos mais básicos de flip-flops são denominados latches. Os latches operam por níveis dos sinais de entrada (diz-se que são sensíveis a nível) e servem como base na construção dos flip-flops mais sofisticados.

Apesar de serem capazes de armazenar informação binária, os *latches* são pouco utilizados na construção de circuitos seqüenciais síncronos por serem menos práticos do que os *flip-flops*. A seguir, serão estudados o *latch* RS, o *latch* RS controlado e o *latch* D.

4.3.1 – O latch RS

O *latch* RS é o *latch* mais simples que existe. Ele pode ser construído com o uso de duas portas nor de 2 entradas cada, conectadas conforme mostra a Figura 4.4. Note que há

duas entradas, chamadas R e S, e duas saídas, Q e Q. Note também que existe uma conexão entre a saída Q e a outra entrada da nor n2. Existe também uma conexão entre a saída Q e a outra entrada da nor n1. Conexões entre saída e entrada são denominadas realimentações, e no caso de circuitos digitais, são responsáveis pela propriedade de armazenamento apresentada pelo circuito.

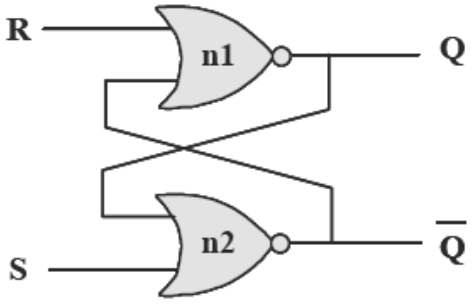


Figura 4.4 - *Latch* RS com portas nor.

Conforme já citado na introdução deste capítulo, circuitos que possuem algum tipo de realimentação são ditos seqüenciais, pois seu comportamento não depende somente dos valores das entradas, mas também do estado em que o circuito se encontra. Assim, a análise do funcionamento do *latch* RS obedecerá os seguintes passos:

1. Identificação de uma combinação de entradas capaz de determinar o estado do *latch* de maneira independente do estado anterior (se isso for possível)

2. Assumindo o estado determinado no passo 1 como sendo o estado inicial, aplicação de uma nova combinação de entradas para verificar como o circuito se comporta (se muda de estado ou não);

Repetição dos passos 1 e 2 para cada combinação de entradas capaz de determinar o estado do circuito de maneira independente.

A partir do procedimento anterior encontrar-se-á uma tabela de comportamento denominada tabela de transição de estados (ou simplesmente, tabela de transição), a qual é característica deste *latch*. Em particular, cada *latch* e cada *flip-flop* possui um comportamento que pode ser expresso em termos de uma tabela de transferência que lhe é própria.

Para o *latch* RS da Figura 4.4, imaginemos que sejam aplicados simultaneamente os valores 1 e 0 às entradas R e S, respectivamente, no instante de tempo t0. Ora, sabemos que o valor 1 aplicado a qualquer uma das entradas de uma porta nor determina o valor da saída desta porta como sendo 0, independente dos valores das demais entradas. Logo, se for aplicado R=1 e S=0 em t_0, a saída Q se estabilizará com valor 0 em $t_0+t_d(n1)$, onde $t_d(n1)$ é o atraso da porta nor n1. Como existe uma ligação física (ou seja, um fio) entre Q e uma das entradas da porta nor n2, a partir do tempo $t_0+t_d(n1)$ ambas entradas desta porta estarão estabilizadas em 0. Então, a partir do tempo $t_0+t_d(n1)+t_d(n2)$, onde $t_d(n2)$ é

o atraso da porta nor n2, a saída Q estará estabilizada com o valor lógico 1.

Imaginemos agora que na seqüência de operação deste latch foram aplicados os valores R=0 e S=0 às suas entradas no instante de tempo t_1, com $t_1>t_0+t_d(n1)+t_d(n2)$ (ou seja, bem depois da aplicação de R=1 e S=0). Em função dos atrasos das portas n1 e n2, as saídas Q e Q não se alterarão imediatamente. Logo, para efeitos de análise, podemos considerar que a entrada de n1 que está conectada a Q continua com o valor lógico 1 e que a entrada de n2 que está conectada a Q continua com o valor lógico 0. Desta forma, logo após o instante t_1, n1 terá 0 e 1 em suas entradas, fazendo com que sua saída, que é a saída Q do circuito, permaneça no valor lógico 0. De maneira semelhante, logo após t_1, n2 terá em suas entradas 0 e 0, fazendo com que sua saída, que é a saída Q do latch, permaneça com o valor lógico 1. As formas de onda que ilustram o resultado da aplicação sucessiva destes dois "vetores de entrada" (R=1;S=0) e (R=0;S=0) no latch RS são mostradas na Figura 4.5.

Suponhamos agora que a seqüência de valores aplicados às entradas do latch é (R=0;S=1) em t_0 e (R=0;S=0), em t_1. Então, em $t_0+t_d(n2)$ a saída Q se estabilizará com o valor lógico 0. Como existe uma ligação física entre a saída Q e uma das entradas da porta nor n1, após o instante $t_0+t_d(n2)$ ambas entradas de n1 estarão estabilizadas em 0. Então, a partir do instante $t_0+t_d(n2)+t_d(n1)$, a saída Q estará estabilizada com o valor lógico 1. Supondo novamente que $t_1>t_0+t_d(n2)+t_d(n1)$, podemos admitir que imediatamente após t1 as saídas Q e Q ainda se mantêm com seus valores anteriores, quais sejam Q=1 e Q =0. Desta forma, n1 terá o valor lógico 0 em ambas entradas, resultando que Q se mantém em 1. De forma similar, n2 terá em suas entradas os valores 1 e 0, resultando que Q se mantém em 0. A Figura 4.6 mostra as formas de onda resultantes da aplicação do vetor de entrada (R=0;S=1) em t0, seguido do vetor (R=0;S=0), em t_1. Note que $t_d(n1)$ e $t_d(n2)$ podem ser valores bem diferentes. Note ainda que em ambos casos, o atraso para a estabilização do latch sempre será $t_d(n1)+t_d(n2)$.

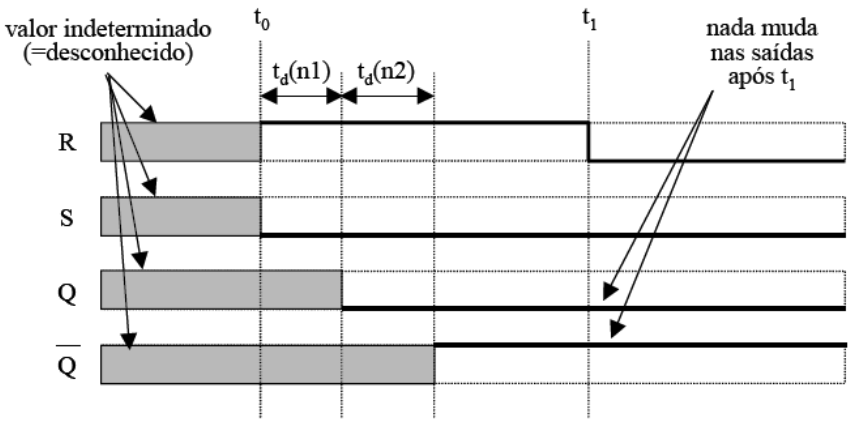


Figura 4.5 -Formas de onda para aplicação do vetor de entrada (R=1;S=0) seguido do vetor (R=0;S=0) no latch RS.

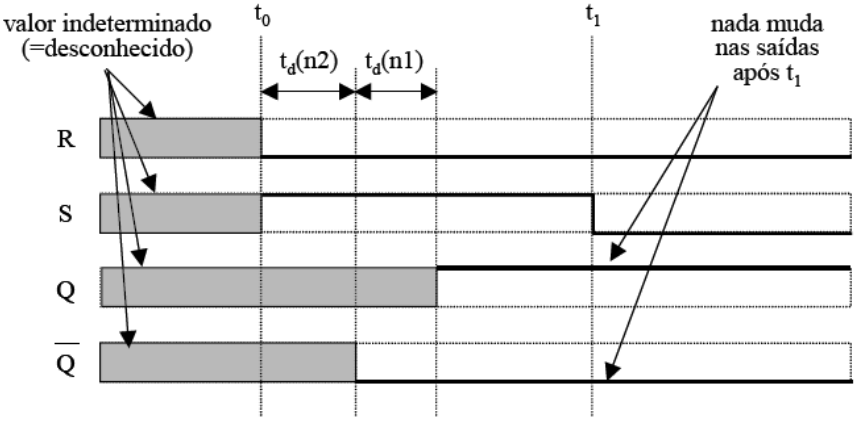


Figura 4.6 -Formas de onda para aplicação do vetor de entrada (R=0;S=1) seguido do vetor (R=0;S=0) no latch RS.

Note que para todas as situações estudadas até aqui, os valores exibidos pelas saídas Q e $\overline{Q}$ são sempre complementares. É justamente por esse motivo que elas recebem essas denominações. Entretanto, se aplicarmos o vetor de entrada (R=1;S=1), ambas saídas se estabilizarão em 1, o que conflita com o que foi colocado anteriormente. Ora, se um *latch* deve ser capaz de armazenar um dentre os dois estados possíveis para uma variável Booleana e se o estado está associado ao valor de Q e $\overline{Q}$ (Q exibe o estado e $\overline{Q}$, o seu complemento), então qual seria o estado representado pela situação Q=1 e $\overline{Q}$ =1? Por não haver uma resposta plausível a essa pergunta, foi convencionado que esse seria um estado proibido (ou indeterminado), de modo que a situação (R=1;S=1) deve sempre ser evitada, no caso do *latch* RS.

Conforme já mencionado na introdução dessa seção, um latch, assim como um *flip-flop*, pode assumir um dentre dois estados possíveis. Esses estados correspondem aos valores que uma variável Booleana pode assumir, ou seja, 0 e 1. O estado 0 também é chamado estado *reset* e o estado 1 é também chamado estado *set*.

Analisando-se a situação mostrada pelas formas de onda da Figura 4.5, conclui-se que a aplicação do vetor (R=1;S=0) faz com que o *latch* vá para o estado *set* (i.e., a saída Q estabiliza com o valor lógico 1), independente de seu estado anterior. Se após isso for aplicado o vetor (R=0;S=0), o *latch* não muda o seu estado. Avaliando-se agora as formas de onda da Figura 4.6, conclui-se que a aplicação do vetor (R=0;S=1) faz com que o *latch* vá para o estado *reset* (i.e., a saída Q estabiliza com o valor lógico 0), independente de seu estado anterior. Se após isso for aplicado o vetor (R=0;S=0), o *latch* não muda o seu estado. Finalmente, pode-se afirmar que a aplicação do vetor (R=0;S=0) não muda o estado em que o latch está. Por outro lado, o vetor (R=1;S=1) deve ser evitado, por conduzir a um estado proibido. Essas informações podem ser resumidas pela tabela que segue:

Tabela 4.1 - Resumo do funcionamento seqüencial do latch RS.

R	S	Q	$\overline{Q}$	ação
1	0	0	1	vai para **estado reset**
0	0	0	1	mantém **estado reset** (= estado anterior)
0	1	1	0	vai para **estado set**
0	0	1	0	mantém **estado set** (= estado anterior)
1	1	0	0	**estado proibido**

A tabela anterior pode ser escrita de maneira mais compacta, de modo a incorporar a informação da dependência temporal.

Tabela 4.2 - Tabela de transição de estados para o latch RS.

R	S	Q_{t+1}	comentário
0	0	Q_t	mantém estado anterior
0	1	1	estado set
1	0	0	estado reset
1	1	-	proibido

A Tabela 4.2 lista os valores possíveis para as entradas nas colunas mais à esquerda, admitindo que esses valores estão sendo aplicados no instante presente t. Para cada situação de entradas, o novo valor da saída (e portanto, o novo estado do *latch*) para o instante imediatamente posterior t+1 encontra-se na coluna mais à direita. Como a saída Q sempre exibe o complemento da saída Q, apenas o valor de Q é listado, ficando Q subentendido.

O comportamento de circuitos seqüenciais pode também ser expresso por meio de um diagrama denominado diagrama de estados. Sendo o latch RS um circuito seqüecial, pode-se usar um diagrama de estados para representar seu funcionamento, conforme mostrado na Figura 4.7.

No diagrama da Figura 4.7, os estados reset e set estão representados por nodos (círculos). A transição entre estados é mostrada por uma aresta (seta). A condição de entradas segundo a qual uma determinada transição pode ocorrer está definida junto a aresta respectiva. Por exemplo, estando o *latch* RS no estado *reset,* para que ele vá para o estado *set* é necessário que R=0 e S=1. Caso R=0 e S=0, o *latch* RS ficará no estado em que se encontra.

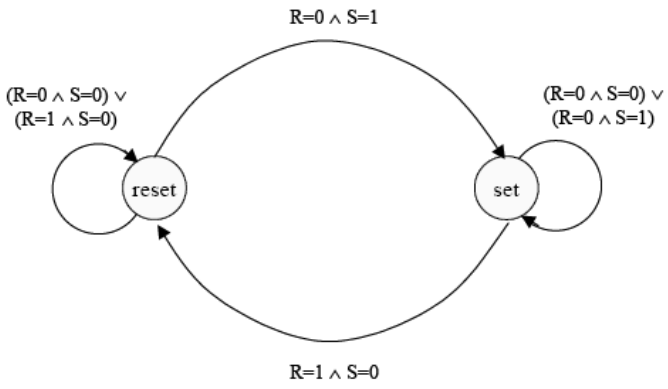


Figura 4.7 - Diagrama de estados para o *latch* RS.

Para evitar que se tenha que desenhar o circuito completo toda a vez que houver uma ocorrência do *latch* RS, costuma-se adotar o símbolo mostrado na Figura 4.8.

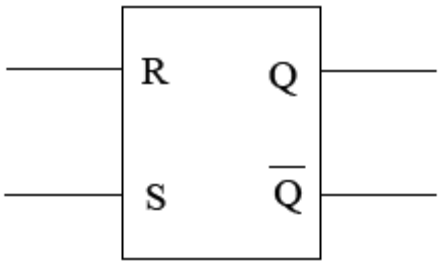
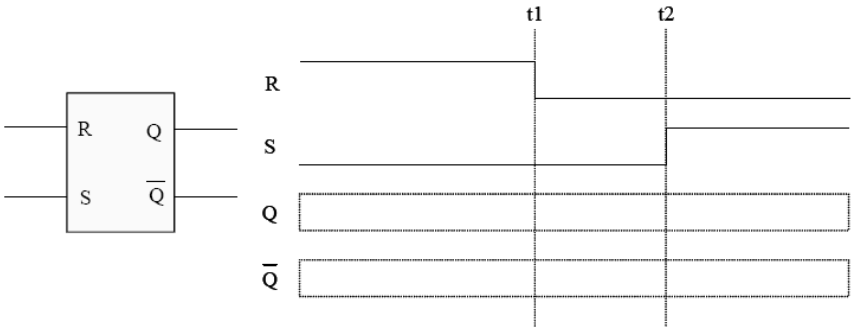


Figura 4.8 - Símbolo do *latch* RS.

Exemplo 4.2: desenhar as formas de onda para as saídas do *latch* RS abaixo, a partir das formas de onda fornecidas para as entradas R e S.



4.3.2 – O latch RS controlado

No *latch* RS, cujo funcionamento foi descrito na subseção 4.1.1, uma alteração das entradas R e S pode acarretar uma troca de estado. Porém, em alguns casos pode ocorrer que os sinais conectados às entradas R e S sofram variações não desejadas, sendo válidos somente em alguns intervalos de tempo bem determinados. Nesse caso, seria interessante que houvesse uma entrada de maior prioridade que fosse encarregada de controlar a habilitação do latch, deixando-o sensível ou não aos valores das entradas R e S.

Nesse sentido, o *latch* RS controlado é um aprimoramento do *latch* RS. Ele é construído a partir do *latch* RS, pela colocação de um par de portas E nas entradas R e S,

conforme mostra a Figura 4.9. A entrada C tem o objetivo de habilitar ou desabilitar o *latch* RS: caso C=0, o *latch* mantém o estado, pois R1=0 e S1=0; caso C=1, o *latch* funciona normalmente, segundo a Tabela 4.2. A tabela de transição desse *latch* é mostrada na Tabela 4.3. Note que se C=0, o *latch* mantém seu estado, independente dos valores de R e S (os X indicam essa independência). Repare também que há ainda outra situação em que o *latch* mantém o estado, qual seja, quando C=1, mas R=0 e S=0.

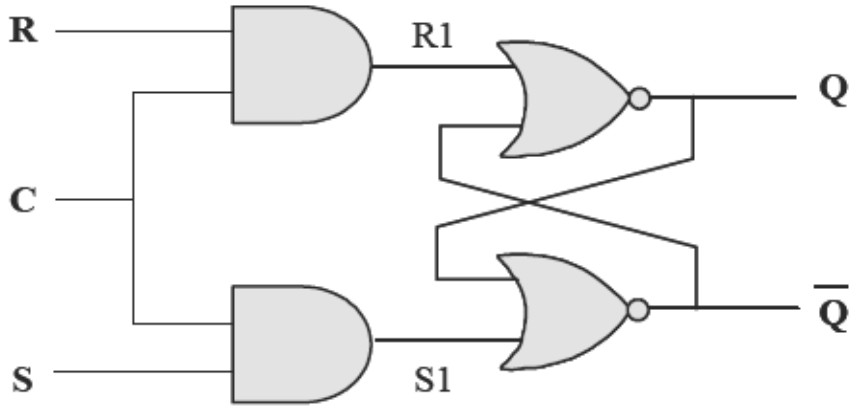


Figura 4.9 -*Latch* RS controlado.

Tabela 4.3 - Tabela de transição de estados para o latch RS controlado.

C	R	S	Q_{t+1}	comentário
0	X	X	Q_t	mantém estado anterior
1	0	0	Q_t	mantém estado anterior
1	0	1	1	estado set
1	1	0	0	estado reset
1	1	1	-	proibido

O diagrama de estados para o *latch* RS controlado é muito semelhante ao diagrama do *latch* RS, conforme mostra a Figura 4.10. Apenas as condições para troca ou manutenção de estado são diferentes: no caso do *latch* RS controlado, as condições são compostas. Por exemplo, para que o *latch* RS controlado se mantenha num mesmo estado é necessário que C=0 ou que C=1 e R=0 e S=0.

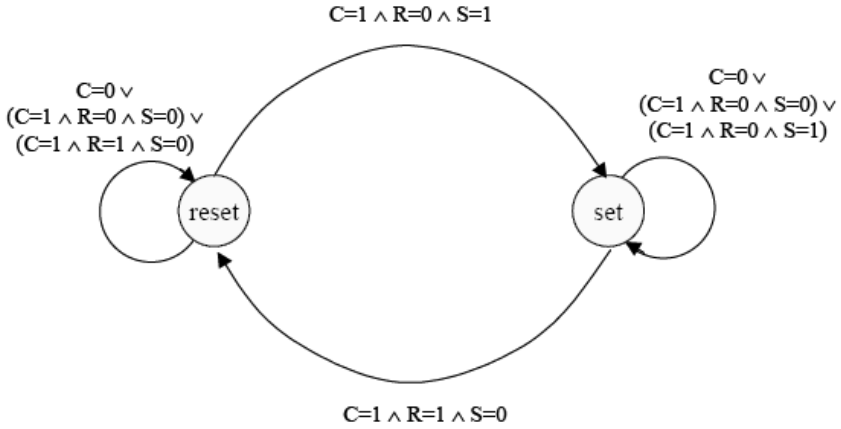


Figura 4.10 -Diagrama de estados para o *latch* RS controlado.

A Figura 4.11 mostra o símbolo do latch RS controlado.

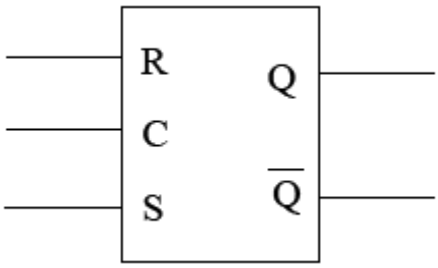
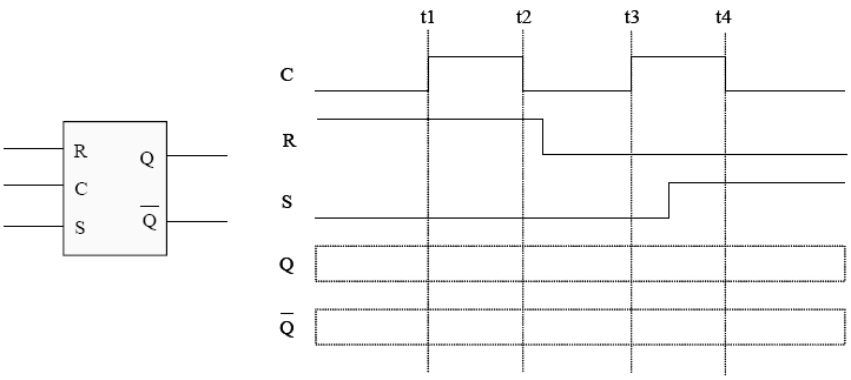


Figura 4.11 - Símbolo do latch RS controlado.

Exemplo 4.3. Desenhar as formas de onda para as saídas do *latch* RS abaixo, a partir das formas de onda fornecidas para as entradas C, R e S.



4.3.3 – O latch D

A necessidade de evitar a ocorrência do estado proibido é um detalhe que dificulta o projeto de circuitos seqüenciais com latches RS. O latch D é construído a partir do latch

RS, de maneira tal que, pela colocação de um inversor entre as entradas S e R, fica assegurado que nunca ocorrerá a situação de entradas R=1 e S=1, responsáveis pelo surgimento do estado proibido (Figura 4.12). Desta forma, a tabela de transição do latch D pode ser derivada da tabela do latch RS controlado, onde as entradas R e S passam a ser a entrada D (com D=S). Duas combinações de entradas desaparecem: uma que resultava na manutenção do estado e outra que resultava no estado proibido. A tabela de transição do *latch* D é mostrada na tabela 4.3 e seu símbolo, na Figura 4.13.

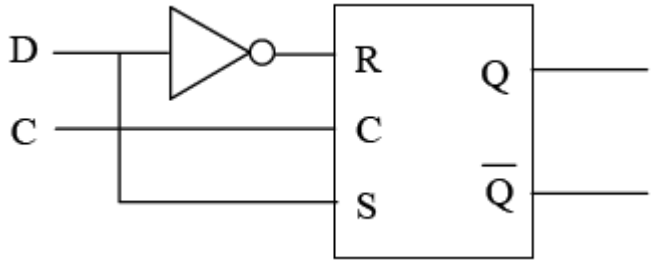


Figura 4.12: *latch* D.

Tabela 4.4 - Tabela de transição de estados para o latch D.

C	D	Q_{t+1}	comentário
0	X	Q_t	mantém estado anterior
1	0	0	estado reset
1	1	1	estado set

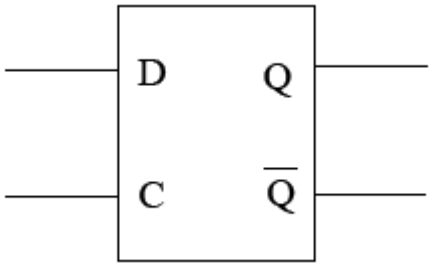
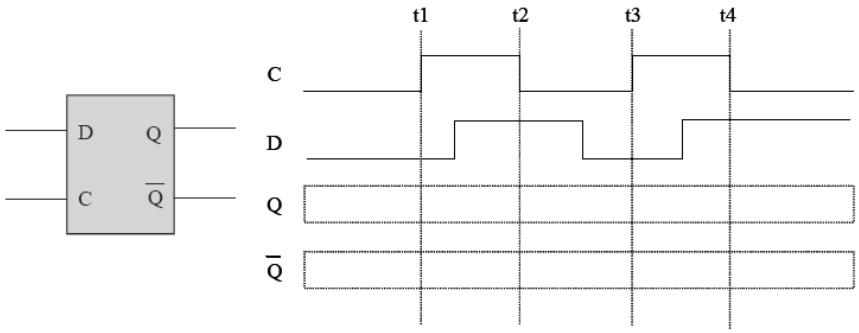


Figura 4.13 - Símbolo do *latch* D.

Exemplo 4.4. Desenhar as formas de onda para as saídas do *latch* D abaixo, a partir das formas de onda fornecidas para as entradas.



4.3.4 *Latches* com lógica de ativação complementar

Os *latches* vistos até aqui apresentam lógica de ativação direta, isto é, estão ativados enquanto o controle estiver no nível lógico 1 e desativados enquanto o controle estiver no nível lógico 0. É possível inverter-se essa lógica de ativação pela simples inserção de um inversor antes da entrada de controle. Assim, um latch com lógica de ativação complementar (ou negada ou invertida) está ativado enquanto o controle vale 0 e desativado enquanto o controle vale 1. A Figura 4.14 mostra os símbolos do latch RS controlado e do latch D, ambos com lógica de ativação complementar. Repare que a indicação da lógica de ativação complementar é feita por meio de um círculo colocado antes da entrada de controle.

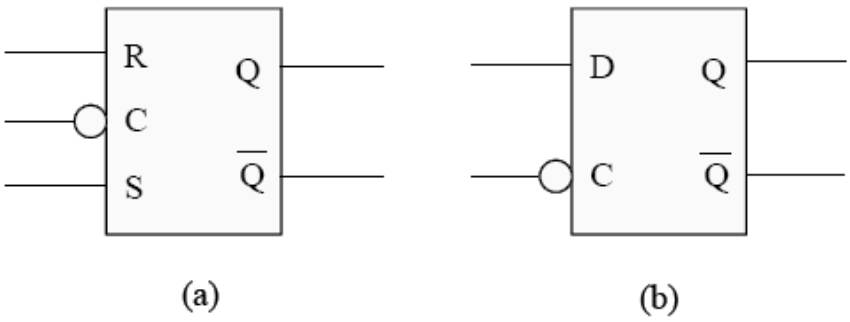


Figura 4.14 - Símbolo do *latch* RS controlado (a) e do *latch* D (b), ambos com lógica de ativação complementar.

As Tabelas 4.5 e 4.6 mostram o funcionamento destes *latches* com lógica de ativação negada. Comparando-se com as tabelas de transição dos latches correspondentes com lógica de ativação direta, nota-se que as ações são as

mesmas; apenas o que muda é o nível do sinal de controle necessário para ativá-los.

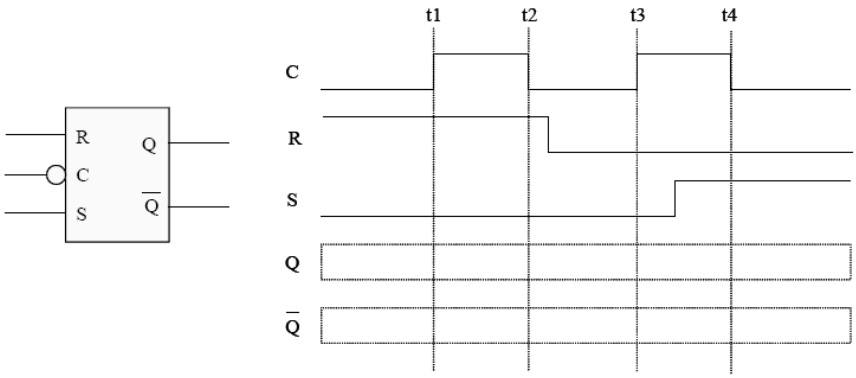
Tabela 4.5 - Tabela de transição de estados para o *latch* RS controlado com lógica de ativação negada.

C	R	S	Q_{t+1}	comentário
1	X	X	Q_t	mantém estado anterior
0	0	0	Q_t	mantém estado anterior
0	0	1	1	estado set
0	1	0	0	estado reset
0	1	1	-	proibido

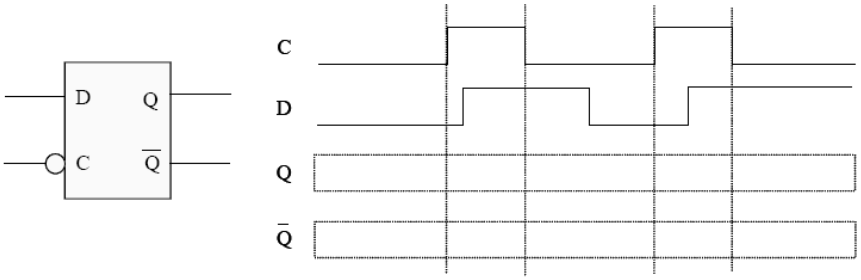
Tabela 4.6 - Tabela de transição de estados para o *latch* D com lógica de ativação negada.

C	D	Q_{t+1}	comentário
1	X	Q_t	mantém estado anterior
0	0	0	estado reset
0	1	1	estado set

Exemplo 4.5. Desenhar as formas de onda para as saídas do *latch* RS abaixo, a partir das formas de onda fornecidas.



Exemplo 4.6 Desenhar as formas de onda para as saídas do latch D abaixo, a partir das formas de onda fornecidas.



4.4 – Flip-flops

Os latches e os flip-flops são muito parecidos. Qual a verdadeira diferença entre eles?

Conforme visto na seção anterior, os latches controlados D e RS são ativados ou controlados pelo nível lógico do sinal de controle. Isso significa que, enquanto o sinal de controle estiver ativando o latch, eventuais variações das entradas D ou R e S serão percebidas pelo latch e este poderá mudar de estado. Essa característica é particularmente imprópria para a construção de circuitos seqüenciais síncronos, uma vez que em tais circuitos qualquer troca de estado deve ocorrer de maneira sincronizada com o sinal de relógio.

Os *flip-flops* são circuitos derivados dos latches, porém ativados pela transição do sinal de controle (i.e., pela borda). Isso faz com que um flip-flop permaneça ativado apenas durante um intervalo de tempo muito pequeno, após a ocorrência de uma transição do sinal de controle. Assim, uma eventual troca de estado só pode ocorrer durante esse breve intervalo de tempo em que o flip-flop está ativado. Entre duas transições sucessivas do mesmo tipo (ou subida ou descida) do sinal de controle, o flip-flop mantém o último estado adquirido.

Dependendo de sua construção, um flip-flop pode ser disparado pela transição de subida ou pela transição de descida do sinal de controle. Diz-se então, que flip-flops são disparados pela borda (ascendente ou descendente, conforme for o caso), enquanto que latches são sensíveis ao nível lógico (alto ou baixo, conforme for o caso). A seguir, serão estudados os flip-flops mais utilizados.

4.4.1 – *Flip-flop* D mestre–escravo

O *flip-flop* D mestre-escravo é composto por dois latches D conectados em cascata, conforme mostra a Figura 4.15: o primeiro é chamado de mestre e o segundo é chamado de escravo. O sinal de controle externo está conectado diretamente ao controle do *latch* mestre e ao inversor cuja saída está conectada ao controle do *latch* escravo.

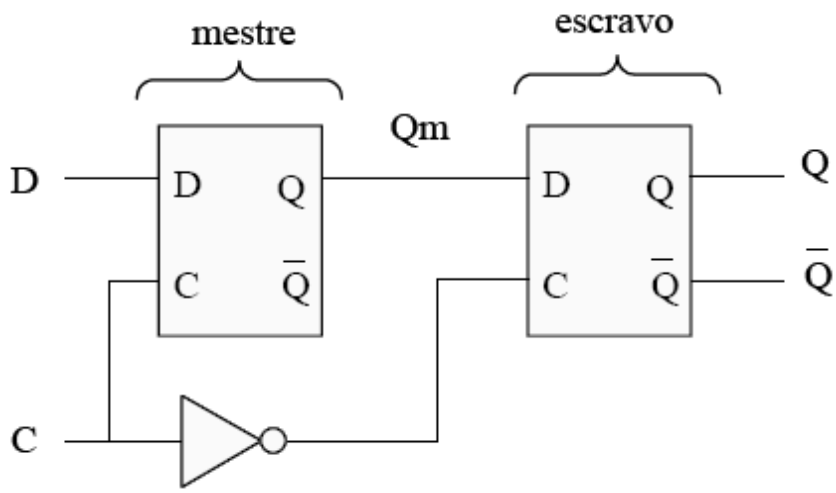


Figura 4.15 - *Flip-flop* D mestre-escravo.

Analisando-se as conexões, é possível deduzir facilmente que os dois *latches* funcionam de maneira complementar com relação ao sinal de controle externo: enquanto o controle vale 1, o mestre está ativado e o escravo está mantendo seu estado anterior e enquanto o controle vale 0, o mestre está mantendo seu estado anterior e o escravo está ativado. Como a entrada do escravo está conectada à saída do mestre, o último valor lido durante a ativação do mestre aparecerá na saída do escravo no semiperíodo seguinte. A Figura 4.16 exemplifica o funcionamento do flip-flop D mestre-escravo a partir de formas de onda arbitrárias para as entradas C e D.

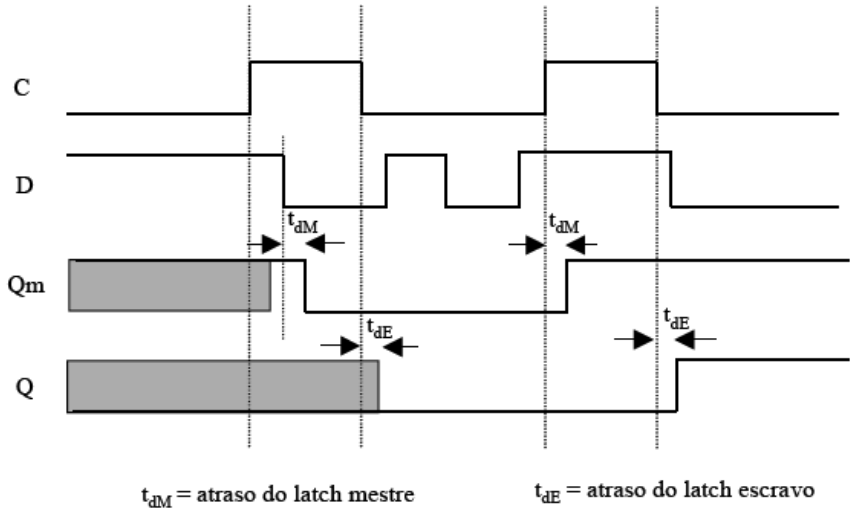
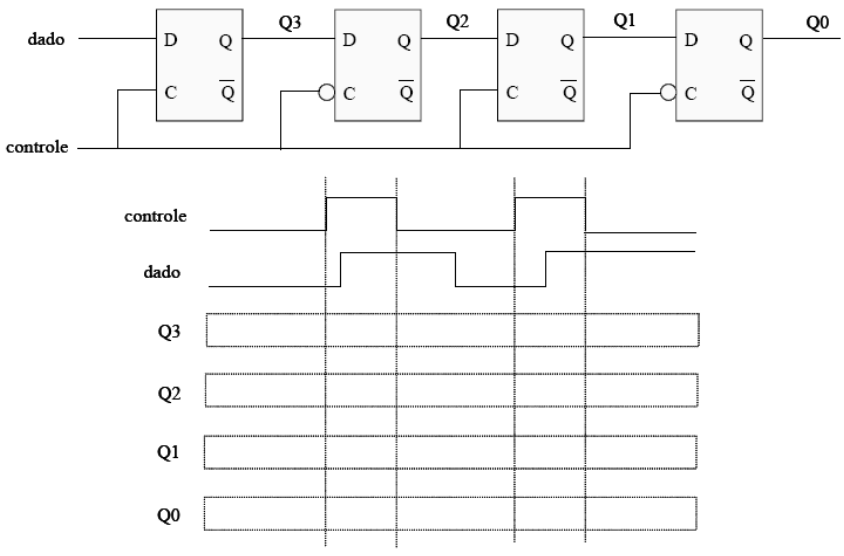


Figura 4.16 - Exemplo do funcionamento do flip-flop D mestre-escravo.

Do ponto de vista externo, o flip-flop D mestre-escravo da Figura 4.15 funciona como se fosse disparado pela borda descendente do sinal de controle: o último valor de D amostrado pelo latch mestre antes da borda descendente fica armazenado, aparecendo na saída Q do latch escravo logo após a mesma borda descendente.

Exemplo 4.7. Traçar as formas de onda para as saídas de cada um dos latches do circuito que segue, a partir das formas de onda fornecidas.



4.4.2 – Flip-flops disparados pela borda

Um flip-flop disparado pela borda (também referenciado por sensível à borda) ignora o sinal de controle enquanto este se encontra estável num dos dois níveis lógicos. Porém, quando o sinal de controle passa por uma transição, o flip-flop disparado pela borda fica ativado por um breve instante durante o qual as entradas podem (ou não) determinar a troca de seu estado. Dependendo da maneira como é construído, o flip-flop será disparado ou somente pela borda ascendente ou somente pela borda descendente. A Figura 4.17 mostra o circuito de um flip-flop D disparado pela borda ascendente, feito com portas nand de duas entradas.

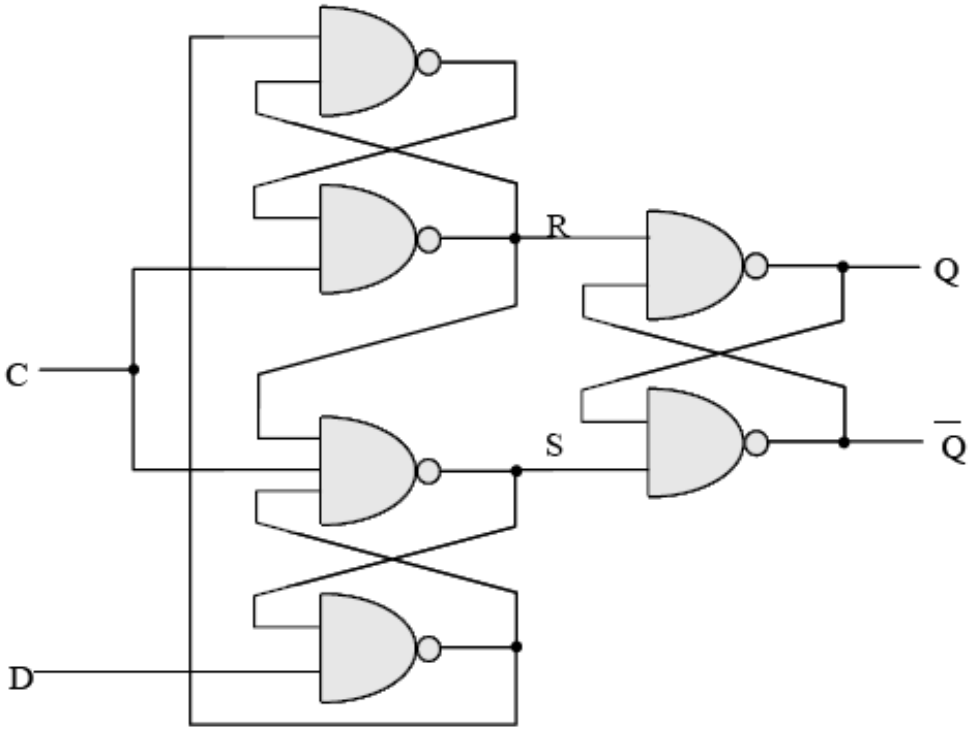


Figura 4.17 - Flip-flop D disparado pela borda ascendente.

A tabela de transição de um flip-flop D disparado pela borda ascendente é mostrada a seguir

Tabela 4.7 - Tabela de transição de estados para o flip-flop D disparado pela borda ascendente.

C	D	Q_{t+1}	comentário
$\neq\uparrow$	X	Q_t	mantém estado anterior
$\uparrow$	0	0	estado reset
$\uparrow$	1	1	estado set

Na tabela anterior, o símbolo ↑indica que a ativação do flip-flop é instantânea e só ocorre durante as bordas ascendentes do sinal de controle C. Por outro lado, entre duas bordas ascendentes consecutivas do sinal de controle, o flip-flop mantém o estado anteriormente armazenado. O símbolo do flip-flop D é mostrado na Figura 4.18; o triângulo colocado na entrada de controle C indica que a ativação se dá pela borda ascendente (e não pelo nível lógico, como ocorre no latch D).

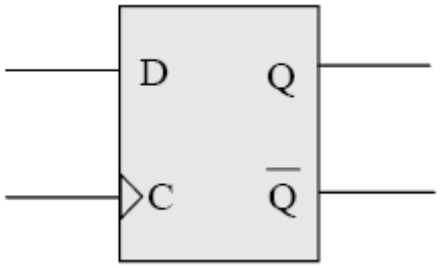
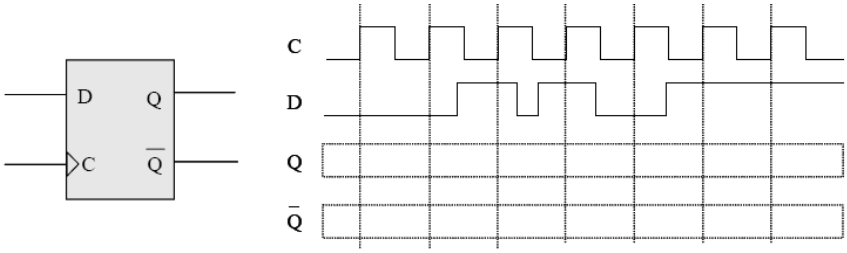


Figura 4.18 - Símbolo do flip-flop D disparado pela borda ascendente.

Exemplo 4.8. Traçar as formas de onda para as saídas do flip-flop que segue, a partir das formas de onda fornecidas.



Além do flip-flop D existe também o flip-flop JK, cujo funcionamento é mostrado na tabela 4.8. Note que seu fun-

cionamento assemelha-se ao do latch RS, exceto que a combinação de entradas (J=1;K=1) não leva a um estado proibido, mas sim à complementação do estado anterior. Da mesma forma que o flip-flop D, esse flip-flop é ativado instantaneamente durante a passagem de uma borda ascendente do sinal de controle. Entre duas bordas ascendentes consecutivas, o flip-flop mantém o estado anterior.

Tabela 4.8 - Tabela de transição de estados para o flip-flop JK disparado pela borda ascendente.

C	J	K	Q_{t+1}	comentário
$\neq\uparrow$	X	X	Q_t	mantém estado anterior
$\uparrow$	0	0	Q_t	mantém estado anterior
$\uparrow$	0	1	0	estado reset
$\uparrow$	1	0	1	estado set
$\uparrow$	1	1	$\overline{Q_t}$	complementa estado anterior

O símbolo do flip-flop JK disparado pela borda ascendente é mostrado na Figura 4.19. Também nesse símbolo, o triângulo na entrada de controle indica que a ativação se dá pela borda ascendente.

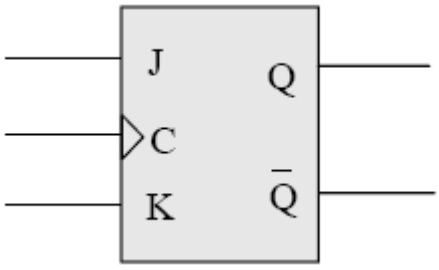
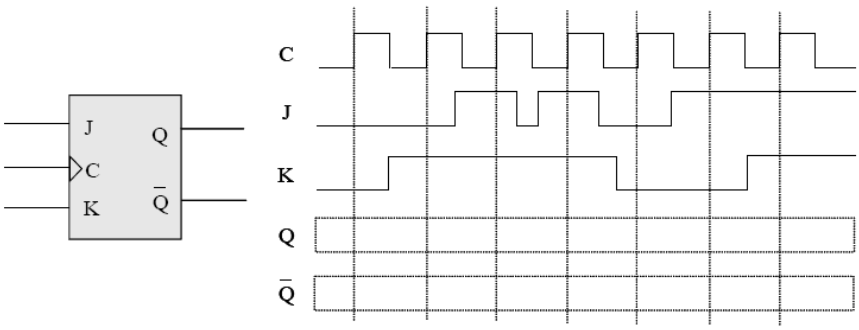


Figura 4.19 - Símbolo do flip-flop JK disparado pela borda ascendente.

Exemplo 4.9: traçar as formas de onda para as saídas do flip-flop JK que segue, a partir das formas de onda fornecidas.



4.4.3 – Flip-flops disparados pela borda descendente

Um flip-flop disparado pela borda descendente é ativado apenas no instante em que o sinal de controle passa pela borda descendente. Nesse instante, o flip-flop amostra os sinais das entradas (D ou J e K), podendo mudar de estado conforme o valor destas entradas. Entre duas bordas descendentes consecutivas, o flip-flop mantém o estado anterior. As tabelas 4.9 e 4.10 mostram o funcionamento do flip-flop D e do flip-flop JK disparados pela borda descendente, respectivamente.

Tabela 4.9 - Tabela de transição de estados para o flip-flop D disparado pela borda descendente.

C	D	Q_{t+1}	comentário
$\neq\downarrow$	X	Q_t	mantém estado anterior
$\downarrow$	0	0	estado reset
$\downarrow$	1	1	estado set

Tabela 4.10 - Tabela de transição de estados para o flip-flop JK disparado pela borda descendente.

C	J	K	Q_{t+1}	comentário
$\neq$	X	X	Q_t	mantém estado anterior
$\downarrow$	0	0	Q_t	mantém estado anterior
$\downarrow$	0	1	0	estado reset
$\downarrow$	1	0	1	estado set
$\downarrow$	1	1	$\overline{Q_t}$	complementa estado anterior

A Figura 4.20 mostra os símbolos do flip-flop D e do flip-flop JK disparados pela borda descendente. Note a existência de um círculo antes da entrada de controle, indicando que os flip-flops são disparados pela borda descendente.

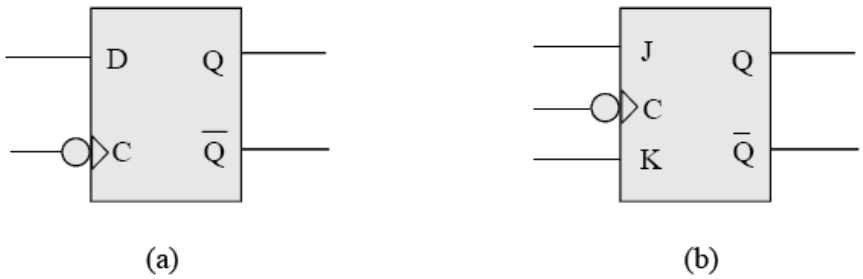
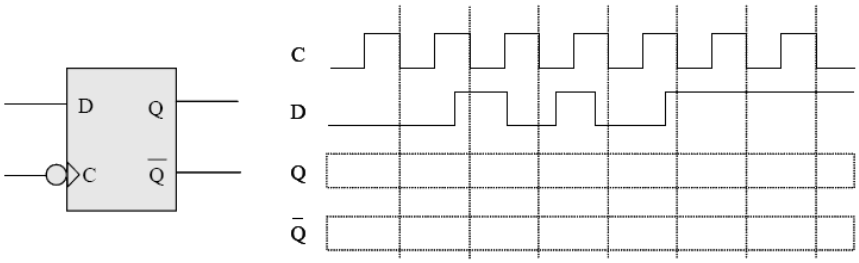


Figura 4.20 - Símbolos para o flip-flop D (a) e para o flip-flop JK (b), ambos disparados pela borda descendente.

Exemplo 4.10: traçar as formas de onda para as saídas do flip-flop D que segue, a partir das formas de onda fornecidas. (Note que o enunciado não diz se o flip-flop é disparado pela borda ascendente ou pela borda descendente, pois essa informação faz parte da interpretação da questão!)



4.4.4 – *Set* e *reset* assíncronos

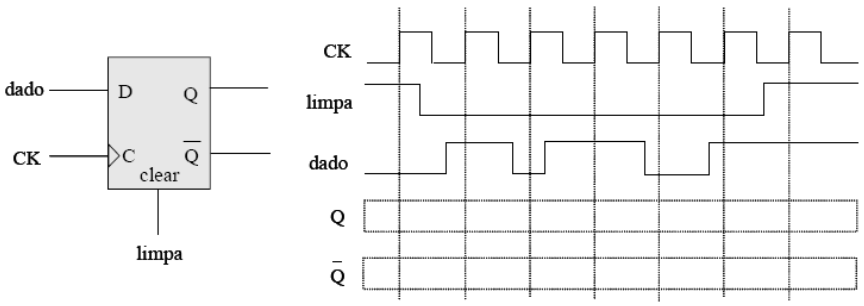
Nos circuitos seqüenciais complexos, muitas vezes é necessário que se possa colocar todos os flip-flops num estado conhecido, o qual pode ser o estado reset (Q=0) ou o estado set (Q=1). Entretanto, todos os flip-flops de um circuito seqüencial síncrono estão sujeitos ao mesmo sinal de controle, que normalmente é o sinal de relógio, de modo que qualquer mudança de estado somente pode ocorrer após uma borda de relógio. Além disso, a operação de "resetar" (i.e., fazer o flip-flop ir para o estado reset) ou "setar" (i.e., fazer o flip-flop ir para o estado set) pode não ser banal.

A fim de permitir que seja possível "resetar" ou "setar" um flip-flop a qualquer tempo, os flip-flops podem ser construídos de modo a possuir um pino de "reset" assíncrono e/ou um pino de "set" assíncrono. A denominação "assíncrono" refere-se ao fato de que a ação deste pino é independente do sinal de controle. Tais pinos são denominados clear (ou DC reset) e preset (ou DC set).

Então, para um flip-flop que tenha o pino de clear (ou DC reset), enquanto este pino estiver ativado, a saída Q do flip-flop estará estável com o valor 0, independente dos valores das demais entradas (incluindo a de controle). De modo similar, para um flip-flop que tenha o pino preset (ou DC set), enquanto este pino estiver ativado, a saída Q do flip-flop estará estável com o valor 1, independente dos valores das demais entradas (incluindo a de controle). Alguns flip-flops podem possuir ambos pinos (clear e preset). Porém, não tem sentido ativar ambos simultaneamente.

A ativação dos pinos clear e preset pode se dar por meio de lógica direta (i.e., nível lógico 1) ou por lógica complementar (i.e., nível lógico 0), o que é possível de ser identificado pelo desenho do flip-flop: caso haja um círculo junto ao pino, a ativação se dá com lógica complementar; caso contrário, a ativação se dá com lógica direta.

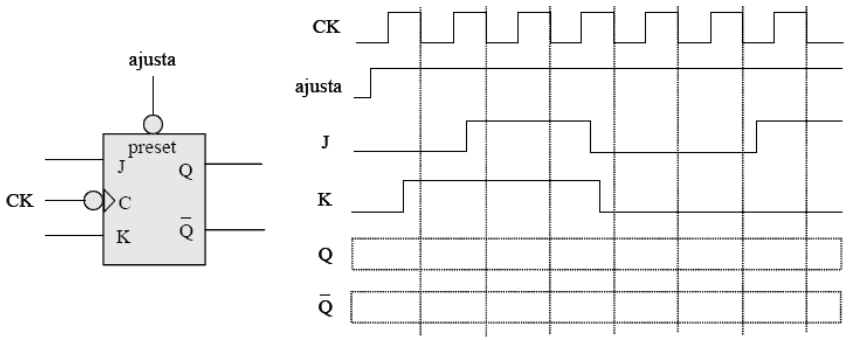
Exemplo 4.11. Traçar as formas de onda para as saídas do flip-flop que segue, a partir das formas de onda fornecidas.



Exemplo 4.12. Traçar as formas de onda para as saídas do flip-flop que segue, a partir das formas de onda fornecidas.

SAIBA MAIS

Existem muitos bons textos e alguns deles estão listados na Bibliografia colocada ao final das unidades. Outros podem ser encontrados na Internet



4.5 – WEB-BIBLIOGRAFIA

www.ufpi.br/uapi (A Página da Universidade Aberta do Piauí - UAPI)

www.uab.gov.br (O Site da Universidade Aberta do Brasil-UAB)

www.seed.mec.gov.br (A Homepage da Secretaria de Educação a Distância do MEC - SEED)

www.abed.org.br (O site da Associação Brasileira de Educação a Distância - ABED)

4.6 – REFERÊNCIAS BIBLIOGRÁFICAS

GAJSKI, Daniel D. Principles of Digital Design, New Jersey: Prentice Hall, 1997 (ISBN 0-13-301144-5)

MANO, M. Morris; Computer Engineering: Hardware Design. New Jersey: Prentice Hall, 1988 (ISBN 0-13-162926-3)

TAUB, H. Circuitos Digitais e Microprocessadores. McGraw-Hill, 1982.

BROWN, Stephen; VRANESIC, Zvonko. Fundamentals of Digital Logic with VHDL Design, McGraw-Hill Higher Education (a McGraw-Hill Company), 2000 (ISBN texto: 0-07-012591-0 CD parte da coleção: 0-07-235596-4)

ERCEGOVAC, Milos; LANG, Tomás; MORENO, Jaime H. Introdução aos Sistemas Digitais. Porto Alegre: Bookman, 2000 (ISBN: 85-7307-698-4)

KATZ, Randy H. Contemporary Logic Design. The Benjamin/Cummings Publishing Company, Inc. , 1994 (ISBN: 0-8053-2703-7)

Unidade 5

ARMAZENAMENTO DE DADOS

Resumo

O objetivo principal desta unidade é apresentar os principais conceitos e circuitos utilizados para armazenar alguns valores, ou seja, as memórias. Estas memórias podem ser os registradores, que são memórias rápidas e estão próximas ao processador do computador, ou também podem ser de outros tipos como apenas de leitura ou de leitura e escrita.

A unidade também contém vários exemplos, e exercícios resolvidos tentando proporcionar ao leitor o entendimento pleno dos conceitos envolvidos, além de serem propostos vários exercícios para sedimentar a teoria apresentada.

A forma de apresentação utilizada é de acordo com o exigido para o ensino à distância, ou seja, tendo em vista sempre esta nova modalidade de ensino.

SUMÁRIO

UNIDADE 5 – ARMAZENAMENTO DE DADOS

5 ARMAZENAMENTO DE DADOS 120
5.1 Introdução 120
5.2 Registradores 120
5.2.1 Registrador com carga paralela 122
5.2.2 Registradores de deslocamento 123
5.2.3 Registrador de deslocamento com sinal de carga paralela 125
5.2.4 Contador assíncrono 126
5.3 Memórias 128
5.3.1 Memória RAM (Random-Access Memory) 129
5.4 SAIBA MAIS 136
5.5 WEB-BIBLIOGRAFIA 137
5.6 REFERÊNCIAS BIBLIOGRÁFICAS 137

5 – ARMAZENAMENTO DE DADOS

5.1 – Introdução

Nos sistemas digitais, e em particular nos computadores, as informações estão representadas por conjuntos de dígitos binários denominados "palavras". Nos computadores atuais o tamanho da palavra é de 32, 64 ou 128 bits. Porém, até há pouco tempo atrás, os computadores pessoais usavam apenas 8 e 16 bits.

Naturalmente, um sistema digital é projetado para trabalhar com um determinado tamanho de palavra, devendo portanto conter recursos de hardware que lhe permitam processar e armazenar simultaneamente conjuntos de n bits, onde n é o tamanho da palavra.

A seguir estudaremos os circuitos digitais responsáveis pelo armazenamento de informação. Alguns destes circuitos são construídos de modo a também poder manipular a informação armazenada. Dentre as operações possíveis estão os deslocamentos (à direita e à esquerda), o incremento e o decremento.

5.2 – Registradores

Um registrador é um circuito digital formado por n flip-flops, de modo a poder armazenar simultaneamente (e de maneira independente) n bits. Trata-se de um tipo de elemento de armazenamento básico: um processador possui um conjunto de registradores que pode variar de três a algumas dezenas. A existência de registradores dentro do processador acelera o processamento, pois os dados que estão sendo manipulados ficam armazenados próximo dos recursos de processamento (ULA, por exemplo), o que reduz os acessos feitos à memória.. A Figura 5.1 mostra um registrador de 4 bits feito com flip-flops D (disparados pela borda ascendente).

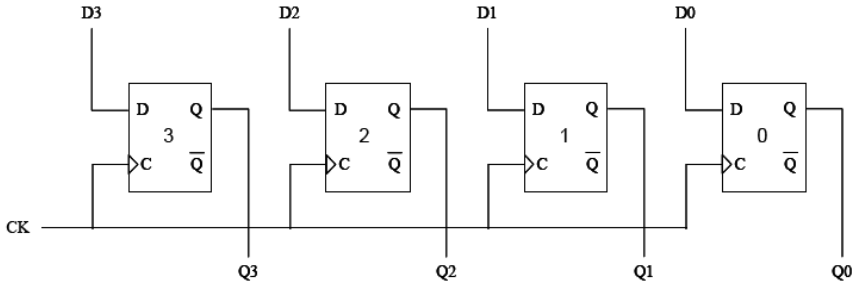


Figura 5.1 - Um registrador de 4 bits, com carga paralela.

Note que cada flip-flop é responsável pelo armazenamento de um bit, seguindo a notação posicional, e que cada bit possui um caminho independente dos demais, tanto para entrada como para a saída. Por isso, o registrador é dito "de carga paralela". Note também que o flip-flop de índice 0 armazena o bit menos significativo e o flip-flop de índice 3 armazena o bit mais significativo de uma palavra de 4 bits. Um registrador funciona como uma barreira: os dados disponíveis nas entradas D0, D1, D2 e D3 somente serão copiados quando o sinal de relógio (CK, no caso) passar por uma borda ascendente. Os valores copiados quando da passagem de uma borda ascendente permanecerão armazenados pelos flip-flops até a ocorrência da próxima borda ascendente. Isto deixa o registrador imune a eventuais mudanças indesejadas dos sinais representados por D0, D1, D2 e D3. O valor armazenado num flip-flop qualquer está sempre presente na sua saída Q. Isto quer dizer que o dado armazenado no registrador pode ser consultado por outro recurso de hardware a qualquer tempo, desde que haja um caminho físico (i.e., um conjunto de fios) entre a saída do registrador e a entrada do outro elemento. O outro elemento pode ser, por exemplo, um somador/subtrator como na Unidade 3.

O registrador da Figura 5.1 apresenta, porém, uma deficiência grave: toda a vez que o sinal de relógio CK passar por uma borda ascendente, os valores das entradas D_0, D_1, D_2 e D_3 serão copiados, mesmo que isso não seja explicitamente desejado. Entretanto, os circuitos de um computador devem seguir as ordens dos sinais provenientes da unidade de controle. O sinal de relógio serve apenas para determinar o momento no qual uma ordem deverá ser cumprida. Logo, um registrador de um computador deve possuir recursos

capazes de realizar a carga do dado (i.e., a carga paralela dos sinais conectados as suas entradas) quanto o relógio passar pela borda ativa somente se o sinal de "carga" (conhecido por "*load*") estiver ativado. A Figura 5.2 mostra um registrador de 4 bits com carga paralela e sinal de carga.

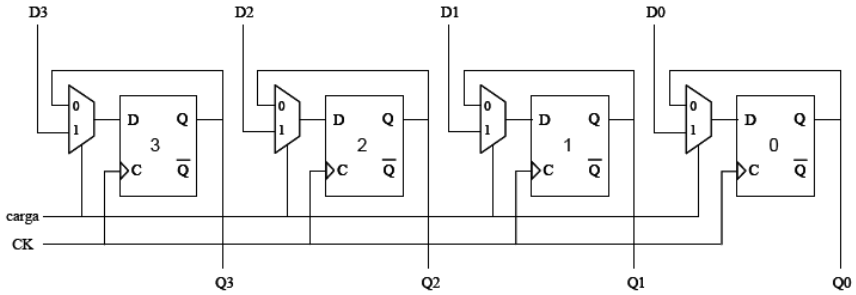


Figura 5.2: - Um registrador de 4 bits, com carga paralela e sinal de carga.

Nos desenhos de circuitos digitais mais complexos, será utilizado preferencialmente o símbolo mostrado na Figura 5.3, ao invés do esquema completo do circuito, a fim de facilitar a compreensão.

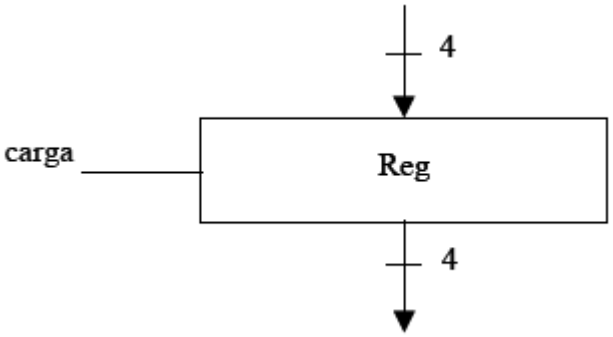


Figura 5.3 - Símbolo para um registrador de 4 bits, com carga paralela e sinal de carga. "Reg" é o nome do registrador.

5.2.1 – Registrador com carga paralela

Exemplo 5.1. Desenhar a forma de onda da saída Q para o circuito que segue.

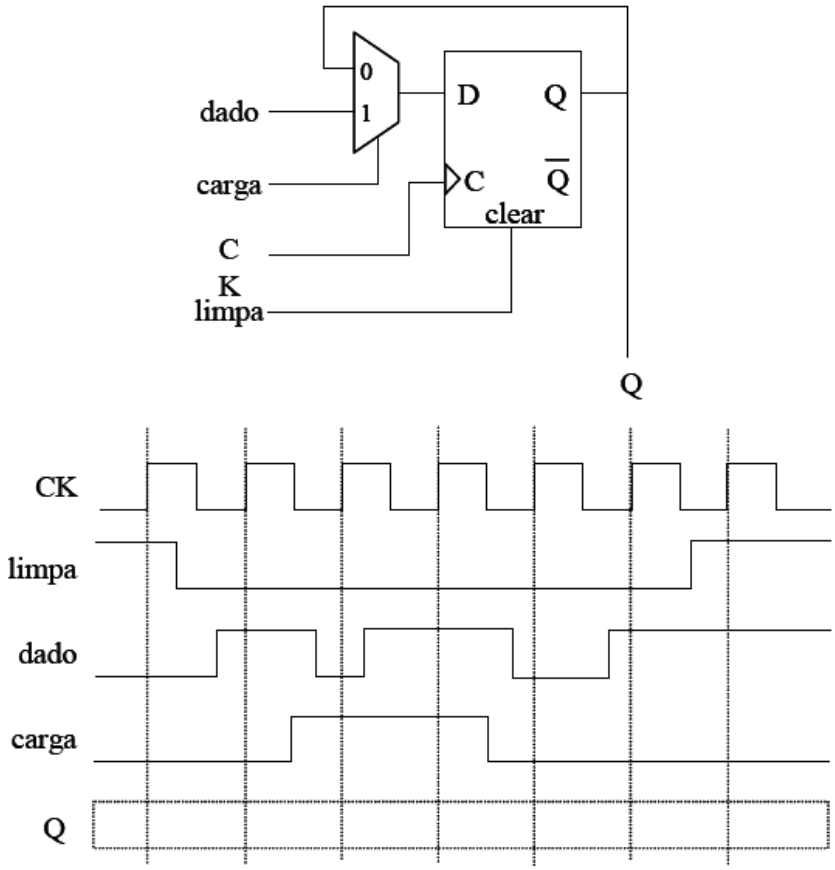


Figura 5.4 - Um bit do registrador com carga paralela.

Este exemplo ilustra o funcionamento do circuito associado a um bit de um registrador de carga paralela com sinal de carga.

5.2.2 Registradores de deslocamento

Uma operação muito importante na aritmética binária é o deslocamento de bits. Essa operação consiste em deslocar o conteúdo de um flip-flop para o seu adjacente. A operação

pode se dar da esquerda para a direita (deslocamento à direita) ou da direita para a esquerda (deslocamento à esquerda).

No primeiro caso, cada flip-flop recebe o conteúdo do seu vizinho imediatamente à esquerda. O flip-flop mais à

esquerda recebe o dado de uma "fonte" externa pela "entrada serial". Já o conteúdo do *flip-flop* mais à direita é descartado.

No segundo caso, cada *flip-flop* recebe o conteúdo do seu vizinho imediatamente à direita. O *flip-flop* mais à direita recebe o dado de uma "fonte" externa pela "entrada serial". Já o conteúdo do flip-flop mais à esquerda é descartado.

Exemplo 5.2. Traçar as formas de onda dos bits armazenados no registrador-deslocador mostrado a seguir, a partir das formas de onde fornecidas.

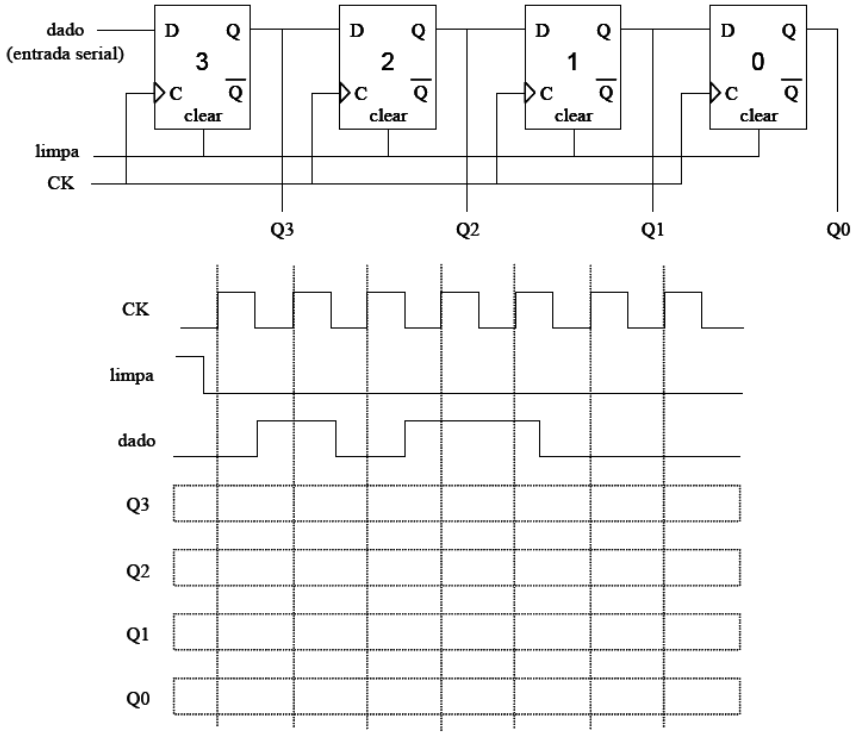


Figura 5.5 - Registrador de deslocamento à direita de 4 bits (com reset assíncrono): exemplo de funcionamento.

Repare que há uma ligação entre a saída de cada flip-flop e a entrada do seu vizinho imediatamente à direita (adjacente a direita). O registrador de deslocamento do exemplo 5.2 não possui sinal de carga. Porém, tal sinal normalmente existe, como será visto mais adiante.

Um registrador de deslocamento à esquerda deve apresentar uma ligação entre a saída de cada flip-flop e a entrada do flip-flop imediatamente à esquerda. Um tal registrador é mostrado na Figura 5.6. Note que a entrada serial está conectada ao *flip-flop* mais à direita (*flip-flop* que armazena o bit menos significativo).

Exemplo 5.3. Traçar as formas de onda dos bits armazenados no registrador-deslocador mostrado a seguir, a partir das formas de onde fornecidas.

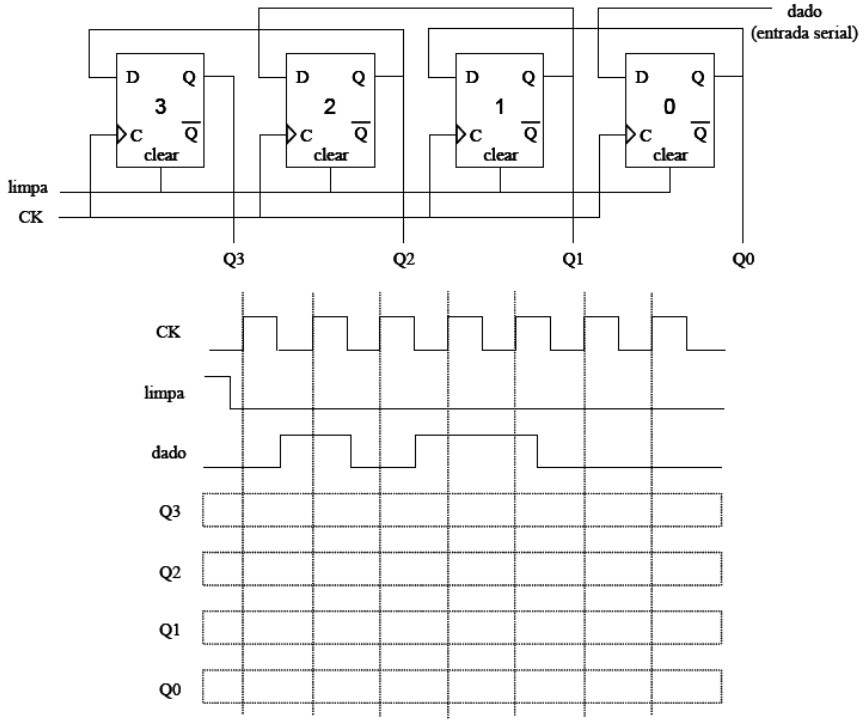


Figura 5.6 - Registrador de deslocamento à esquerda de 4 bits (com reset assíncrono): exemplo de funcionamento.

5.2.3 – Registrador de deslocamento com sinal de carga paralela

Um registrador muito útil é aquele que, além de permitir a carga paralela por meio de sinal de carga, ainda permite deslocamentos à direita e à esquerda. Isso é possível se, na

entrada de cada flip-flop houver um seletor capaz de escolher de onde vem o dado a ser armazenado no flip-flop corrente: de uma fonte externa (no caso de uma carga paralela), da direita, da esquerda (no caso de deslocamento) ou do próprio flip-flop (no caso de simplesmente se querer manter o conteúdo inalterado). Um tal registrador é mostrado na Figura 5.7.

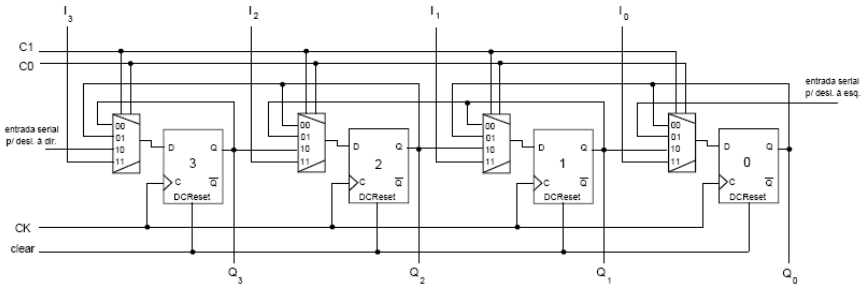


Figura 5.7 - Um registrador-deslocador de 4 bits com sinal de carga e reset assíncrono.

As operações possíveis para o registrador-deslocador (também conhecido como shift-register) da Figura 5.7 são:

1. Carga paralela;
2. Mantém conteúdo;
3. Zera o conteúdo (fazendo-se clear=1);
4. Desloca à direita e desloca à esquerda.

E seu funcionamento se dá como segue:
Se o sinal clear=1, Q3=Q2=Q1=Q0=0;
caso contrário, vale a tabela verdade a seguir

CK	C1	C0	operação
≠↑	X	X	mantém conteúdo
↑	0	0	mantém conteúdo
↑	0	1	desloca à esquerda (shift left)
↑	1	0	desloca à direita (shift right)
↑	1	1	carga paralela

5.2.4 Contador assíncrono

Um contador (ou incrementador) é um registrador que "conta" em binário, ou seja, a cada sinal de relógio, o conte-

údo do registrador é incrementado de uma unidade. Logo, um registrador contador de 4 bits é capaz de contar de 0 (0000) até 15 (1111).

Primeiramente, vejamos como funciona um contador de um bit.

Exemplo 5.4: traçar a forma de onda de Q para o circuito a seguir.

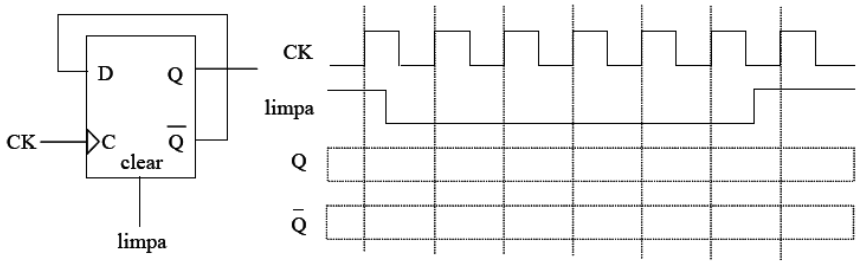
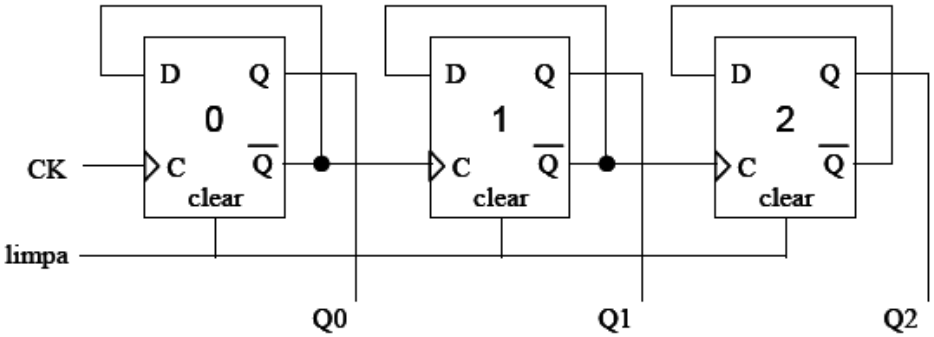


Figura 5.8 - Contador de um bit (com reset assíncrono).

Um circuito contador de mais bits possui uma conexão entre cada flip-flop vizinho, de modo que cada flip-flop de maior ordem é responsável pela ordem de incremento de seu vizinho de menor ordem.

Exemplo 5.5. Encontrar as formas de onda para as saídas dos flip-flops do registrador abaixo.



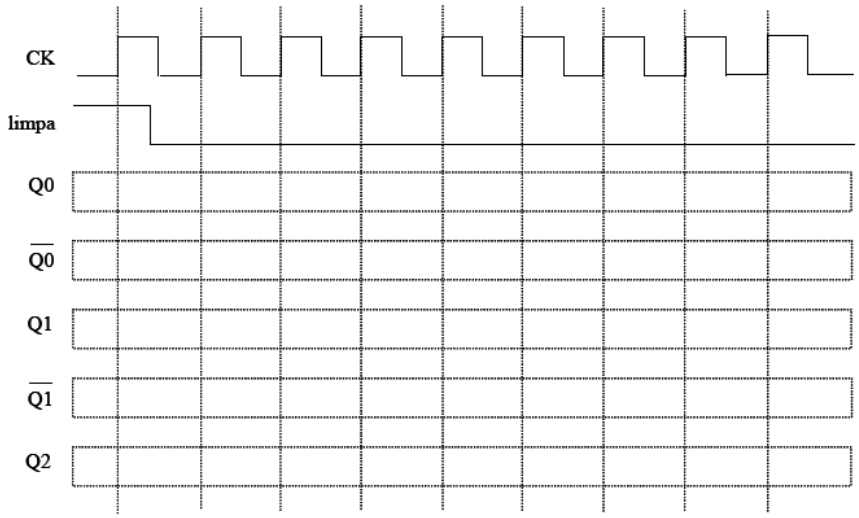


Figura 5.9 - Contador assíncrono de 3 bits (com reset assíncrono).

5.3 – Memórias

Na seção anterior, vimos como são construídos diversos tipos de registradores. Apesar de serem muito rápidos, os registradores têm capacidade de armazenamento reduzidíssima: cada registrador é capaz de armazenar somente uma palavra por vez. Porém, nos sistemas digitais em geral, e particularmente nos computadores, grandes quantidades de informação (palavras) devem poder ser armazenadas. Para tanto, é necessário que o sistema digital possua um conjunto específico de circuitos que sejam mais apropriados ao armazenamento simultâneo de um grande número de palavras. Tais circuitos efetivamente existem e são genericamente denominados de memórias.

Do ponto de vista do funcionamento, as memórias podem permitir apenas a consulta (leitura) ao seu conteúdo ou podem permitir a consulta (leitura) e a modificação (escrita) de seu conteúdo. As memórias que só permitem a leitura são chamadas de ROMs (Read-Only Memories, que significa memórias de leitura apenas), enquanto que as memórias que permitem a leitura e a escrita são genericamente denominadas RAMs (Random-Access Memories, que significa memórias de acesso randômico).

O conteúdo das ROMs pode ser escrito (gravado) quando da fabricação ou mesmo após, por um usuário, que no caso pode ser o fabricante do computador, por exemplo. A característica principal é que uma vez gravadas as informações na ROM, estas não poderão ser modificados, mas somente consultadas (lidas). Já as memórias RAM possuem circuitos capazes de armazenar as informações binárias, as quais podem ser modificadas um número indeterminado de vezes.

Tanto as memórias ROM como as memórias RAM são fabricadas com a tecnologia CMOS, que aliás é a mesma tecnologia de fabricação dos microprocessadores. Porém, o que origina a diferença de funcionamento entre ROMs e RAMs é o tipo de circuitos utilizados. Na seção que segue, veremos detalhes de implementação das memórias RAM, as quais são utilizadas na implementação da memória principal dos computadores.

5.3.1 Memória RAM (Random-Access Memory)

Nesta seção iremos estudar a estrutura física das memórias RAM, isto é, os circuitos que as compõem.

Uma memória RAM é organizada como uma matriz de 2n linhas com m bits armazenados em cada linha, perfazendo um total de 2n x m bits. Em geral, n está entre 16 e 32, enquanto que m pode ser 1, 4, 8, 16 ou 32. A Figura 5.10 ilustra essa organização matricial. Note que existem 2n linhas, também chamadas posições. A cada posição é associado um endereço, iniciando pelo endereço 0 (zero). Portanto, são necessários n bits para decodificar os 2n endereços existentes.

Suponha que se deseje fabricar num único chip (circuito integrado) uma memória RAM capaz de armazenar 2n x m bits, seguindo a organização descrita anteriormente. A Figura 5.11 mostra duas representações gráficas possíveis para um tal chip. Este chip deverá possuir n entradas de endereço (An-1, …, A1,A0), de modo a se poder selecionar

uma (e somente uma) dentre todas as 2n posições existentes na matriz. Este chip também deverá conter m entradas (In-1, …, I1,I0), e m saídas (On-1, …, O1,O0), de modo a permitir a leitura (=consulta) do conteúdo de uma das 2n linhas ou a escrita (=gravação) de uma nova informação numa das 2n linhas da matriz. Como existem duas operações possíveis sobre o conteúdo da matriz (leitura e escrita), é natural que deva existir uma entrada de seleção de operação. Esta entrada será denominada RWS (*Read/Write Select*). Quando RWS=0, a operação a ser realizada será a leitura do conteúdo da posição cujo endereço está presente na entrada de endereços. O valor lido aparecerá na saída do chip. Quando RWS=1, a operação a ser realizada será a escrita da informação binária presente na entrada do chip na linha cujo endereço está presente na entrada de endereço. Por fim, deve existir um sinal de habilitação do chip como um todo (CS - Chip Select). Caso CS=0, o chip está desativado. Caso CS=1, o chip estará realizando a operação especificada pelo valor da entrada RWS.

endereço em binário	endereço em decimal	conteúdo (exemplo)
0…000	0	011…0100
0…001	1	011…0100
0…010	2	101…1100
0…011	3	101…0001
0…100	4	011…0101
0…101	5	111…0110
0…110	6	101…0001
0…111	7	000…1101
	:	:
	:	:
1…110	2^n-2	000…1100
1…111	2^n-1	100…1100

m bits

Figura 5.10 - Organização de uma memória RAM: linhas e endereços.

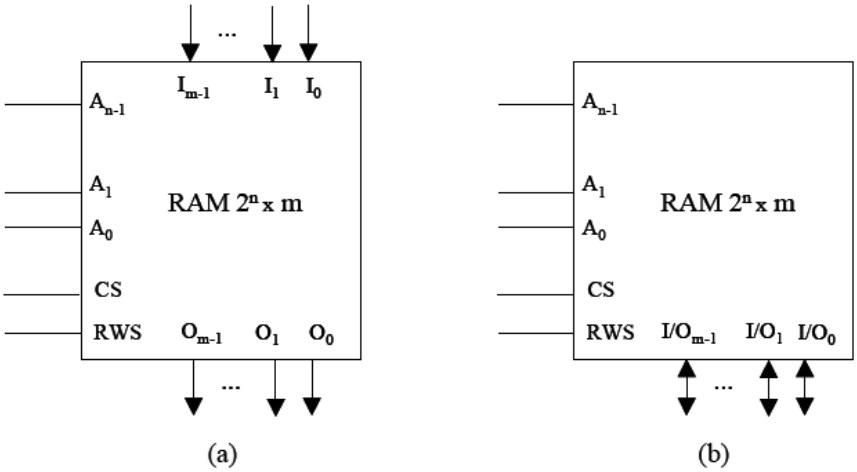


Figura 5.11 - Representações gráficas possíveis para um chip de memória RAM.

O sinal de habilitação do chip, CS, serve para o caso em que seja necessário mais de um chip para implementar a memória do computador.

Quando o número de bits a serem armazenados em cada posição de memória m for pequeno, o chip RAM poderá ter as entradas separadas das saídas, conforme representado pelo símbolo da Figura 5.11a. Entretanto, o mais comum é que um mesmo conjunto de pinos do chip sirvam como entradas e como saídas, situação representada pelo símbolo da Figura 5.11b. A função irá depender da operação, selecionada por meio do pino RWS. Esse uso compartilhado reduz o número de pinos do chip, tornando-o mais barato.

Do ponto de vista estrutural, uma memória RAM é organizada como uma matriz de elementos básicos de memória, buffers de entrada e saída e um decodificador de endereços. Como mostrado na Figura 5.12, uma célula de memória (CM) pode ser representada simbolicamente como sendo constituída por um latch D controlado cuja entrada está conectada a saída de uma porta E e cuja saída se conecta à entrada de um buffer.

Um buffer (*driver)* pode ter duas funções: reforçar o sinal lógico (o que é necessário quando existem muitas portas

conectadas a uma mesma saída ou quando o sinal deve percorrer um fio muito longo) ou servir como uma chave eletrônica que isola fisicamente a saída da entrada (o que serve para disciplinar o acesso de vários sinais a um mesmo fio ou barramento).

Quando o sinal de seleção de linha é igual a 1, o bit armazenado no *latch* passará pelo buffer, ficando disponível na saída da célula. Se o sinal de habilitação de escrita também valer 1, o valor presente na entrada será armazenado no latch. O sinal de habilitação de escrita serve como controle para o latch.

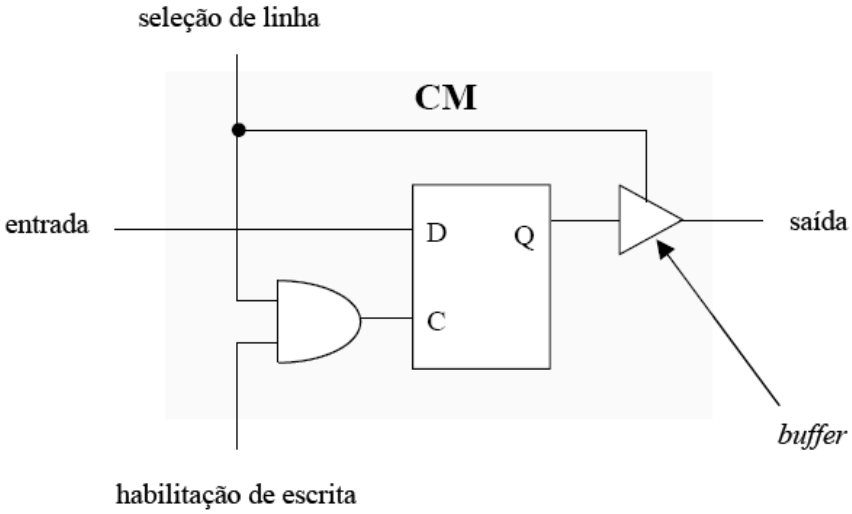


Figura 5.12 - Célula de memória (CM).

A Figura 5.13 mostra um exemplo de memória RAM 4 x 4, constituída por 16 CMs. Para cada acesso à memória, o decodificador de endereços ativa o sinal de seleção de linha associado ao endereço aplicado as suas entradas, o que ativa todos os CMs da linha selecionada. Neste momento, se RWS=1 e CS=1, o novo conjunto de bits será armazenado nas CMs da linha selecionada. Por outro lado, se RWS=0 e CS=1, os bits que estão armazenados na linha selecionada passarão pelos buffers de saída das respectivas CMs e pelos buffers de entrada/saída do chip.

A organização da memória RAM implica em restrições de tempo nas operações de escrita e leitura. Por exemplo,

como o caminho crítico da entrada a saída passa pelo decodificador, as entradas de endereço devem estar estáveis antes de quaisquer outros sinais. Isto significa que durante o ciclo de leitura mostrado pela Figura 5.14 as entradas de endereço deverão ser fornecidas em t0, seguidas por CS em t_1. Assim, os dados da memória estarão disponíveis somente em t_2. O atraso t_2-t_0 é denominado tempo de acesso à memória (memory-access time), enquanto que o tempo t_2-t_1 é denominado tempo de habilitação da saída (*output-enable time*). Note que após os valores das entradas de endereço terem sido modificadas em t_3, os dados ainda estarão disponíveis até t_5. O intervalo t_5-t_3 é denominado tempo de manutenção da saída (*output-hold time*). Já o intervalo t_5-t_4 é denominado tempo de desabilitação da saída (output-disable time). Como o caminho entre as entradas de endereço e as saídas é maior do que o caminho entre CS até as saídas, o tempo de acesso determina a validade dos dados sempre que o endereço e CS forem aplicados ao mesmo tempo. Por outro lado, se o endereço e CS deixarem de ser válidos (CS=0, no caso), o tempo de desabilitação determinará a validade dos dados.

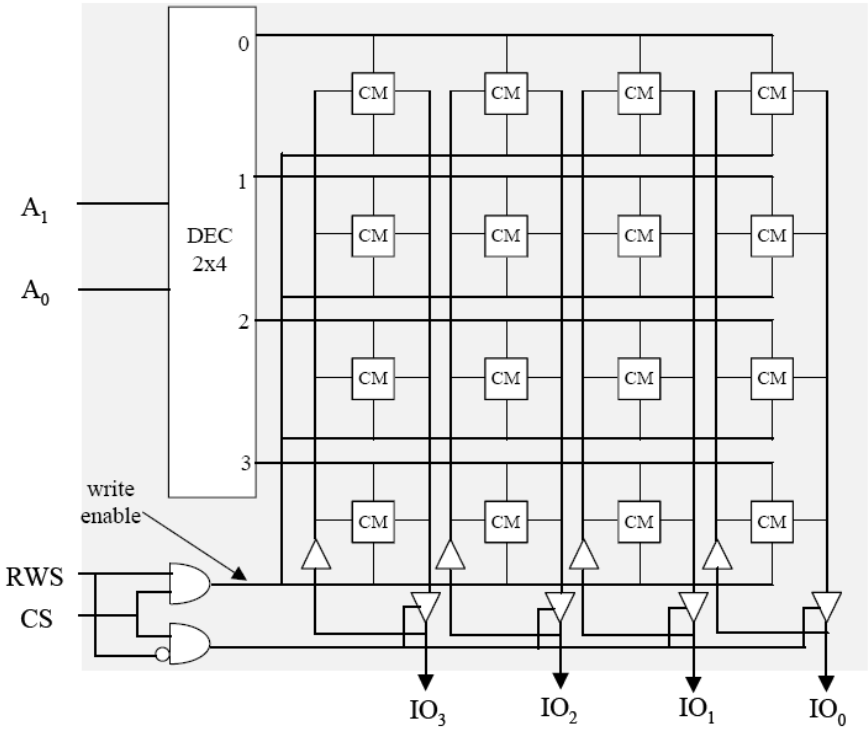


Figura 5.13 - Diagrama de blocos de uma memória RAM

A Figura 5.15 mostra as restrições temporais para o caso de um ciclo de escrita numa memória RAM. No exemplo, foi assumido que CS e RWS foram aplicados simultaneamente, no instante t_1. Como o atraso entre o endereço e a saída é maior do que o atraso entre CS ou RWS e a saída, o endereço deve ser aplicado algum tempo antes, como por exemplo em t_0. O atraso t_1-t_0 é denominado tempo de preparação do endereço (*address setup time*). Como cada CM é feita a partir de um *latch* D controlado com CS fazendo o papel de controle, cada bit do dado na borda de descida de CS (t*3*) ficará armazenado no respectivo *latch*. Entretanto, é necessário que o dado esteja estável por algum tempo antes e depois da borda de descida de CS para garantir a escrita. Na Figura 5.15 esses tempos são anotados como tempo de preparação do dado (data setup time) e tempo de manutenção do dado (*data hold time*), sendo definidos respectivamente como t_3-t_2 e t_4-t_3.

Também é importante ressaltar que CS ou RWS devem ter uma duração igual ou maior que o intervalo de tempo t_3-t_1, o qual é denominado duração do pulso de escrita (*write-pulse- width*). Além disso, o endereço deve estar válido por algum tempo após a borda de descida de RWS ou CS. Este tempo é chamado tempo de manutenção do endereço (*address-hold time)*, e é definido como t_5-t_3.

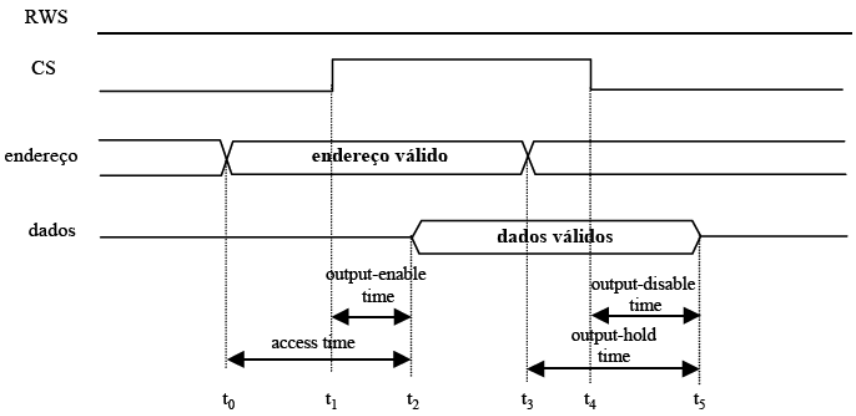


Figura 5.14 - Ciclo de leitura em uma memória RAM.

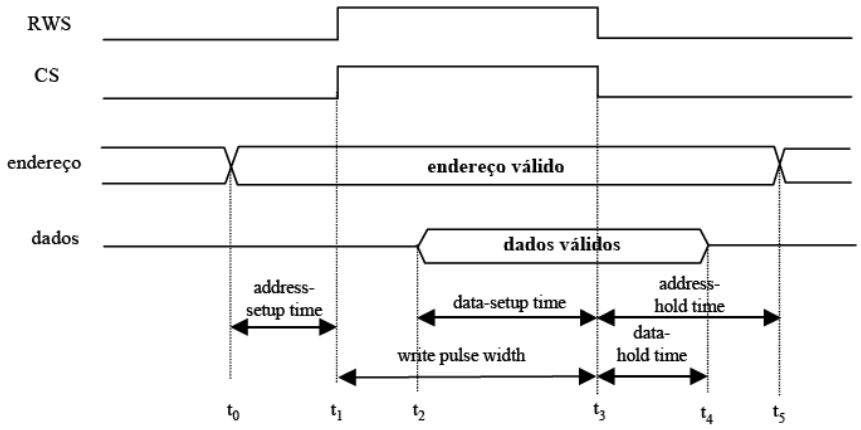


Figura 5.15 - Ciclo de escrita em uma memória RAM.

Apesar da CM ter sido representada como sendo constituída por um latch D e duas portas, na prática sua fabricação pode ser levada a cabo com estruturas mais simples, que utilizam menos transistores. A forma de implementação de CMs leva a classificação das memórias RAM em estáticas e dinâmicas. No caso da RAM estática (conhecida por SRAM, static RAM), a CM é feita com 6 transistores, onde quatro deles formam dois inversores conectados em laço de realimentação, fazendo o papel do latch D. No lugar da porta E e do buffer de saída há um transistor (para cada um, no caso), o qual serve como chave de liga-desliga. A memória SRAM é capaz de manter seu conteúdo por tempo indeterminado, desde que este a alimentação não seja interrompida. No caso da memória dinâmica (conhecida por DRAM, dynamic RAM), cada CM é implementada com somente um transistor. A desvantagem deste tipo de RAM é que o conteúdo da CM é perdido após a operação de leitura, devendo ser reescrito. Para piorar, devido às imperfeições do processo de fabricação, o conteúdo da CM só se mantém por um curto período de tempo. Estes dois problemas são contornados pela utilização de um mecanismo de reflash construído dentro da memória, o qual periodicamente reforça o conteúdo de cada linha de CMs. Durante a operação de reflash, as operações de leitura e escrita são suspensas, o que reduz o desempenho deste tipo de memória.

As memórias DRAM apresentam uma densidade muito grande, o que se traduz em maiores capacidades de armazenamento. Elas também apresentam um custo bem redu-

zido. Devido a estas duas características, as DRAMs são muito utilizadas no projeto de produtos eletrônicos.

Por outro lado, as memórias SRAM são mais caras e apresentam menores capacidades de armazenamento. Porém, são mais velozes do que as memórias DRAM, sendo portanto apropriadas para os casos em que a quantidade de dados a serem armazenados não é grande e uma velocidade maior de operação é necessária.

As memórias SRAM e DRAM são ditas memórias voláteis, pois, uma vez interrompido o fornecimento de energia, elas perdem seu conteúdo. Já as memórias ROM são ditas não voláteis, pois não perdem seu conteúdo quando o fornecimento de energia é interrompido. As memórias ROM tem uma organização semelhante às memórias RAM (matriz de CMs com decodificador de endereço e buffers de saída). Porém, a CM de uma ROM é bem mais simples, normalmente constituída por um único transistor ou diodo, o qual pode ser conFigurado uma vez para permitir o acesso ao 0 lógico (=0V) ou ao 1 lógico (de 5V a 1,5 V, conforme a tecnologia). Nos computadores, as memórias ROM servem para armazenar todas as conFigurações básicas que jamais serão alteradas, como por exemplo, as rotinas de entrada e saída (denominadas de ROM-BIOS, nos computadores tipo PC).

SAIBA MAIS

Existem muitos bons textos e alguns deles estão listados na Bibliografia colocada ao final das unidades. Outros podem ser encontrados na Internet .

5.4 – WEB-BIBLIOGRAFIA

www.ufpi.br/uapi (A Página da Universidade Aberta do Piauí - UAPI)

www.uab.gov.br (O Site da Universidade Aberta do Brasil-UAB)

www.seed.mec.gov.br (A Homepage da Secretaria de Educação a Distância do MEC - SEED)

www.abed.org.br (O site da Associação Brasileira de Educação a Distância - ABED)

5.5 REFERÊNCIAS BIBLIOGRÁFICAS

GAJSKI, Daniel D. Principles of Digital Design, New Jersey: Prentice Hall, 1997 (ISBN 0-13-301144-5)

MANO, M. Morris; Computer Engineering: Hardware Design. New Jersey: Prentice Hall, 1988 (ISBN 0-13-162926-3)

TAUB, H. Circuitos Digitais e Microprocessadores. McGraw-Hill, 1982.

BROWN, Stephen; VRANESIC, Zvonko. Fundamentals of Digital Logic with VHDL Design, McGraw-Hill Higher Education (a McGraw-Hill Company), 2000 (ISBN texto: 0-07-012591-0 CD parte da coleção: 0-07-235596-4)

ERCEGOVAC, Milos; LANG, Tomás; MORENO, Jaime H. Introdução aos Sistemas Digitais. Porto Alegre: Bookman, 2000 (ISBN: 85-7307-698-4)

KATZ, Randy H. Contemporary Logic Design. The Benjamin/Cummings Publishing Company, Inc. , 1994 (ISBN: 0-8053-2703-7)